

A high-contrast, black and white photograph of a massive, curved concrete structure, likely a dam or a large bridge. The structure's surface is composed of large, rectangular panels with visible vertical joints. A single person stands on a narrow walkway along the top edge of the curve, providing a sense of scale. The sky is a uniform, dark grey.

BASICS OF SIP

VOIPENGINEERTRAINING.COM

CORE SIP

WHAT IS SIP?

- ▶ SIP Stands for "Session Information Protocol"
- ▶ It is a Layer 7 (Application Layer) protocol.
- ▶ Really.. it's not a layer 5 protocol, it's layer 7 despite the word "session" in the name.
- ▶ SIP is used to establish, modify and tear down multi media sessions such as internet based phone calls, video calls and instant messages.
- ▶ It can also be used to invite other participants into an existing session.



RFC 3261

- ▶ SIP is defined by RFC 3261.
- ▶ The original SIP RFC was 2543, but has been obsolete since 2002.
- ▶ SIP has many additional RFCs that define further functionality, such as RFC 2833 (DTMF Transport), RFC 3264 (Answer Offer Model) and many more

WHAT IS AN RFC AND WHY DO I CARE

- ▶ A Request for Comments (RFC) is a formal document from the Internet Engineering Task Force (IETF) that is the result of committee drafting and subsequent review by interested parties. Some RFCs are informational in nature. Of those that are intended to become Internet standards, the final version of the RFC becomes the standard and no further comments or changes are permitted. Change can occur, however, through subsequent RFCs that supersede or elaborate on all or parts of previous RFCs.

IN OTHER WORDS...

- ▶ An RFC is a really long document drafted by a bunch of smart people!
- ▶ The smart people have combined their forces to create a document that is full of terms such as SHOULD, MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, RECOMMENDED, MAY, and of course...
OPTIONAL!
- ▶ This document is determined to set rules of standardizations, which will be loosely followed by different vendors.
- ▶ Referring to an RFC in a conversation can and WILL create a heated debate.

BUT REALLY...

- ▶ RFC 3261 is where SIP is defined. You should have it bookmarked.
- ▶ There will be many times during your career in VOIP where you will reference this, then stand your ground when a vendor does something "outside of the RFC".

OVERVIEW OF SIP FUNCTIONALITY

- ▶ There are five facets of establishing and terminating multimedia sessions :
- ▶ **User Location** : determination of the end system to be used for communication
- ▶ **User availability**: determination of the willingness of the called party to engage in communications
- ▶ **User capabilities**: determination of the media and media parameters to be used;
- ▶ **Session setup**: "ringing", establishment of session parameters at both called and calling party;
- ▶ **Session management**: including transfer and termination of sessions, modifying session parameters, and invoking services.

OVERVIEW OF SIP FUNCTIONALITY

- ▶ SIP alone does not provide the media services. It relies on other protocols for that. If you are interacting with SS7 networks the MEGACO protocol is used. In terms of IP networks SIP will rely on the Real Time Protocol, or RTP for transport of media.
- ▶ SIP is based off of the Hyper Text Transport Protocol (HTTP) . It relies on a request / response architecture for delivery of services. We will discuss this in depth later in this course.

OVERVIEW OF SIP FUNCTIONALITY

- ▶ Before we dig into the details of SIP, we need to go over a few terms and definitions that you will hear throughout this course.

AN ADDRESS-OF-RECORD (AOR) IS A SIP OR SIPS URI THAT POINTS TO A DOMAIN WITH A LOCATION SERVICE THAT CAN MAP THE URI TO ANOTHER URI WHERE THE USER MIGHT BE AVAILABLE. TYPICALLY, THE LOCATION SERVICE IS POPULATED THROUGH REGISTRATIONS. AN AOR IS FREQUENTLY THOUGHT OF AS THE "PUBLIC ADDRESS" OF THE USER.

Address of Record

A BACK-TO-BACK USER AGENT (B2BUA) IS A LOGICAL ENTITY THAT RECEIVES A REQUEST AND PROCESSES IT AS A USER AGENT SERVER (UAS). IN ORDER TO DETERMINE HOW THE REQUEST SHOULD BE ANSWERED, IT ACTS AS A USER AGENT CLIENT (UAC) AND GENERATES REQUESTS. UNLIKE A PROXY SERVER, IT MAINTAINS DIALOG STATE AND MUST PARTICIPATE IN ALL REQUESTS SENT ON THE DIALOGS IT HAS ESTABLISHED. SINCE IT IS A CONCATENATION OF A UAC AND UAS, NO EXPLICIT DEFINITIONS ARE NEEDED FOR ITS BEHAVIOR.

Back to Back User Agent

A CALL IS AN INFORMAL TERM THAT REFERS TO SOME COMMUNICATION BETWEEN PEERS, GENERALLY SET UP FOR THE PURPOSES OF A MULTIMEDIA CONVERSATION.

Call

ANOTHER TERM FOR SIP DIALOG – NOT USED IN THE CURRENT RFC

Call Leg

PROXY IS CALL STATEFUL IF IT RETAINS STATE FOR A DIALOG FROM THE INITIATING INVITE TO THE TERMINATING BYE REQUEST. A CALL STATEFUL PROXY IS ALWAYS TRANSACTION STATEFUL, BUT THE CONVERSE IS NOT NECESSARILY TRUE.

Call Stateful

A CLIENT IS ANY NETWORK ELEMENT THAT SENDS SIP REQUESTS AND RECEIVES SIP RESPONSES. CLIENTS MAY OR MAY NOT INTERACT DIRECTLY WITH A HUMAN USER. USER AGENT CLIENTS AND PROXIES ARE CLIENTS.

Client

A DIALOG IS A PEER-TO-PEER SIP RELATIONSHIP BETWEEN TWO UAS THAT PERSISTS FOR SOME TIME. A DIALOG IS ESTABLISHED BY SIP MESSAGES, SUCH AS A 2XX RESPONSE TO AN INVITE REQUEST. A DIALOG IS IDENTIFIED BY A CALL IDENTIFIER, LOCAL TAG, AND A REMOTE TAG. A DIALOG WAS FORMERLY KNOWN AS A CALL LEG IN RFC 2543.

Dialog

A DIRECTION OF MESSAGE FORWARDING WITHIN A TRANSACTION THAT FLOWS FROM THE UAC TO TO THE UAS.

Downstream

A RESPONSE THAT TERMINATES A SIP TRANSACTION, AS OPPOSED TO A PROVISIONAL RESPONSE THAT DOES NOT. ALL 2XX, 3XX, 4XX, 5XX AND 6XX RESPONSES ARE FINAL.

Final Response (code)

A HEADER IS A COMPONENT OF A SIP MESSAGE THAT CONVEYS INFORMATION ABOUT THE MESSAGE. IT IS STRUCTURED AS A SEQUENCE OF HEADER FIELDS.

INVITE sip:bob@biloxi.com SIP/2.0

Via: SIP/2.0/UDP pc33.atlanta.com;branch=z9hG4bK776asdhds

Max-Forwards: 70

To: Bob <sip:bob@biloxi.com>

From: Alice <sip:alice@atlanta.com>;tag=1928301774

SIP Header

A HEADER FIELD IS A COMPONENT OF THE SIP MESSAGE HEADER. A HEADER FIELD CAN APPEAR AS ONE OR MORE HEADER FIELD ROWS. HEADER FIELD ROWS CONSIST OF A HEADER FIELD NAME AND ZERO OR MORE HEADER FIELD VALUES. MULTIPLE HEADER FIELD VALUES ON A GIVEN HEADER FIELD ROW ARE SEPARATED BY COMMAS. SOME HEADER FIELDS CAN ONLY HAVE A SINGLE HEADER FIELD VALUE, AND AS A RESULT, ALWAYS APPEAR AS A SINGLE HEADER FIELD ROW.

SIP Header Field

THE PARTY INITIATING A SESSION (AND DIALOG) WITH AN INVITE REQUEST. A CALLER RETAINS THIS ROLE FROM THE TIME IT SENDS THE INITIAL INVITE THAT ESTABLISHED A DIALOG UNTIL THE TERMINATION OF THAT DIALOG.

Initiator, Calling Party, Caller

A LOCATION SERVICE IS USED BY A SIP REDIRECT OR PROXY SERVER TO OBTAIN INFORMATION ABOUT A CALLEE'S POSSIBLE LOCATION(S). IT CONTAINS A LIST OF BINDINGS OF ADDRESS-OF- RECORD KEYS TO ZERO OR MORE CONTACT ADDRESSES. THE BINDINGS CAN BE CREATED AND REMOVED IN MANY WAYS; THIS SPECIFICATION DEFINES A REGISTER METHOD THAT UPDATES THE BINDINGS.

Location Services

A SIP REQUEST THAT HAS BEEN FORWARDED BY A PROXY, YET THE REQUEST ARRIVES BACK AT THE PROXY THAT FORWARDED THE INITIAL REQUEST.

Loop

A PROXY IS SAID TO BE LOOSE ROUTING IF IT FOLLOWS THE PROCEDURES DEFINED IN THIS SPECIFICATION FOR PROCESSING OF THE ROUTE HEADER FIELD. THESE PROCEDURES SEPARATE THE DESTINATION OF THE REQUEST (PRESENT IN THE REQUEST-URI) FROM THE SET OF PROXIES THAT NEED TO BE VISITED ALONG THE WAY (PRESENT IN THE ROUTE HEADER FIELD). A PROXY COMPLIANT TO THESE MECHANISMS IS ALSO KNOWN AS A LOOSE ROUTER.

Loose Routing

**DATA SENT BETWEEN THE SIP ELEMENTS OF AS PART OF THE PROTOCOL.
A MESSAGE CAN BE A REQUEST OR A RESPONSE.**

Message

THE METHOD IS THE PRIMARY FUNCTION THAT A REQUEST IS MEANT TO INVOKE ON A SERVER. THE METHOD IS CARRIED IN THE REQUEST MESSAGE ITSELF. EXAMPLE METHODS ARE INVITE AND BYE.

Method

A PROXY THAT RECEIVES REQUESTS FROM A CLIENT, EVEN THOUGH IT MAY NOT BE THE SERVER RESOLVED BY THE REQUEST-URI. TYPICALLY, A UA IS MANUALLY CONFIGURED WITH AN OUTBOUND PROXY, OR CAN LEARN ABOUT ONE THROUGH AUTO-CONFIGURATION PROTOCOLS.

Outbound Proxy

A RESPONSE USED BY THE SERVER TO INDICATE PROGRESS, BUT THAT DOES NOT TERMINATE A SIP TRANSACTION. 1XX RESPONSES ARE PROVISIONAL, OTHER RESPONSES ARE CONSIDERED FINAL.

Provisional Response

AN INTERMEDIARY ENTITY THAT ACTS AS BOTH A SERVER AND A CLIENT FOR THE PURPOSE OF MAKING REQUESTS ON BEHALF OF OTHER CLIENTS. A PROXY SERVER PRIMARILY PLAYS THE ROLE OF ROUTING, WHICH MEANS ITS JOB IS TO ENSURE THAT A REQUEST IS SENT TO ANOTHER ENTITY "CLOSER" TO THE TARGETED USER. PROXIES ARE ALSO USEFUL FOR ENFORCING POLICY (FOR EXAMPLE, MAKING SURE A USER IS ALLOWED TO MAKE A CALL). A PROXY INTERPRETS, AND, IF NECESSARY, REWRITES SPECIFIC PARTS OF A REQUEST MESSAGE BEFORE FORWARDING IT.

Proxy / Proxy Server

A CLIENT RECURSES ON A 3XX RESPONSE WHEN IT GENERATES A NEW REQUEST TO ONE OR MORE OF THE URIS IN THE CONTACT HEADER FIELD IN THE RESPONSE.

Recursion

A REDIRECT SERVER IS A USER AGENT SERVER THAT GENERATES 3XX RESPONSES TO REQUESTS IT RECEIVES, DIRECTING THE CLIENT TO CONTACT AN ALTERNATE SET OF URIS.

Redirect Server

A REGISTRAR IS A SERVER THAT ACCEPTS REGISTER REQUESTS AND PLACES THE INFORMATION IT RECEIVES IN THOSE REQUESTS INTO THE LOCATION SERVICE FOR THE DOMAIN IT HANDLES.

Registrar

A TRANSACTION WHERE THE METHOD IS NOT INVITE, ACK OR CANCEL.

Regular Transaction

**A SIP MESSAGE SENT FROM A USER AGENT CLIENT TO A USER AGENT SERVER
THAT INVOKES A CERTAIN OPERATION.**

Request

A SIP MESSAGE SENT FROM A UAS TO A UAC INDICATING THE STATUS OF A REQUEST.

Response (code)

YEAH... YOU KNOW WHEN YOU MAKE A CALL AND BEFORE THE OTHER PERSON ANSWERS YOU HEAR RINGING. THAT'S RINGBACK. IT'S USED TO INFORM YOU THAT THE OTHER PARTY IS BEING ALERTED.

Ringback

A ROUTE SET IS A COLLECTION OF ORDERED SIP OR SIPS URI WHICH REPRESENT A LIST OF PROXIES THAT MUST BE TRAVERSED WHEN SENDING A PARTICULAR REQUEST. A ROUTE SET CAN BE LEARNED, THROUGH HEADERS LIKE RECORD-ROUTE, OR IT CAN BE CONFIGURED.

Route Set

A SERVER IS A NETWORK ELEMENT THAT RECEIVES REQUESTS IN ORDER TO SERVICE THEM AND SENDS BACK RESPONSES TO THOSE REQUESTS. EXAMPLES OF SERVERS ARE PROXIES, USER AGENT SERVERS, REDIRECT SERVERS, AND REGISTRARS.

Server

IN A SEQUENTIAL SEARCH, A PROXY SERVER ATTEMPTS EACH CONTACT ADDRESS IN SEQUENCE, PROCEEDING TO THE NEXT ONE ONLY AFTER THE PREVIOUS HAS GENERATED A FINAL RESPONSE. A 2XX OR 6XX CLASS FINAL RESPONSE ALWAYS TERMINATES A SEQUENTIAL SEARCH.

Sequential Searching

FROM THE SDP SPECIFICATION: "A MULTIMEDIA SESSION IS A SET OF MULTIMEDIA SENDERS AND RECEIVERS AND THE DATA STREAMS FLOWING FROM SENDERS TO RECEIVERS. A MULTIMEDIA CONFERENCE IS AN EXAMPLE OF A MULTIMEDIA SESSION." (RFC 2327 [1]) (A SESSION AS DEFINED FOR SDP CAN COMPRISE ONE OR MORE RTP SESSIONS.) AS DEFINED, A CALLEE CAN BE INVITED SEVERAL TIMES, BY DIFFERENT CALLS, TO THE SAME SESSION. IF SDP IS USED, A SESSION IS DEFINED BY THE CONCATENATION OF THE SDP USER NAME, SESSION ID, NETWORK TYPE, ADDRESS TYPE, AND ADDRESS ELEMENTS IN THE ORIGIN FIELD.

Session

A SIP TRANSACTION OCCURS BETWEEN A CLIENT AND A SERVER AND COMPRISES ALL MESSAGES FROM THE FIRST REQUEST SENT FROM THE CLIENT TO THE SERVER UP TO A FINAL (NON-1XX) RESPONSE SENT FROM THE SERVER TO THE CLIENT. IF THE REQUEST IS INVITE AND THE FINAL RESPONSE IS A NON-2XX, THE TRANSACTION ALSO INCLUDES AN ACK TO THE RESPONSE. THE ACK FOR A 2XX RESPONSE TO AN INVITE REQUEST IS A SEPARATE TRANSACTION.

SIP Transaction

IS A SIP REQUEST THAT IS SENT TO A PROXY, FORWARDED ONWARDS, BUT ARRIVES BACK AT THE PROXY, HOWEVER THE DESTINATION HAS BEEN MODIFIED. THIS TYPICALLY IMPLIES THAT THE REQUEST URI IS LISTED AS SOMETHING OTHER THAN WHAT THE FIRST MESSAGE STATED.

A PRIME EXAMPLE : A CALLS THAT IS FORWARDED.

A LOGICAL ENTITY THAT MAINTAINS THE CLIENT AND SERVER TRANSACTION STATE MACHINES DEFINED BY THIS SPECIFICATION DURING THE PROCESSING OF A REQUEST, ALSO KNOWN AS A TRANSACTION STATEFUL PROXY. THE BEHAVIOR OF A STATEFUL PROXY IS FURTHER DEFINED IN SECTION 16. A (TRANSACTION) STATEFUL PROXY IS NOT THE SAME AS A CALL STATEFUL PROXY.

Stateful Proxy

A LOGICAL ENTITY THAT DOES NOT MAINTAIN THE CLIENT OR SERVER TRANSACTION STATE MACHINES DEFINED IN THIS SPECIFICATION WHEN IT PROCESSES REQUESTS. A STATELESS PROXY FORWARDS EVERY REQUEST IT RECEIVES DOWNSTREAM AND EVERY RESPONSE IT RECEIVES UPSTREAM.

Stateless proxy

A PROXY IS SAID TO BE STRICT ROUTING IF IT FOLLOWS THE ROUTE PROCESSING RULES OF RFC 2543 AND MANY PRIOR WORK INPROGRESS VERSIONS OF THIS RFC. THAT RULE CAUSED PROXIES TO DESTROY THE CONTENTS OF THE REQUEST-URI WHEN A ROUTE HEADER FIELD WAS PRESENT. STRICT ROUTING BEHAVIOR IS NOT USED IN THIS SPECIFICATION, IN FAVOR OF A LOOSE ROUTING BEHAVIOR.

PROXIES THAT PERFORM STRICT ROUTING ARE ALSO KNOWN AS STRICT ROUTERS.

Strict Routing

A DIRECTION OF MESSAGE FLOW WHERE THE MESSAGE FLOWS FROM THE UAS TO THE UAC

Upstream

A USER AGENT CLIENT IS A LOGICAL ENTITY THAT CREATES A NEW REQUEST, AND THEN USES THE CLIENT TRANSACTION STATE MACHINERY TO SEND IT. THE ROLE OF UAC LASTS ONLY FOR THE DURATION OF THAT TRANSACTION. IN OTHER WORDS, IF A PIECE OF SOFTWARE INITIATES A REQUEST, IT ACTS AS A UAC FOR THE DURATION OF THAT TRANSACTION. IF IT RECEIVES A REQUEST LATER, IT ASSUMES THE ROLE OF A USER AGENT SERVER FOR THE PROCESSING OF THAT TRANSACTION. I WILL REFER TO THIS AS A “UAC”

User Agent Client

A USER AGENT SERVER IS A LOGICAL ENTITY THAT GENERATES A RESPONSE TO A SIP REQUEST. THE RESPONSE ACCEPTS, REJECTS, OR REDIRECTS THE REQUEST. THIS ROLE LASTS ONLY FOR THE DURATION OF THAT TRANSACTION. IN OTHER WORDS, IF A PIECE OF SOFTWARE RESPONDS TO A REQUEST, IT ACTS AS A UAS FOR THE DURATION OF THAT TRANSACTION. IF IT GENERATES A REQUEST LATER, IT ASSUMES THE ROLE OF A USER AGENT CLIENT FOR THE PROCESSING OF THAT TRANSACTION.
I WILL OFTEN REFER TO THIS IS A “UAS”

User Agent Server

A LOGICAL ENTITY THAT CAN ACT AS BOTH A USER AGENT CLIENT AND USER AGENT SERVER.

User Agent

CLIENTS AND SERVERS

CLIENTS AND SERVERS

- ▶ SIP endpoints are known as User-Agents. In the most basic form, there are two types of user agents the required to setup a call.

USER AGENT CLIENT

- ▶ User Agent Client (UAC): A user agent client is a logical entity that creates a new request, and then uses the client transaction state machinery to send it. The role of UAC lasts only for the duration of that transaction. In other words, if a piece of software initiates a request, it acts as a UAC for the duration of that transaction. If it receives a request later, it assumes the role of a user agent server for the processing of that transaction.



USER AGENT CLIENTS

- ▶ Imagine this. You see your friend across the room. As you approach, you say "Hey Dan, want to go to a movie".
- ▶ In this scenario, you are acting as the UAC, because you initiated the request. Until Dan decides to initiate a request to you, you remain the UAC in this conversation.



USER AGENT SERVER :

- ▶ A user agent server is a logical entity that generates a response to a SIP request. The response accepts, rejects, or redirects the request. This role lasts only for the duration of that transaction. In other words, if a piece of software responds to a request, it acts as a UAS for the duration of that transaction. If it generates a request later, it assumes the role of a user agent client for the processing of that transaction.



RFC Definition

USER AGENT SERVER

- ▶ A moment ago you ask Dan if he wanted to see a movie. He received your request. In this situation, Dan is acting as the User Agent Server. Of course.. Dan should reply to the request that you sent.
- ▶ Dan simply declines your request by saying "No".
- ▶ Dan doesn't really like you.
- ▶ You steal his popcorn.



BACK TO BACK USER AGENT

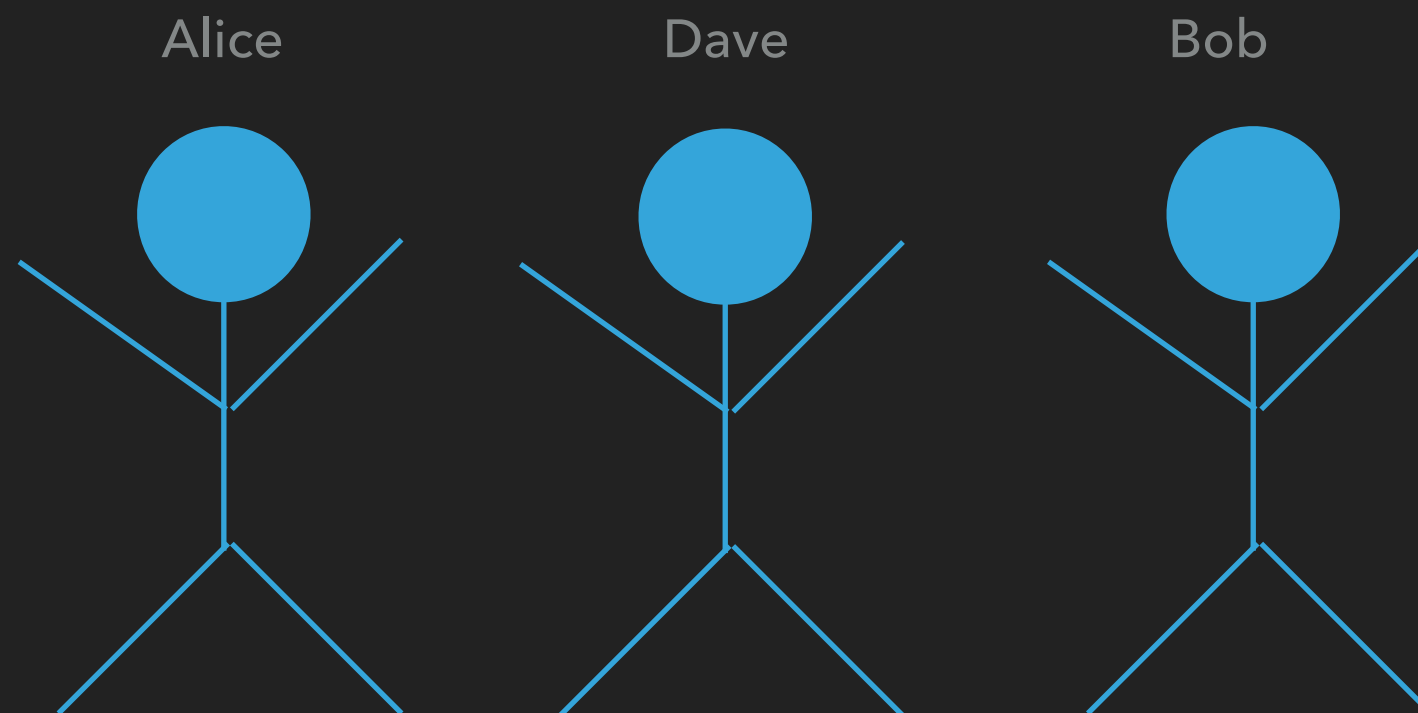
- ▶ A back-to-back user agent (B2BUA) is a logical entity that receives a request and processes it as a user agent server (UAS). In order to determine how the request should be answered, it acts as a user agent client (UAC) and generates requests. Unlike a proxy server, it maintains dialog state and must participate in all requests sent on the dialogs it has established. Since it is a concatenation of a UAC and UAS, no explicit definitions are needed for its behavior.



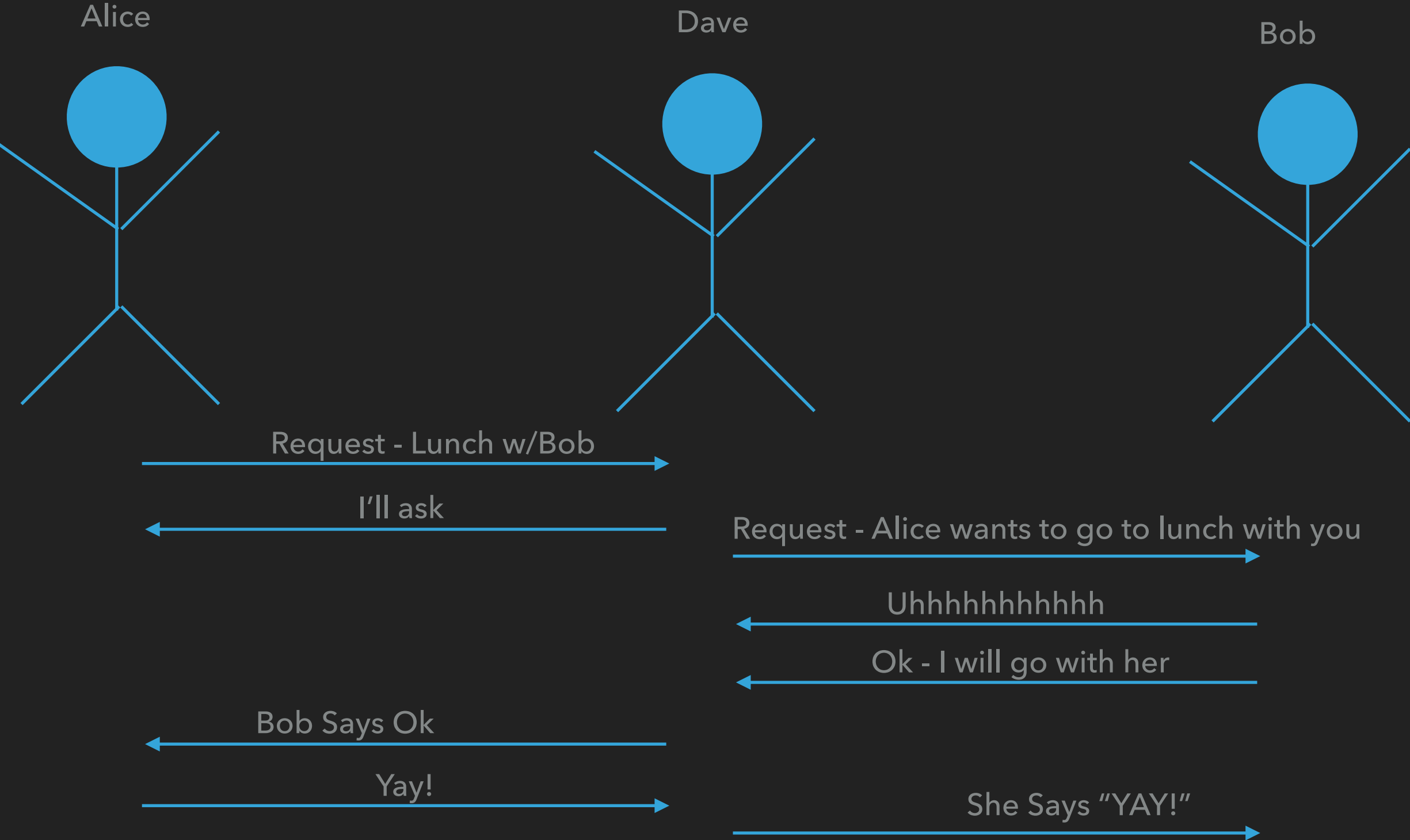
RFC Definition

BACK TO BACK USER AGENT

- ▶ Now we have Alice that want to go to lunch. She walks over and asks Dave if Bob wants to go to lunch.
- ▶ Clearly Dave is not Bob, so Dave Decides to ask Bob on behalf of Alice

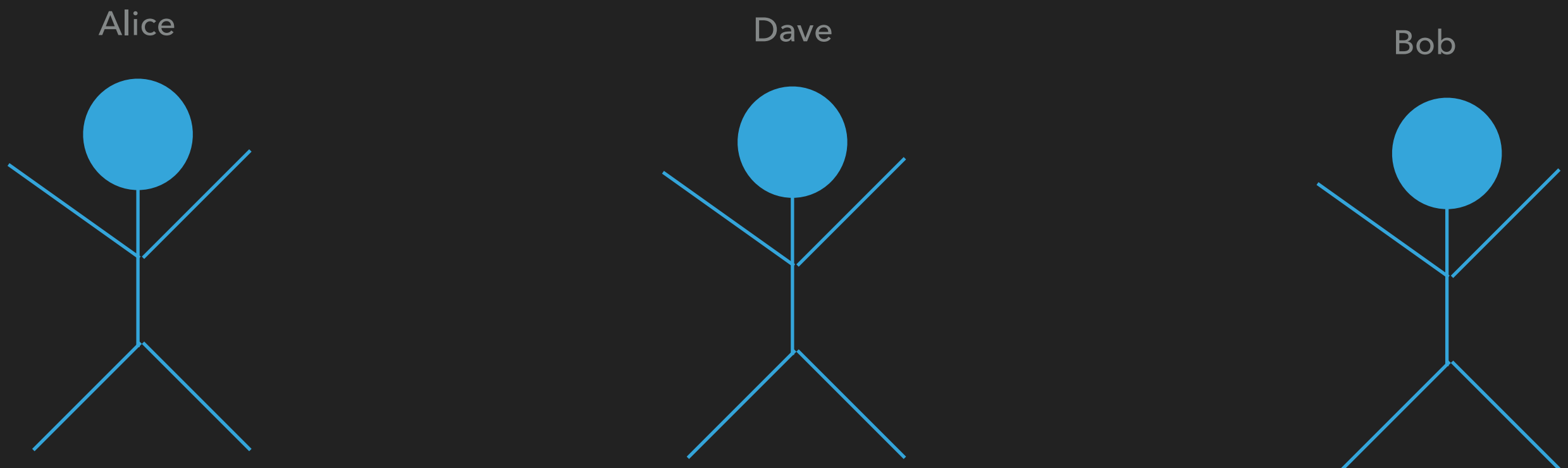


BACK TO BACK USER AGENT



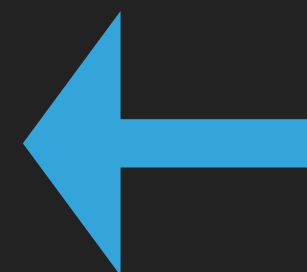
BACK TO BACK USER AGENT

- ▶ See what happened? Dave acted as the back to back user agent in this scenario. He received the request from Alice, then created a new request toward Bob. Dave also remained in the conversation and kept track of what was going on.



PROXY

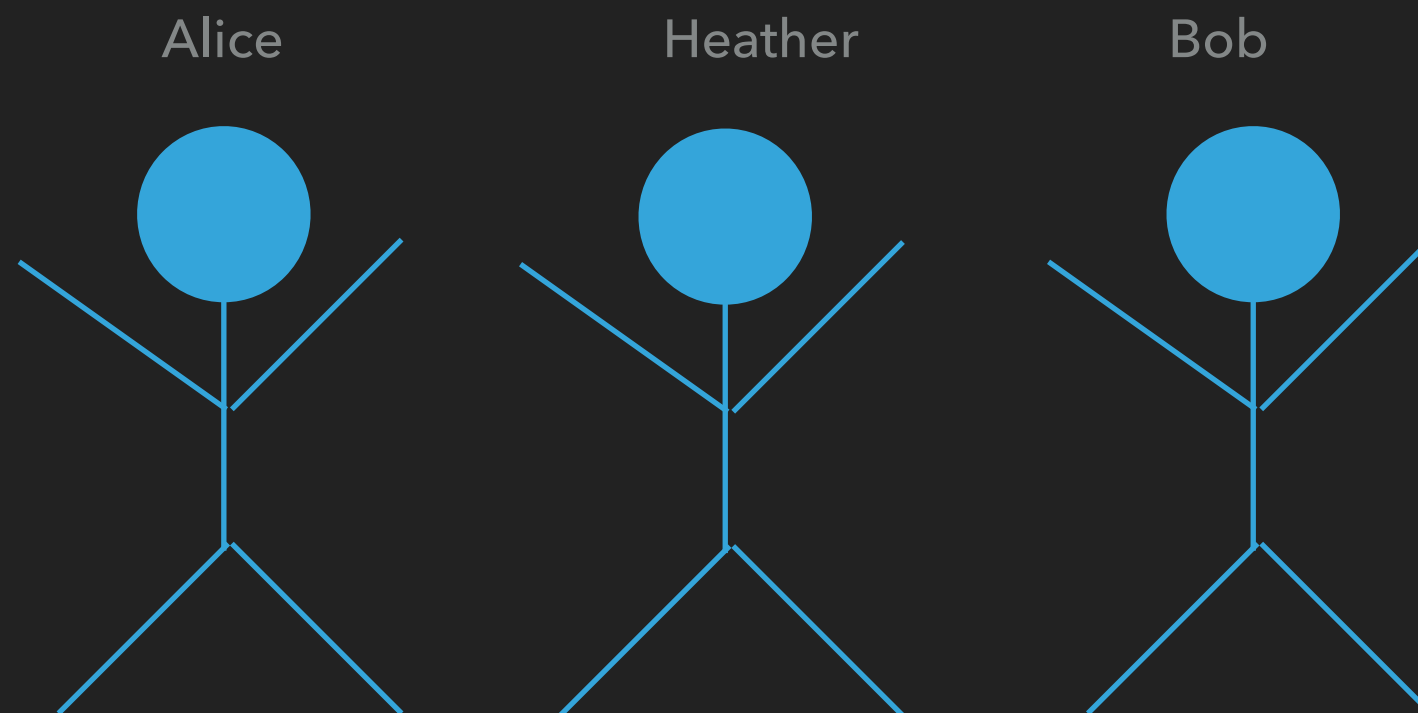
- ▶ Proxy Server: An intermediary entity that acts as both a server and a client for the purpose of making requests on behalf of other clients. A proxy server primarily plays the role of routing, which means its job is to ensure that a request is sent to another entity "closer" to the targeted user. Proxies are also useful for enforcing policy (for example, making sure a user is allowed to make a call). A proxy interprets, and, if necessary, rewrites specific parts of a request message before forwarding it.



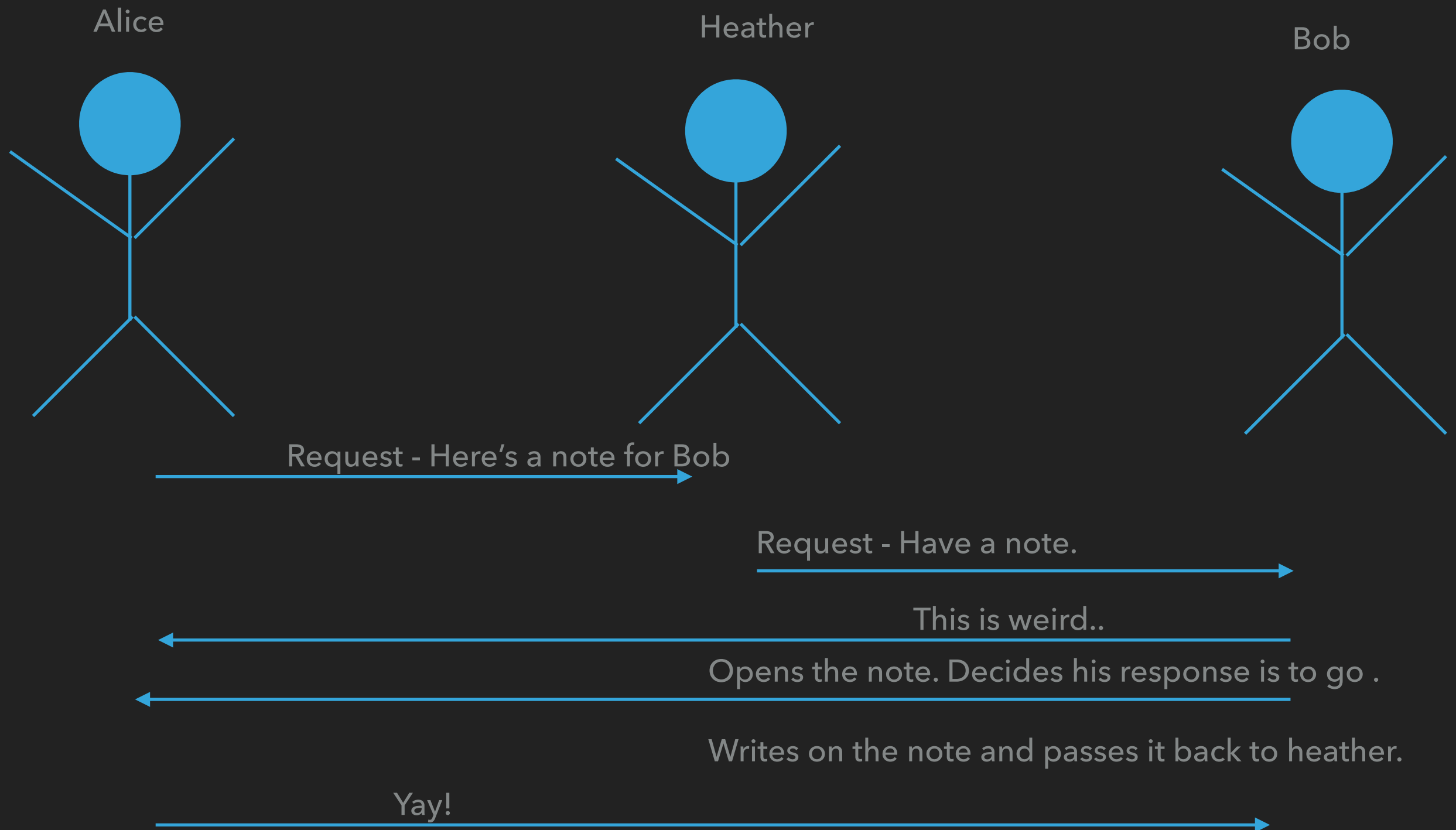
RFC Definition

PROXY

- ▶ Now we have Alice that want to go to lunch. She walks over to Heather and passes her a note. This note says "To Bob"
- ▶ Heather sees that Bob is next to her and passes him the note.



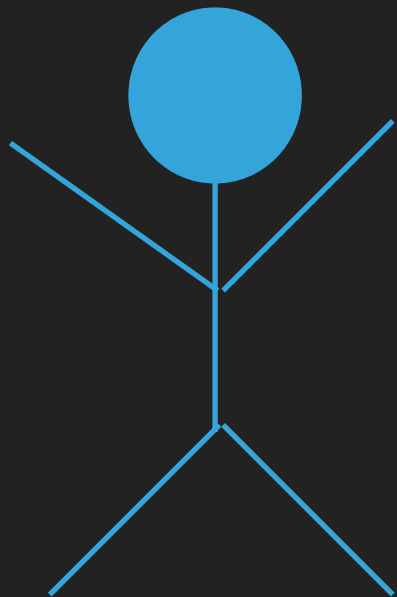
PROXY



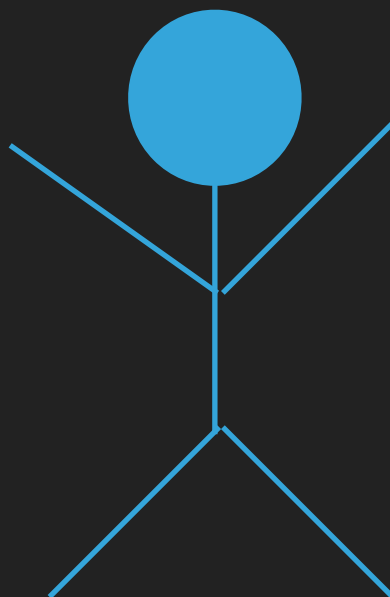
PROXY

- ▶ See what happened? Heather routed the note to Bob. Bob told Heather that he received it, but Heather didn't care what happened after that. She let Bob and Alice do their own thing without her involved.

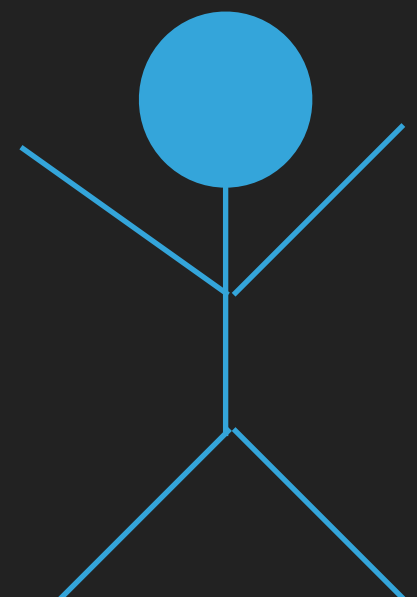
Alice



Heather



Bob



SIP MESSAGES

SIP MESSAGES

- ▶ As seen in the earlier definitions, a SIP Message can be a SIP Request or Response.
- ▶ SIP has a set of rules that must be followed for all messages.
- ▶ A generic message looks like this :

Start line:

Message Headers

Empty Line

Message Body

SIP REQUESTS

- ▶ A SIP Request is used to establish, modify, or terminate a session. It is generated by the UAC and sent downstream toward the UAS.
- ▶ A SIP request is distinguished from a response code, because the start line, contains and request-line rather than a status-line.



SIP REQUESTS

- ▶ Lets break this down a bit more. As this is a request, the start line will present the Request Line
 - ▶ The Request Line will contain a method name (INVITE in this case), a request uri, and the protocol version, separated by a single space character.
- ▶ Below that (red box) we have the message headers.
 - ▶ Via, Max Forward, From, To, CSeq, Contact, Content-Type, and Content-Length are the headers in this example
- ▶ Then we have an empty line that separates the headers from the body.
- ▶ Finally, we have the message body. In this example the Content-Type is "application / SDP" , so the message body is an SDP here.

```
INVITE sip:bob@biloxi.example.com SIP/2.0
```

```
Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0

o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.0.2.101
t=0 0

m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

SIP REQUEST METHODS

- ▶ RFC 3261 defines six methods :
 - ▶ REGISTER: Used to register contact information
 - ▶ INVITE : Setting up a session
 - ▶ ACK : Acknowledge and start the party
 - ▶ CANCEL : For canceling a session before the other party answers.
 - ▶ BYE: Tear down the session (Hangup)
 - ▶ OPTIONS : Used to query the UAS to determine capabilities. Also used as a "ping" method

SIP REQUEST METHODS

- ▶ There are other RFCs that define additional methods.
 - ▶ RFC 3265 :
 - ▶ SUBSCRIBE : I want to subscribe to a resource, such as when someone is on or off hook.
 - ▶ NOTIFY: Used to notify the subscribed party that an event they care about has occurred.
 - ▶ RFC 3515:
 - ▶ REFER : This is used in many call transfer scenarios.

SIP REQUEST METHODS

- ▶ RFC 3262 :
 - ▶ PRACK : Used to keep track of provisional responses
- ▶ RFC 3903 :
 - ▶ Publish : Used to Publish an event to the UAS
- ▶ RFC 6086 :
 - ▶ INFO : Sends mid-session information that does not modify the session state

SIP REQUEST METHODS

- ▶ RFC 3428 :
 - ▶ MESSAGE: Used for Instant Message Applications
- ▶ RFC 3311 :
 - ▶ UPDATE : Modifies the state of the session, without changing the state of the dialog

SIP RESPONSE CODES

SIP RESPONSE CODES

- ▶ A SIP Response is generated by the UAS to inform the UAC of the transaction state.
- ▶ A SIP response is distinguished from a request , because the start line, contains a status-line instead of a request-line.

SIP RESPONSES

- ▶ Here we have an example of a SIP response. Notice the Start-Line is different here. Instead of a Request-Line, we have a status line.
 - ▶ The status-line will present the protocol version, a three digit status code (180 in this example), and an associated text description of the status code which is known as the Reason-Phrase.
 - ▶ The status code required and will be interpreted by the UAC's SIP stack.
 - ▶ The reason-phrase... it could be whatever is set by the UAS SIP stack.
 - ▶ The status-line could say "SIP/2.0 180 POTATO", and it will still be understood by the UAC's SIP stack.
- ▶ Again, we have the sip headers.
 - ▶ In this example : via, from, to, call-id, cseq, contact, and content-length are the headers.

SIP/2.0 180 Ringing

```
Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74b1
;received=192.0.2.101
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>;tag=8321234356
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:bob@client.biloxi.example.com;transport=tcp>
Content-Length: 0
```

SIP RESPONSES

- ▶ Where's the Body?
 - ▶ Notice the content-length is "0"? Due to that, we do not have a body specified. While this message does not have a body, some do.
 - ▶ However... The blank line followed by a CRLF is still there under the Content-Length Header.

SIP/2.0 180 Ringing

```
Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74btf
;received=192.0.2.101
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>;tag=8321234356
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:bob@client.biloxi.example.com;transport=tcp>
Content-Length: 0
```

1XX PROVISIONAL RESPONSES

Provisional responses, also known as informational responses indicate that the server contacted is performing some further action and does not yet have a definitive response. A server sends a 1xx response if it expects to take more than 200 ms to obtain a final response. Note that 1xx responses are not transmitted reliably. They never cause the client to send an ACK.

Provisional (1xx) responses MAY contain message bodies, including session descriptions.

100 TRYING

1xx Provisional Responses This response indicates that the request has been received by the next-hop server and that some unspecified action is being taken on behalf of this call (for example, a database is being consulted).

This response, like all other provisional responses, stops retransmissions of an INVITE by a UAC. The 100 (Trying) response is different from other provisional responses, in that it is never forwarded upstream by a stateful proxy.

180 RINGING

This implies that the UAS received the INVITE Request and that the UAC should initiate local ringback while further call processing takes place.

181 CALL IS BEING FORWARDED

A server MAY use this status code to indicate that the call is being forwarded to a different set of destinations.

182 QUEUED

The called party is temporarily unavailable, but the server has decided to queue the call rather than reject it. When the callee becomes available, it will return the appropriate final status response. The reason phrase MAY give further details about the status of the call, for example, "5 calls queued; expected waitingtime is 15 minutes". The server MAY issue several 182 (Queued) responses to update the caller about the status of the queued call.

183 SESSION IN PROGRESS

The 183 (Session Progress) response is used to convey information about the progress of the call that is not otherwise classified. The Reason-Phrase, header fields, or message body MAY be used to convey more details about the call progress.

Typically - the 183 response will contain an SDP and establish what is known as an "early media session". This implies that audio (or video) is being sent before the call is actually answered.

Example : In the early 2000's many cellular providers started to offer "custom ring back tones". When you called someone, it could play a song, instead of the normal ring tone.

183 SESSION IN PROGRESS

- ▶ Early media is also used by some companies to play auto attendant / IVR messages before the call is actually answered. UPS and FedEx utilize this.. but why?
- ▶ Well for starters a phone company doesn't charge for usage until a call is truly answered with a final response. If a company is able to "queue" a call before billing begins, how many millions of minutes can they bypass each year?
- ▶ Fraudsters also use this trick. They will call an international destination where the call is never "answered" yet they are able to have a 5-10 minute conversation with the other party before the originating carriers kills the call for having a long setup time.

2XX SUCCESSFUL RESPONSE

- ▶ There is only one 2xx response : the 200 OK
- ▶ The 200 OK implies that the request was successful. In terms of a phone call, when the called party picks up their handset, their IP phone will send a 200 OK upstream.

3XX REDIRECTIONS

- ▶ Sometimes a call can be completed in more than one way. A Prime example is having multiple paths to the PSTN, or an alternate method to reach a subscribed user.

300 MULTIPLE CHOICES

- ▶ The address in the request resolved to several choices, each with its own specific location, and the user (or UA) can select a preferred communication end point and redirect its request to that location.
- ▶ The response MAY include a message body containing a list of resource characteristics and location(s) from which the user or UA can choose the one most appropriate, if allowed by the Accept request header field. However, no MIME types have been defined for this message body.

301 MOVED PERMANENTLY

- ▶ The user can no longer be found at the address in the Request-URI, and the requesting client SHOULD retry at the new address given by the Contact header field (Section 20.10). The requestor SHOULD update any local directories, address books, and user location caches with this new value and redirect future requests to the address(es) listed.
- ▶ Have you ever moved and notified your postal service that you no longer live at a certain address? Sometimes the postal service will reject delivery of new mail, and send it back to the sender stating that you have moved. When they do this, the postal service lets the sender know your new address

302 TEMPORARILY MOVED

- ▶ The requesting client SHOULD retry the request at the new address(es) given by the Contact header field (Section 20.10). The Request-URI of the new request uses the value of the Contact header field in the response.
- ▶ The duration of the validity of the Contact URI can be indicated through an Expires (Section 20.19) header field or an expires parameter in the Contact header field. Both proxies and UAs MAY cache this URI for the duration of the expiration time. If there is no explicit expiration time, the address is only valid once for recursing, and MUST NOT be cached for future transactions.

380 ALTERNATIVE SERVICE

- ▶ The call was not successful, but alternative services are possible.
- ▶ The alternative services are described in the message body of the response. Formats for such bodies are not defined here, and may be the subject of future standardization.

4XX REQUEST FAILURES

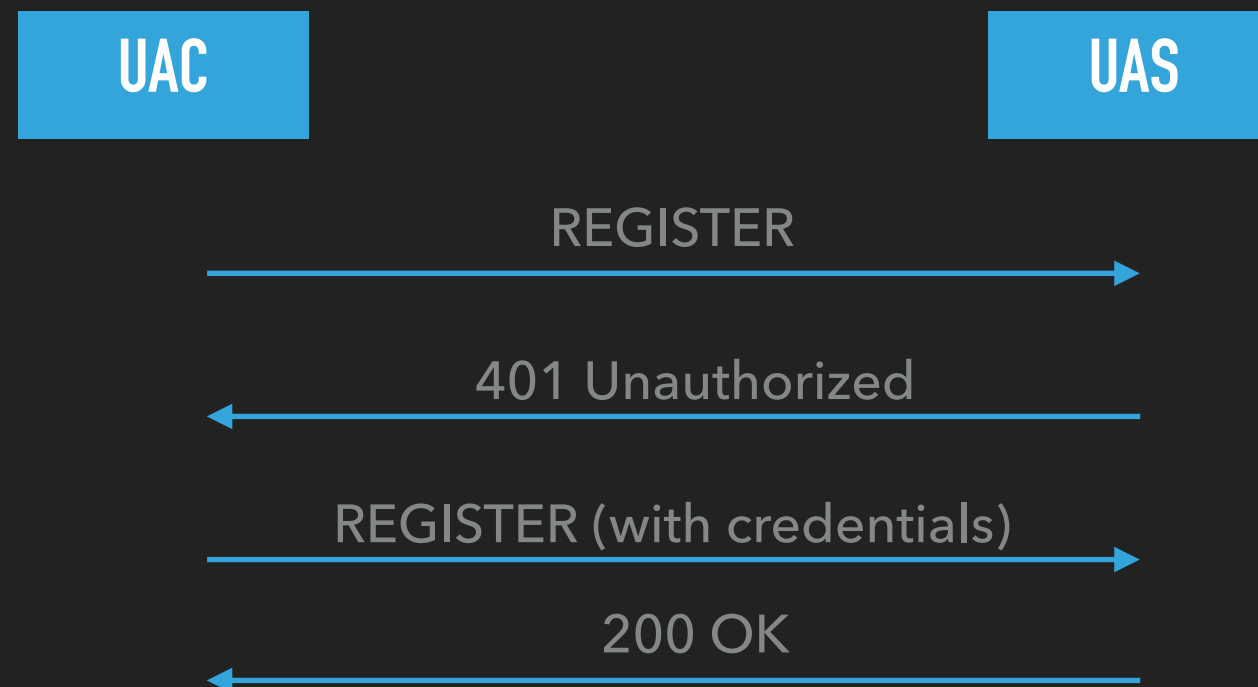
- ▶ 4xx responses are definite failure responses from a particular server. The client **SHOULD NOT** retry the same request without modification (for example, adding appropriate authorization).
- ▶ However, the same request to a different server might be successful.

400 BAD REQUEST

- ▶ Have you been around a friend that says something that makes absolutely no sense, yet you want to let him or her know? That's similar to the 400 Bad Request.
- ▶ Basically the UAS cannot understand the request, due to a malformed syntax. The Reason-Phrase SHOULD define what is wrong so the UAC can fix the glitch.
- ▶ This is often seen when "fuzzing" a SIP stack. Fuzzing is the act of sending malformed data to a server in an attempt to overload, or compromise a security weakness.

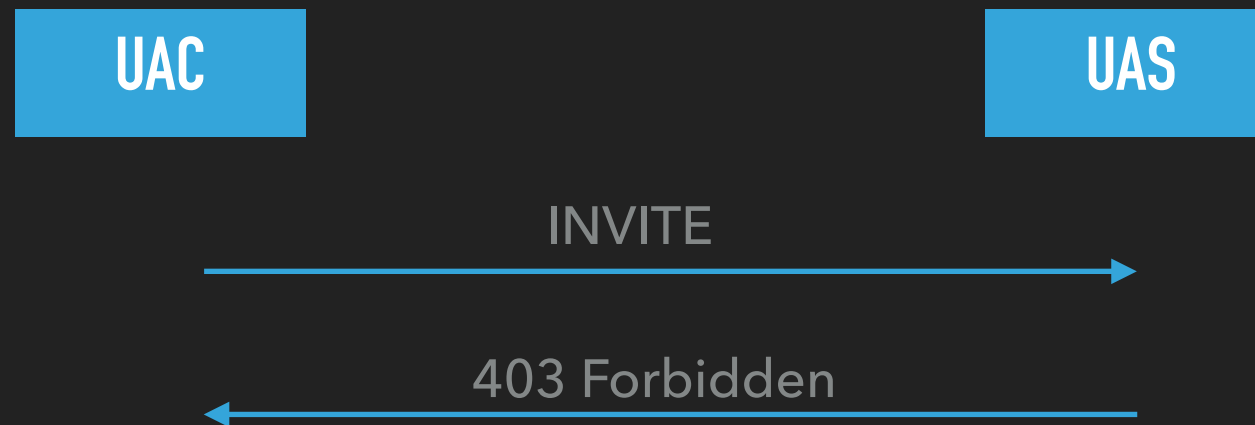
401 UNAUTHORIZED (OR 401 PROXY REQUIRES AUTHENTICATION)

- ▶ This states that the UAS wants the UAC to authenticate. A common scenario for this is when a UAC is attempting to register.
- ▶ It can also be used for other Methods



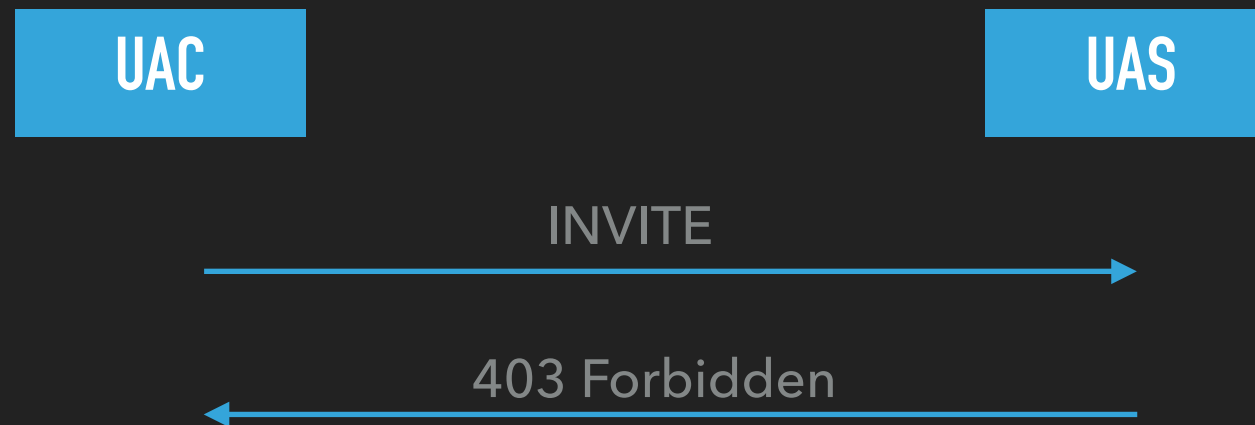
403 FORBIDDEN

- ▶ This implies that the UAS understood the request, but refuses to fulfill it. Authorization will do nothing, and the request **SHOULD NOT** be repeated.



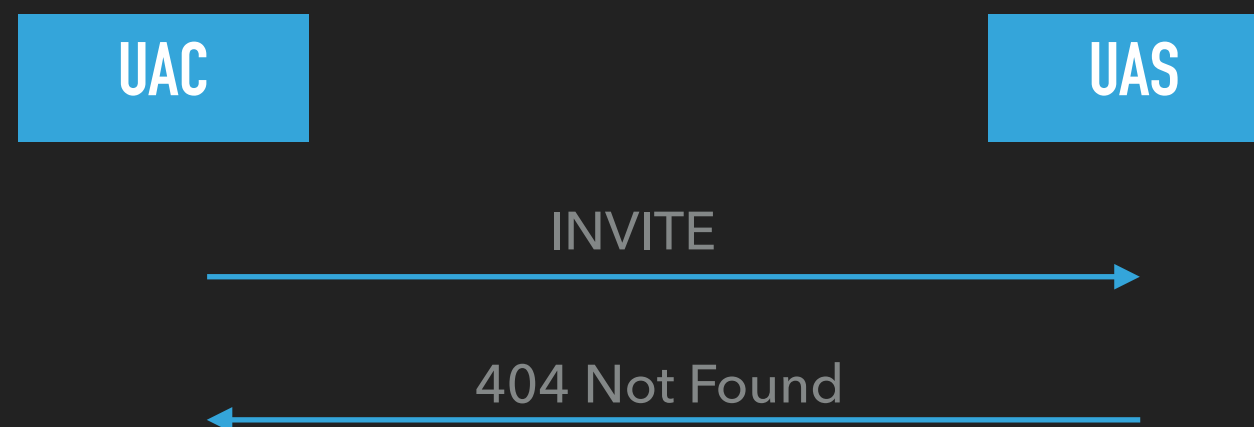
403 FORBIDDEN

- ▶ You will see this often as a security measure. Many systems will issue this if a UAC sends an INVITE and is not registered.



404 NOT FOUND

- ▶ This implies that the UAS received the response, however the user does not exist at the domain specified within the Request URI. This could also be sent if the domain doesn't match.
- ▶ Call the wrong number, you can expect to see a SIP 404.



405 METHOD NOT ALLOWED

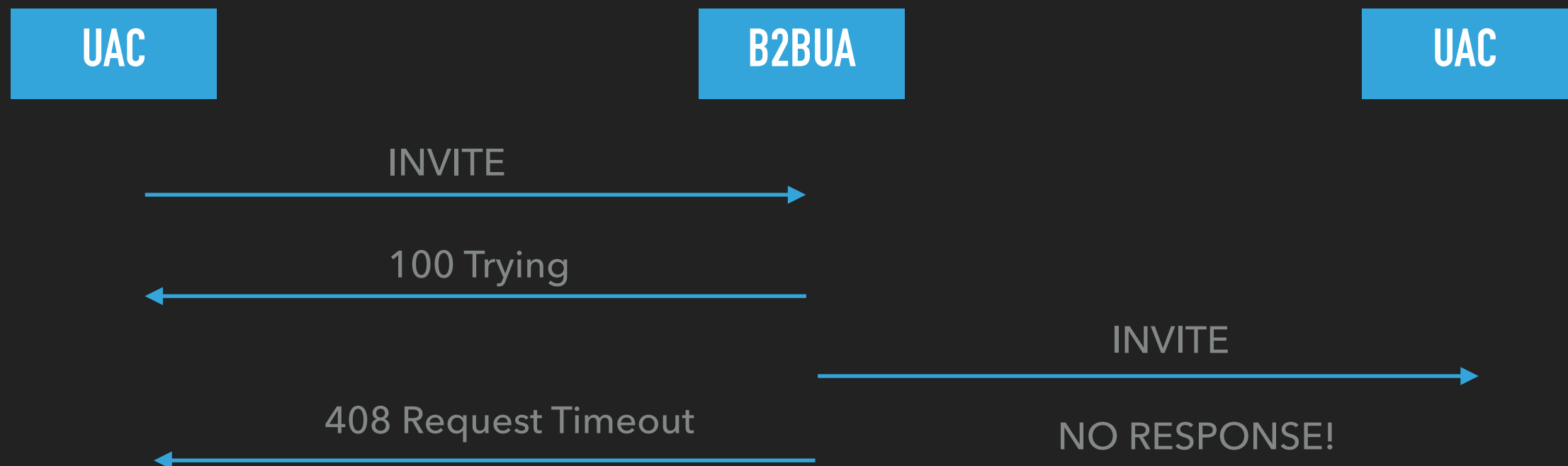
- ▶ The Method in the request-line is understood however it is not allowed for the address identified within the RURI.
- ▶ The response MUST include an `ALLOW` header that contains a list of valid methods for the address.

406 NOT ACCEPTABLE

- ▶ The resource identified by the request is only capable of generating response entities that have content characteristics not acceptable according to the Accept header field sent in the request.

408 REQUEST TIMEOUT

The server could not produce a response within a suitable amount of time, for example, if it could not determine the location of the user in time. The client MAY repeat the request without modifications at any later time.



410 GONE

The requested resource is no longer available at the server and no forwarding address is known. This condition is expected to be considered permanent. If the server does not know, or has no facility to determine, whether or not the condition is permanent, the status code 404 (Not Found) **SHOULD** be used instead.

413 REQUEST ENTITY TOO LARGE

The server is refusing to process a request because the request entity-body is larger than the server is willing or able to process. The server MAY close the connection to prevent the client from continuing the request.

If the condition is temporary, the server SHOULD include a Retry-After header field to indicate that it is temporary and after what time the client MAY try again.

If a request is sent with too many headers and /or a massive SDP, you can expect to receive this response.

414 REQUEST-URI TOO LONG

The server is refusing to service the request because the Request-URI is longer than the server is willing to interpret.

415 UNSUPPORTED MEDIA TYPE

The server is refusing to service the request because the message body of the request is in a format not supported by the server for the requested method. The server **MUST** return a list of acceptable formats using the `Accept`, `Accept-Encoding`, or `Accept-Language` header field, depending on the specific problem with the content.

416 UNSUPPORTED URI SCHEME

The server cannot process the request because the scheme of the URI in the Request-URI is unknown to the server.

420 BAD EXTENSION

The server did not understand a protocol extension specified within the proxy-require or require header field.

When this is sent the UAS MUST include a list of the unsupported extensions in an Unsupported header field within the response.

Note - This does not imply that the Extension NUMBER dialed is bad. That would be more inline with a SIP 404.

421 EXTENSION REQUIRED

The UAS needs a particular extension to process the request, but this extension is not listed in a Supported header field in the request.

Responses with this status code **MUST** contain a Require header field listing the required extensions.

A UAS **SHOULD NOT** use this response unless it truly cannot provide any useful service to the client. Instead, if a desirable extension is not listed in the Supported header field, servers **SHOULD** process the request using baseline SIP capabilities and any extensions supported by the client.

423 INTERVAL TOO BRIEF

The UAS is rejecting the request because the expiration time of the resources to be refreshed is too short.

480 TEMPORARILY UNAVAILABLE

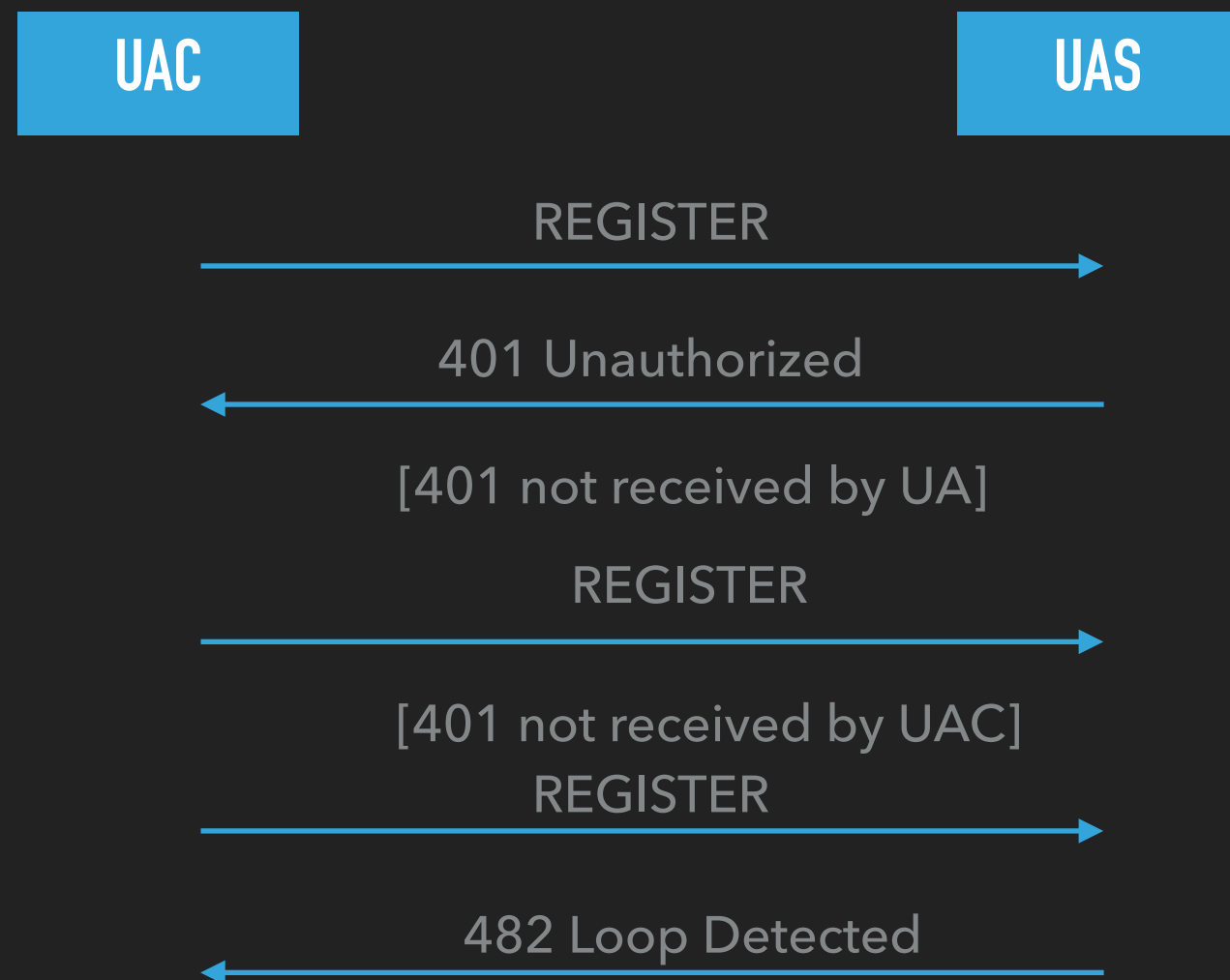
The callee's end system was contacted successfully but the callee is currently unavailable (for example, is not logged in, logged in but in a state that precludes communication with the callee, or has activated the "do not disturb" feature). The response MAY indicate a better time to call in the Retry-After header field. The user could also be available elsewhere (unknownst to this server). The reason phrase SHOULD indicate a more precise cause as to why the callee is unavailable. This value SHOULD be settable by the UA. Status 486 (Busy Here) MAY be used to more precisely indicate a particular reason for the call failure.

481 CALL/TRANSACTION DOES NOT EXIST

This status indicates that the UAS received a request that does not match any existing dialog or transaction.

482 LOOP DETECTED

The Server has detected a loop. In my experience, I've seen this frequently where packet loss is occurring.



483 TOO MANY HOPS

If the UAS receives a request where the Max-Forwards value is zero, it will generate this response

484 ADDRESS INCOMPLETE

The server received a request with a Request-URI that was incomplete.

Additional information **SHOULD** be provided in the reason phrase.

This status code allows overlapped dialing. With overlapped dialing, the client does not know the length of the dialing string. It sends strings of increasing lengths, prompting the user for more input, until it no longer receives a 484 (Address Incomplete) status response.

486 BUSY HERE

The callee's end system was contacted successfully, but the callee is currently not willing or able to take additional calls at this end system. The response MAY indicate a better time to call in the Retry-After header field. The user could also be available elsewhere, such as through a voice mail service.

Status 600 (Busy Everywhere) SHOULD be used if the client knows that no other end system will be able to accept this call.

487 REQUEST TERMINATED

The request was terminated by a BYE or CANCEL request. This response is never returned for a CANCEL request itself.

488 NOT ACCEPTABLE HERE

The response has the same meaning as 606 (Not Acceptable), but only applies to the specific resource addressed by the Request-URI and the request may succeed elsewhere.

A message body containing a description of media capabilities MAY be present in the response, which is formatted according to the Accept header field in the INVITE (or application/sdp if not present), the same as a message body in a 200 (OK) response to an OPTIONS request.

491 REQUEST PENDING

The request was received by a UAS that had a pending request within the same dialog. In traditional telecom terms, this would be considered a “glare condition”.

5XX SERVER FAILURES

Responses in this range occur when the UAS experiences an error.

501 NOT IMPLEMENTED

The server does not support the functionality required to fulfill the request. This is the appropriate response when a UAS does not recognize the request method and is not capable of supporting it for any user. (Proxies forward all requests regardless of method.)

Note that a 405 (Method Not Allowed) is sent when the server recognizes the request method, but that method is not allowed or supported.

501 NOT IMPLEMENTED

The server does not support the functionality required to fulfill the request. This is the appropriate response when a UAS does not recognize the request method and is not capable of supporting it for any user. (Proxies forward all requests regardless of method.)

Note that a 405 (Method Not Allowed) is sent when the server recognizes the request method, but that method is not allowed or supported.

502 BAD GATEWAY

The server, while acting as a gateway or proxy, received an invalid response from the downstream server it accessed in attempting to fulfill the request.

503 SERVER NOT AVAILABLE

The server is temporarily unable to process the request due to a temporary overloading or maintenance of the server. The server MAY indicate when the client should retry the request in a Retry-After header field. If no Retry-After is given, the client MUST act as if it had received a 500 (Server Internal Error) response.

A client (proxy or UAC) receiving a 503 (Service Unavailable) SHOULD attempt to forward the request to an alternate server. It SHOULD NOT forward any other requests to that server for the duration specified in the Retry-After header field, if present.

Servers MAY refuse the connection or drop the request instead of responding with 503 (Service Unavailable).

503 SERVER NOT AVAILABLE

The server is temporarily unable to process the request due to a temporary overloading or maintenance of the server. The server **MAY** indicate when the client should retry the request in a Retry-After header field. If no Retry-After is given, the client **MUST** act as if it had received a 500 (Server Internal Error) response.

A client (proxy or UAC) receiving a 503 (Service Unavailable) **SHOULD** attempt to forward the request to an alternate server. It **SHOULD NOT** forward any other requests to that server for the duration specified in the Retry-After header field, if present.

Servers **MAY** refuse the connection or drop the request instead of responding with 503 (Service Unavailable).

504 SERVER TIME-OUT

The server did not receive a timely response from an external server it accessed in attempting to process the request. 408 (Request Timeout) should be used instead if there was no response within the period specified in the Expires header field from the upstream server.

505 VERSION NOT SUPPORTED

The server does not support, or refuses to support, the SIP protocol version that was used in the request. The server is indicating that it is unable or unwilling to complete the request using the same major version as the client, other than with this error message.

513 MESSAGE TOO LARGE

The server was unable to process the request since the message length exceeded its capabilities.

513 MESSAGE TOO LARGE

The server was unable to process the request since the message length exceeded its capabilities.

6XX GLOBAL FAILURES

6xx responses indicate that a server has definitive information about a particular user, not just the particular instance indicated in the Request-URI.

600 BUSY EVERYWHERE

The callee's end system was contacted successfully but the callee is busy and does not wish to take the call at this time. The response MAY indicate a better time to call in the Retry-After header field. If the callee does not wish to reveal the reason for declining the call, the callee uses status code 603 (Decline) instead. This status response is returned only if the client knows that no other end point (such as a voice mail system) will answer the request. Otherwise, 486 (Busy Here) should be returned

603 DECLINE

The callee's machine was successfully contacted but the user explicitly does not wish to or cannot participate. The response MAY indicate a better time to call in the Retry-After header field. This status response is returned only if the client knows that no other end point will answer the request.

604 DOES NOT EXIST ANYWHERE

The server has authoritative information that the user indicated in the Request-URI does not exist anywhere.

606 NOT ACCEPTABLE

The user's agent was contacted successfully but some aspects of the session description such as the requested media, bandwidth, or addressing style were not acceptable.

A 606 (Not Acceptable) response means that the user wishes to communicate, but cannot adequately support the session described. The 606 (Not Acceptable) response MAY contain a list of reasons in a Warning header field describing why the session described cannot be supported.

A message body containing a description of media capabilities MAY be present in the response, which is formatted according to the Accept header field in the INVITE (or application/sdp if not present), the same as a message body in a 200 (OK) response to an OPTIONS request.

It is hoped that negotiation will not frequently be needed, and when a new user is being invited to join an already existing conference, negotiation may not be possible. It is up to the invitation initiator to decide whether or not to act on a 606 (Not Acceptable) response.

This status response is returned only if the client knows that no other end point will answer the request.

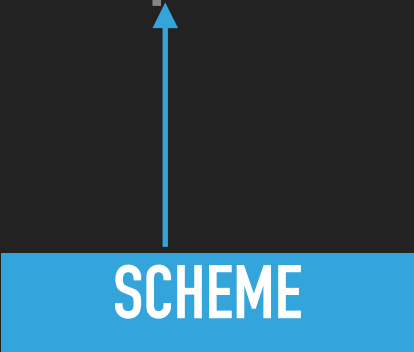
THE SIP URI

THE SIP URI

- ▶ You've seen an email address right?
- ▶ SIP is based off of the same URI (uniform resource indicator) structure. As defined in RFC 2396
- ▶ A sip UR could look like:
- ▶ sip:user:password@host:port;parameters or sip:user@host:port;parameters
- ▶ While it is permitted, the RFC states that it is NOT RECOMMENDED to contain the password within the URI.
- ▶

THE SIP URI

- ▶ To better understand, lets go over the basics of the URI structure:
- ▶ sip:bob@domain.com:5060;user=phone



SCHEME

The diagram consists of a blue rectangular box containing the word 'SCHEME' in white, uppercase letters. A blue arrow points vertically upwards from the top center of this box to the 'sip' portion of the URI 'sip:bob@domain.com:5060;user=phone' in the list item above.

The scheme provides a description of the type of URI that is in use. Typically, it's going to represent the name of the protocol. In the case of SIP, the scheme can be SIP or SIPS. SIPS is used for secure SIP

THE SIP URI

► sip:bob@domain.com:5060;user=phone

SCHEME

USER

The user section describes the resources that is being referenced. While it's common to see this as an extension or phone number, it is not required.

THE SIP URI

▶ sip:bob@domain.com:5060;user=phone

SCHEME

USER

HOST

The host section describes the host that is providing the resource. This can be presented as an FQDN (fully qualified domain name) IPv4 address or IPV6 address. According to the RFC, it is RECOMMENDED to use an FQDN where possible.

THE SIP URI

► sip:bob@domain.com:5060;user=phone

SCHEME

USER

HOST

PORT

This will list the port number where the request is sent to

THE SIP URI

▶ sip:bob@domain.com:5060;user=phone

SCHEME

USER

HOST

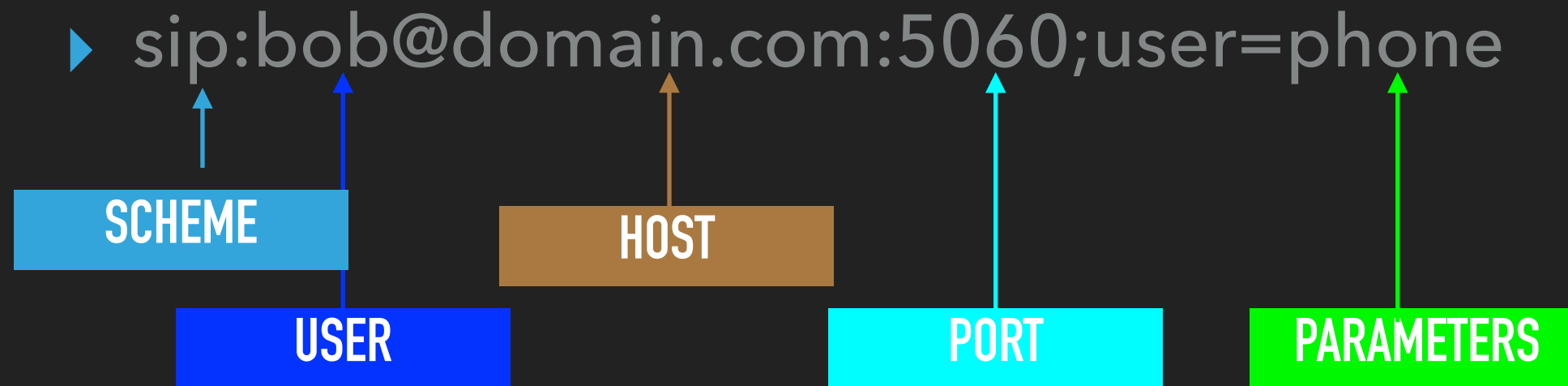
PORT

PARAMETERS

Parameters are used to add extra information to the URI. They are added after the port and separated by a semi-colon (;).

There is no limit on the number of parameters that can be included in a URI, however the same parameter name **MUST NOT** appear more than once.

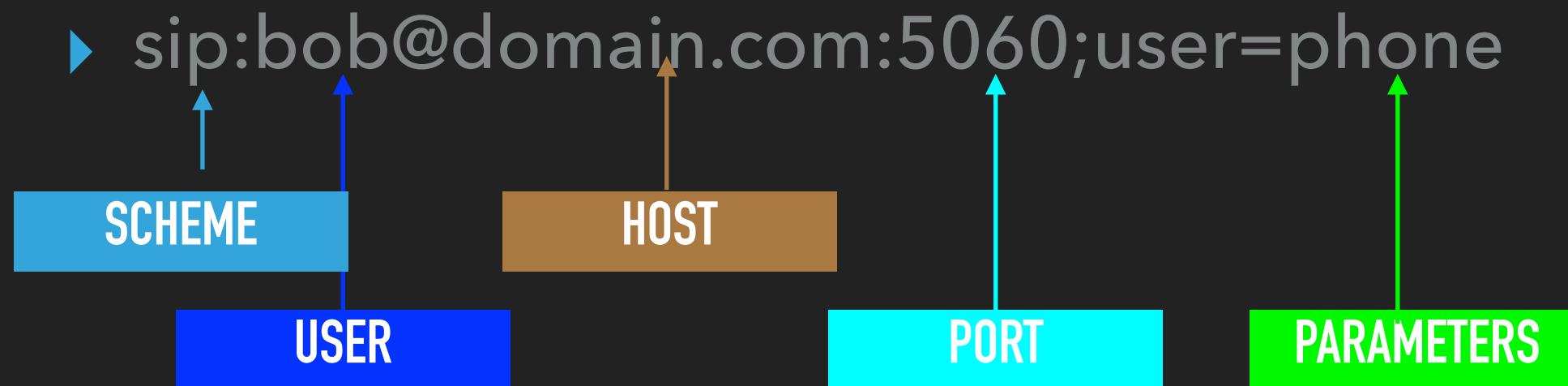
THE SIP URI



Parameters can be a proprietary to a certain SIP stack, however common ones are:

transport, maddr, ttl, user, method, and lr

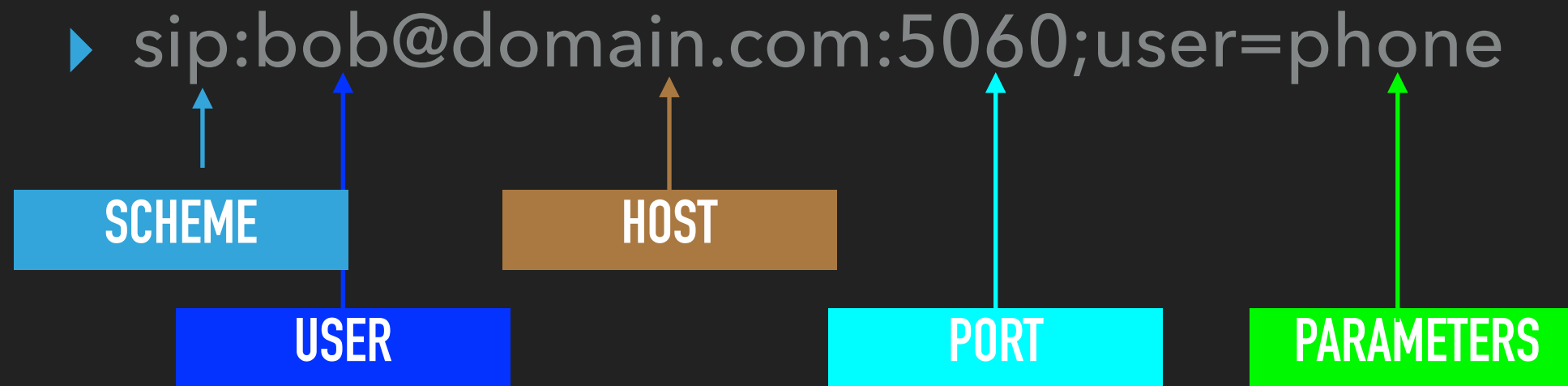
THE SIP URI



The transport parameter determines the transport method for sending the message. You will see either UDP or TCP listed here.

For SIPS URIs, a reliable transport method must be listed. Such as TLS

THE SIP URI



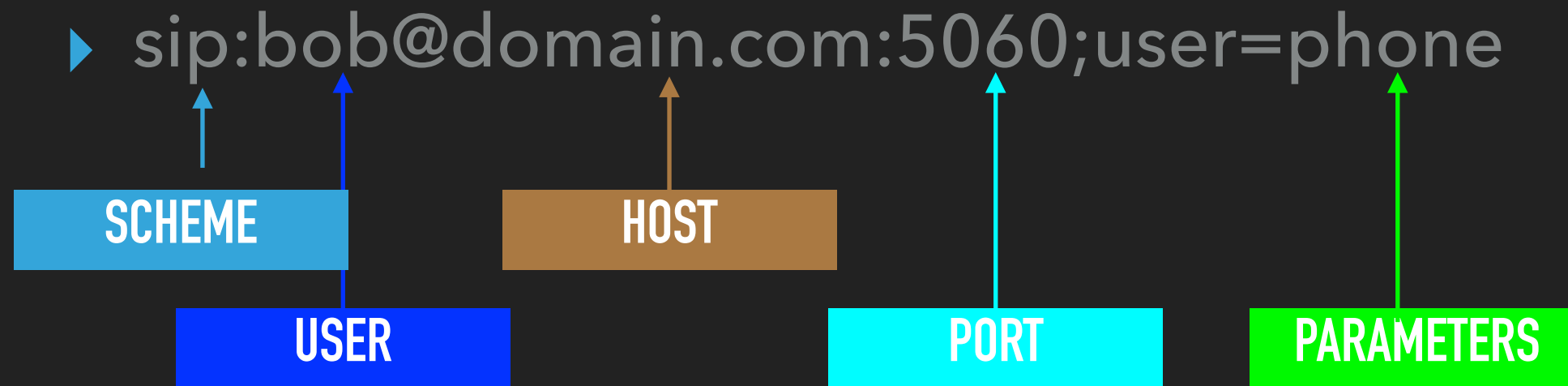
The maddr parameter has caused me a lot of grief in the past :)

This defines the server to be contacted for this user. This completely overrides what you see in the host portion of the URI.

For example <sip:4801001001@192.168.1.1:5060;maddr=192.168.4.10>

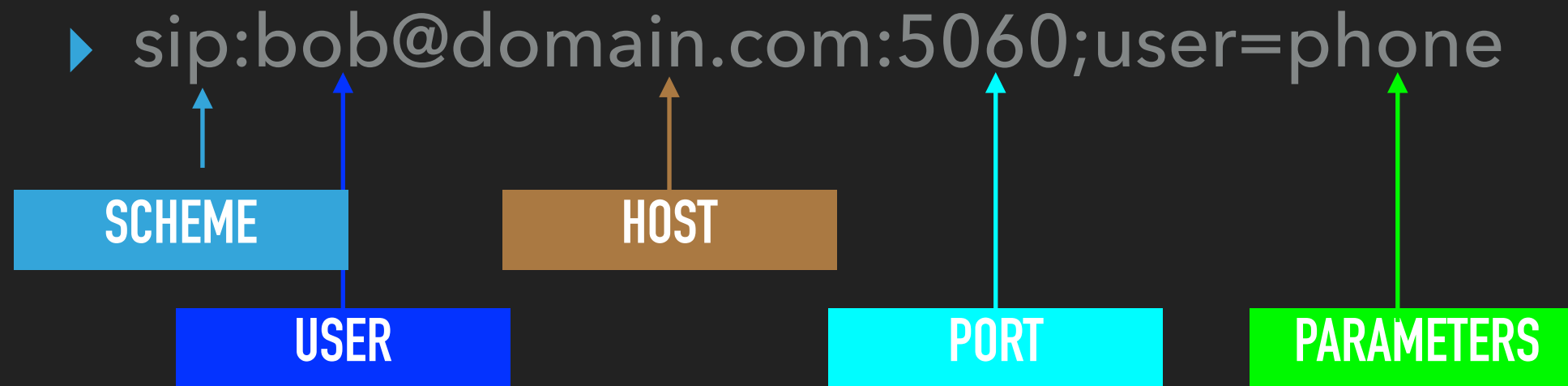
This request will be sent to 192.168.4.10, rather than 192.168.1.1

THE SIP URI



The ttl parameter determines the time-to-live value of the UDP multicast packet. This **MUST** only be used if the maddr parameter is a multicast address

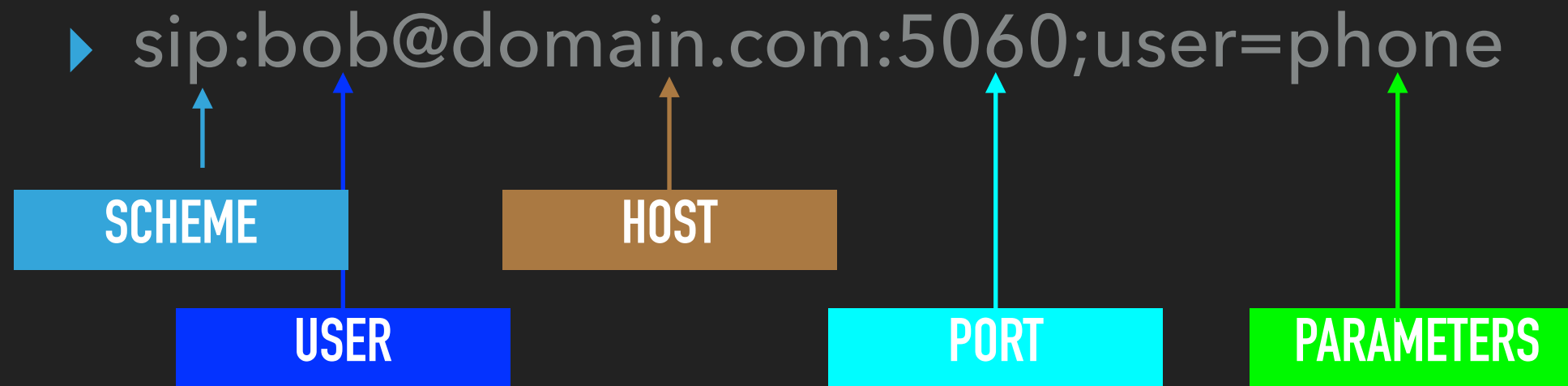
THE SIP URI



The user parameter is used to distinguish telephone numbers usernames that look like telephone numbers. If the user portion of the URI contains a telephone number formatted as a subscriber, the user value **SHOULD** be "phone".

Even without this param a recipient of a SIP or SIPS request may interpret the pre @ sign section as a phone number.

THE SIP URI



The *lr* parameter indicates that the device responsible for the resource implements the routing methods as defined in RFC 3261. If a record-route header is added to a message, you will see the *lr* parameter added to the URI.

This also indicates that the router (proxy) is a *loose router*.

SIP HEADERS

SIP HEADERS

- ▶ SIP headers fields are named attributes that provide additional information about a SIP message.
- ▶ They describe, who is calling, what is allowed in the call in terms of new requests and much more

Start Line

INVITE sip:bob@biloxi.example.com SIP/2.0

Header Fields

Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 151

SDP

v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.0.2.101
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000

SIP HEADERS

- ▶ A valid request from a UAC must contain a To, From, Cseq, Call-Id, Max-Forwards, and VIA header field.
- ▶ These six headers are the fundamental building blocks in a SIP message. .

```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.0.2.101
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

SIP HEADERS

- ▶ They provide information for :
- ▶ Critical message routing services.
- ▶ Addressing of messages.
- ▶ Routing of responses.
- ▶ Limiting message propagation.
- ▶ Ordering of messages.
- ▶ Unique identification of transactions.

SIP HEADERS

- ▶ They provide information for :
- ▶ Critical message routing services.
- ▶ Addressing of messages.
- ▶ Routing of responses.
- ▶ Limiting message propagation.
- ▶ Ordering of messages.
- ▶ Unique identification of transactions.

SIP HEADERS

- ▶ Of course the Request URI and SIP version are included as mandatory fields.

HEADER DEFINITIONS

- ▶ Request-URI

- ▶ ***INVITE sip:bob@biloxi.example.com SIP/2.0***

- ▶ The initial RURI SHOULD be set to the same value found in the To field.

- ▶ Should - In reality I see a lot of scenarios where this is not true. One example is where the RURI matches the pilot user AOR, and TO field matches the actual resource you are trying to send the message to.

- ▶ In a REGISTER request, the Ruri MUST NOT list the user info and "@" components of the SIP URI. It simply lists the domain (or IP) of the location service.

HEADER DEFINITIONS

▶ To

- ▶ To: Bob <sip:bob@biloxi.example.com>
- ▶ This specifies the logical recipient of the request.
- ▶ It can present a display name (Bob). This is optional.
- ▶ Remember the initial request does not contain a tag, as it is gleaned from the UAS that receives the request.

HEADER DEFINITIONS

▶ From:

- ▶ From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
- ▶ This specifies the logical identity of the request initiator.
- ▶ It can present a display name (Alice).
- ▶ It is often used by SIP elements to determine call processing rules that apply to a request. As such, the host portion usually will not list the FQDN or IP of the device originating the request.
- ▶ A Tag is added to begin establishment of a dialog.

HEADER DEFINITIONS

▶ Call-ID

- ▶ Call-ID: [3848276298220188511@atlanta.example.com](#)
- ▶ This acts as a unique identifier to group a series of messages.
- ▶ It MUST be the same for all requests within the same dialog.
- ▶ If a request times out and is retransmitted, it is not considered a new request, and therefor does NOT need a new Call-ID.
- ▶ It is recommended that the UAC generate a cryptographic random identifier when creating a Call-ID as they must be globally unique.
- ▶ Yes, these are case sensitive!

HEADER DEFINITIONS

- ▶ CSeq

- ▶ CSeq: 1 INVITE

- ▶ This serves as a way to identify the order of transactions. It will consist of a sequence number and a method.
 - ▶ The method MUST match the method listed in the start-line.
 - ▶ The UAC generating the request does NOT need to start at "1" with a CSeq value. As long as additional transactions increase monotonically (+1).

HEADER DEFINITIONS

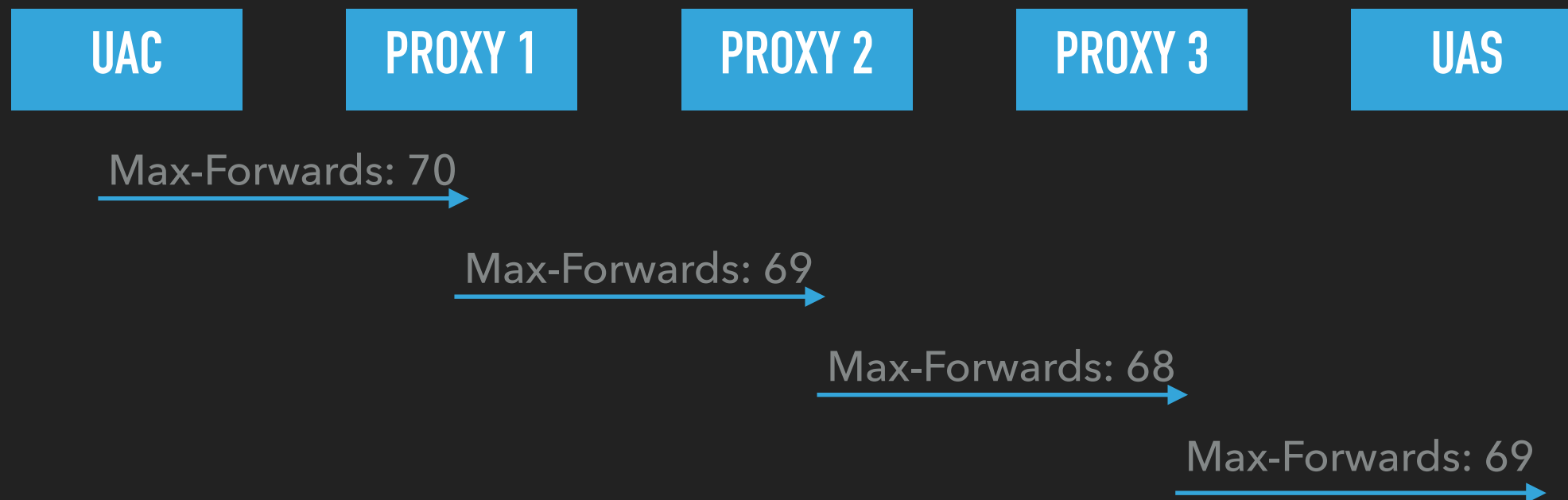
▶ Max-Forwards

- ▶ Max-Forwards: 70
- ▶ This is used to prevent loops.
- ▶ As a request goes through each hop, the number is decreased by 1.
- ▶ If this reaches "0" by the time it reaches the destination, the UAS will reject the call with a "483 - Too Many Hops response.
- ▶ According to the RFC - the UAC generating the request SHOULD set this value to 70.
- ▶ This value was chosen to guarantee the message would not be dropped when no loops occur.

HEADER DEFINITIONS

- ▶ Max-Forwards

- ▶ Max-Forwards: 70



HEADER DEFINITIONS

▶ Via

- ▶ Via: SIP/2.0/TCP [client.atlanta.example.com](#):5060;branch=z9hG4bK74bf9
- ▶ This is used for **routing responses** and identifies the transport used for this transaction.
- ▶ This header must contain a branch parameter.
 - ▶ This is used to identify the transaction created by this request.
 - ▶ The branch parameter is used by both the UAC and UAS

HEADER DEFINITIONS

▶ Via

- ▶ Via: SIP/2.0/TCP [client.atlanta.example.com](#):5060;branch=z9hG4bK74bf9
- ▶ The branch parameter must be unique across “all time and space” for requests sent by this UAC.
- ▶ Of course... there are exceptions.
 - ▶ A CANCEL request will match the INVITE that it is cancelling.
 - ▶ An ACK for a non 200 response will match the INVITE that it is acknowledging.

HEADER DEFINITIONS

▶ Via

- ▶ Via: SIP/2.0/TCP [client.atlanta.example.com](#):5060;branch=**z9hG4bK**74bf9
 - ▶ All branch parameters that are compliant with RFC 3261 must begin with **"z9hG4bK"**
 - ▶ *This is known as the "Magic Cookie"*

HEADER DEFINITIONS

▶ Contact

- ▶ Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
- ▶ The contact header provides a URI that can be used to contact the UA for subsequent requests.
- ▶ When the Request is REGISTER:
 - ▶ The 200 OK response will list an expires parameter.
 - ▶ When the expires param is listen the UAC should attempt to re-register in less than the time specified.

HEADER DEFINITIONS

► Contact

- Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
- If the request is an INVITE :
 - You may receive a 30X Redirect, which presents different URIs that you should attempt to contact.

```
SIP/2.0 302 Moved Temporarily
Via: SIP/2.0/UDP client.atlanta.example.com:5060;branch=z9hG4bKbf9f44;received=192.0.2.101
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76s1
To: Bob <sip:bob@biloxi.example.com>;tag=53fHlqlQ2
Call-ID: 2xTb9vxSit55XU7p8@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:bob@chicago.example.com;transport=tcp>
Content-Length: 0
```

HEADER DEFINITIONS

▶ Contact

- ▶ Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
- ▶ If the request is an INVITE :
 - ▶ You may receive a 30X Redirect, which presents different URIs that you should attempt to contact.

```
SIP/2.0 302 Moved Temporarily
Via: SIP/2.0/UDP client.atlanta.example.com:5060;branch=z9hG4bKbf9f44;received=192.0.2.101
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76s1
To: Bob <sip:bob@biloxi.example.com>;tag=53fHlqlQ2
Call-ID: 2xTb9vxSit55XU7p8@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:bob@chicago.example.com;transport=tcp>
Content-Length: 0
```

HEADER DEFINITIONS

▶ Alert-Info

- ▶ Alert-Info: <http://www.example.com/sounds/moo.wav>
- ▶ When present in an INVITE request, the Alert-Info header field specifies an alternative ring tone to the UAS. When present in a 180 (Ringing) response, the Alert-Info header field specifies an alternative ringback tone to the UAC. A typical usage is for a proxy to insert this header field to provide a distinctive ring feature.
- ▶ Can also be used for “auto-answering” of calls by certain endpoints

HEADER DEFINITIONS

- ▶ Allow

- ▶ Allow: INVITE, ACK, OPTIONS, CANCEL, BYE

- ▶ This specifies the types of request METHODS supported by the UA generating the message

HEADER DEFINITIONS

- ▶ Authorization

- ▶ Authorization: Digest username="Alice",
realm="atlanta.com",nonce="84a4cc6f3082121f32b42a21
87831a9e",response="7587245234b3434cc3412213e5f11
3a5432"
- ▶ This contains the authentications credentials of the UA.

HEADER DEFINITIONS

▶ Call-Info

- ▶ Call-Info: <<http://www.example.com/alice/photo.jpg>> ;purpose=icon,
- ▶ This provides additional information about the request originator or responder.

HEADER DEFINITIONS

- ▶ Content-Disposition

- ▶ Content-Disposition: session

- ▶ Describes how the message body is to be interpreted by the UAC or UAS.

HEADER DEFINITIONS

- ▶ Content-Length
 - ▶ Content-Length: 349
 - ▶ This describes the size of the message body.
 - ▶ Does not include the the CRLF splitting the header fields from the body.

```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP client.atlanta.example.com:
5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact:
<sip:alice@client.atlanta.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4
client.atlanta.example.com
s=-
c=IN IP4 192.0.2.101
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

HEADER DEFINITIONS

- ▶ Content-Type
 - ▶ Content-Type: application/sdp
 - ▶ This describes the type of media sent in the message body.
 - ▶

HEADER DEFINITIONS

- ▶ Min-Expires
 - ▶ Min-Expires: 60
 - ▶ This specifies the minimum refresh interval of soft-state elements managed by the UA.

HEADER DEFINITIONS

- ▶ Record-Route

- ▶ Record-Route: <sip:server10.biloxi.com;lr>,<sip:bigbox3.site3.atlanta.com;lr>
- ▶ This is inserted by proxies in a request to force all future in-dialog requests to route through the proxy.

HEADER DEFINITIONS

- ▶ Require
 - ▶ Require: 100rel
 - ▶ This is used by UACs to tell UASs about options that the UAC expects the UAS to support in order to process a request.
 - ▶ The above "Require: 100rel" is common example. You will see this where the UAC requires all provisional responses to be "reliable". This is accomplished by the use of a PRACK, or Provisional Acknowledgement.
 - ▶ If you touch Lync / Skype for Business you will definitely see this.

HEADER DEFINITIONS

- ▶ Route

- ▶ Route: <sip:bigbox3.site3.atlanta.com;lr>, <sip:server10.biloxi.com;lr>
- ▶ The Route header field is used to force routing for a request through the listed set of proxies.

HEADER DEFINITIONS

- ▶ Supported
 - ▶ Supported: 100rel
 - ▶ This lists all extensions supported by the UAC or UAS.

HEADER DEFINITIONS

- ▶ User-Agent
 - ▶ User-Agent: Softphone Beta1.5
 - ▶ This header contains information about the UA. It could contain the Manufacturer name, SIP UA version, and other information.
 - ▶ Note - This is often “spoofed” to that attackers hide the name of their scanning tools.

HEADER DEFINITIONS

- ▶ WWW-Authenticate
 - ▶ WWW-Authenticate: Digest realm="atlanta.com", domain="sip:boxesbybob.com", qop="auth", nonce="f84f1cec41e6cbe5aea9c8e88d359", opaque="", stale=FALSE, algorithm=MD5
 - ▶ This header contains an authentication challenge.

COMPACT HEADERS

- ▶ Occasionally you will see a SIP message with “compact headers”.
- ▶ These messages act the same as those with long format headers, they just look different.
- ▶ The following slide contains a list of full to compact header mappings.
- ▶

COMPACT HEADERS

Abbreviation	Header
a	Accept-Contact
b	Referred-By
c	Content-Type
e	Content-Encoding
f	From
i	Call-ID
k	Supported
l	Content-Length
m	Contact
o	Event
r	Refer-To
s	Subject
t	To
u	Allow-Events
v	Via

SDP OVERVIEW

SESSION DESCRIPTION PROTOCOL

- ▶ The Session Description Protocol is defined within RFC 4566.
- ▶ When used with SIP, it provides a standard representation about how information is to be transported.
- ▶ It is purely a format for the session description. It does not carry media on it's own.

SDP IN A SIP MESSAGE

Start Line

INVITE sip:bob@biloxi.example.com SIP/2.0

Header Fields

Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@client.atlanta.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 151

SDP

v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.0.2.101
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000

SESSION DESCRIPTION PROTOCOL

- ▶ An SDP includes the following information:
 - ▶ The session name and purpose
 - ▶ Time(s) the session is active
 - ▶ The media type of used by the session
 - ▶ Information about how to receive media (IP, port, format, etc)

SESSION DESCRIPTION PROTOCOL

- ▶ In general the SDP presents enough information for UAs to join a session.
- ▶ The SDP contains a number of lines in the format of "`<type>=<value>`"
- ▶ "`type`" is exactly one case-significant character
- ▶ "`value`" depends on the format type.
- ▶ There are 2 parts to an SDP.

SESSION DESCRIPTION PROTOCOL

- ▶ Session Level -
 - ▶ This begins with the "v=" line
 - ▶ This ends when the first "m=" line is present.
- ▶ Media Level -
 - ▶ Begins with the first "m=" line.
 - ▶ An SDP can have multiple media levels. This is often seen where voice and video capabilities are present.

SESSION DESCRIPTION PROTOCOL

- ▶ Just like some SIP header fields are mandatory, If an SDP is present, there are mandatory fields as well.
- ▶ Let's go over these different type values :

SESSION DESCRIPTION PROTOCOL

▶ V=

- ▶ Required field.
- ▶ This is the version of the SDP.
- ▶ According to RFC 4566, the current defined version is "0".

SESSION DESCRIPTION PROTOCOL

▶ o=

▶ Required field.

▶ This presents information about the originator of the session.

▶ The format is o=<username> <session-id> <session-version> <nettype> <address type> <unicast-address>

SESSION DESCRIPTION PROTOCOL

▶ O=

- ▶ <username> is the user's login on the originating host, or it is "-" if the originating host does not support the concept of user IDs. The <username> MUST NOT contain spaces.

SESSION DESCRIPTION PROTOCOL

▶ ○=

- ▶ <session-id> This is a globally unique string to identify the session.
- ▶ The method of creating the session-id is not defined by the RFC, however it is often created by an NTP timestamp format.

SESSION DESCRIPTION PROTOCOL

▶ O=

▶ <session-version> This is a version number for the SDP.
Note - this is not the same as the "v=" line.

▶ When a change to the session is made, this version is incremented.

SESSION DESCRIPTION PROTOCOL

▶ ○=

- ▶ <nettype> - This is a text string that defines the type of network used for this session.
- ▶ The most common value you will see here is "IN", short for "internet"

▶

SESSION DESCRIPTION PROTOCOL

▶ O=

- ▶ <addrtype> is used to define the type of address that is found in the <unicast-address> field
- ▶ You will see IP4 to specify IPV4 addresses
- ▶ or IP6 for IPv6 addresses

SESSION DESCRIPTION PROTOCOL

- ▶ O=

- ▶ <unicast-address> this is the address of the machine that created the session.

- ▶ For IPv4-

- ▶ It can be listed as an FQDN or as the dotted decimal IP address

- ▶ IPv6 -

- ▶ This can be the FQDN or the compressed text version of the IPv6 address

SESSION DESCRIPTION PROTOCOL

▶ s=

▶ Required.

▶ This is the name of the session.

▶ There must be one and only one "s=" line in a valid SDP.

▶ The "s=" fields must also not be blank.

▶ This can be tricky as a "space" is a valid character for the session name, and can be used if the session does not have a meaningful name.

▶ I also see "-" frequently as the session name

SESSION DESCRIPTION PROTOCOL

- ▶ `t=`

- ▶ Required.

- ▶ format : `t=<start time> <stop time>`

- ▶ This specifies the start and stop time of the session.

- ▶ If non zero, these are listed as NTP values.

- ▶ If the stop time is set to 0, the session is not bound to a certain time, and it will not become active until after the start time.

- ▶ If the start time is 0 - the session is regarded as permanent.

SESSION DESCRIPTION PROTOCOL

- ▶ `m=`
 - ▶ Required.
 - ▶ format : `m=<media> <port> <proto> <fmt>`
 - ▶ An SDP may list multiple `m` lines.
 - ▶ `<media>` is presented as the media type. It can be "audio", "video", "text", "application", or "message".
 - ▶ `<port>` is the transport port used by by this session. This depends on what is presented in the "c=" line.

SESSION DESCRIPTION PROTOCOL

- ▶ m=

- ▶ <proto> is the protocol

- ▶ udp - denotes an unspecified protocol running over UDP

- ▶ RTP/AVP = Real Time Protocol for Audio and Video running over UDP

- ▶ RTP/SAVP = Secure RTP running over UDP

SESSION DESCRIPTION PROTOCOL

▶ m=

- ▶ <fmt> is the format of the media description. This is the fourth and any subsequent value in the m line.
- ▶ If the protocol is RTP/AVP or RTP/SAVP the format fields will list the RTP Payload types.
 - ▶ 0 = g.711U
 - ▶ 9 = g.722
 - ▶ 18 = g.729a

SESSION DESCRIPTION PROTOCOL

- ▶ **c=**

- ▶ This is the connection information.
- ▶ format : `c=<nettype> <addrtype> <connection-address>`
- ▶ You can have one `c=` line presented at the session level, or a `c=` line within each media section.



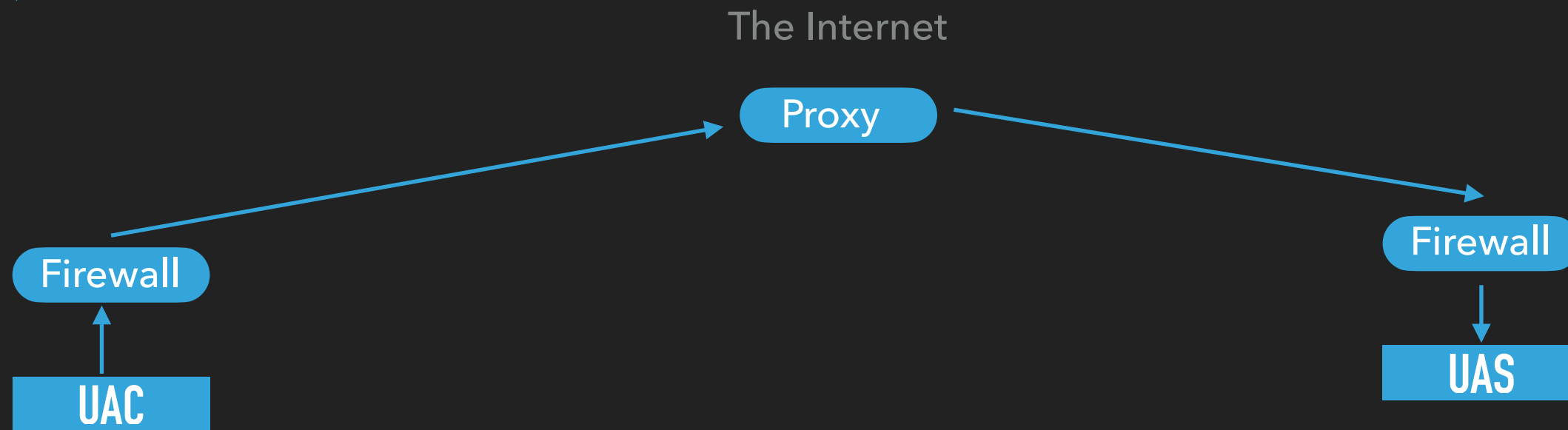
SESSION DESCRIPTION PROTOCOL

▶ C=

- ▶ <nettype> = Network type - most likely going to be "IN" for "internet"
- ▶ <addrtype> = will be IP4 or IP6
- ▶ <connection-address> = This is the IP that the remote host should send RTP packets toward.
- ▶ One-way audio issues - Always example this.

SESSION DESCRIPTION PROTOCOL

► C=



```
INVITE sip:bob@biloxi.example.com SIP/2.0  
<headers removed>
```

```
v=0  
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com  
s=-  
c=IN IP4 192.0.2.101  
t=0 0  
m=audio 49172 RTP/AVP 0  
a=rtpmap:0 PCMU/8000
```

```
SIP/2.0 200 OK  
<headers removed>
```

```
v=0  
o=alice 2890844526 2890844526 IN IP4 192.168.1.1  
s=-  
c=IN IP4 192.1.1.101  
t=0 0  
m=audio 49172 RTP/AVP 0  
a=rtpmap:0 PCMU/8000
```

SESSION DESCRIPTION PROTOCOL

- ▶ Addition session-level field (optional)

- ▶ i = session information

- ▶ u = uri of description

- ▶ e = email address

- ▶ p = phone number

- ▶ b = bandwidth

- ▶ z = timezone adjustments

- ▶ k = encryption key

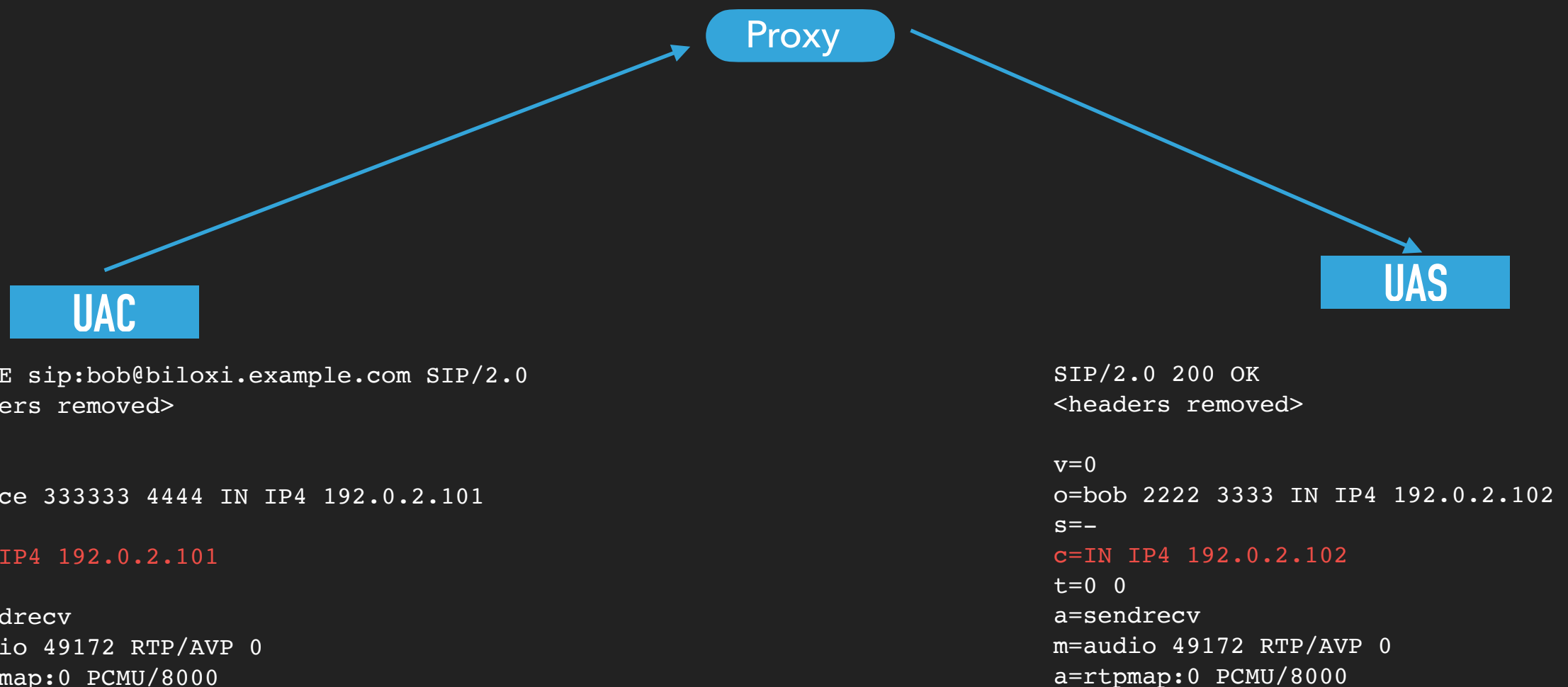
- ▶ **a = attributes**

SESSION DESCRIPTION PROTOCOL

- ▶ Additional Media level fields
 - ▶ i= media title
 - ▶ c= connection information (this is optional if included at the session level)
 - ▶ b= bandwidth information
 - ▶ k= encryption key
 - ▶ **a = attributes**

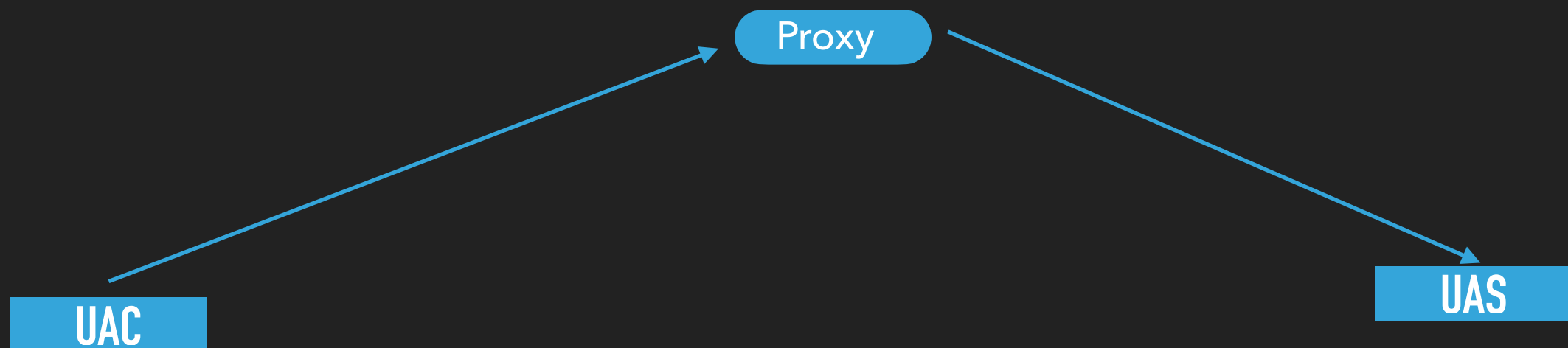
SESSION DESCRIPTION PROTOCOL

- ▶ There are times where the attributes can be extremely helpful identifying what is going on with a session.
- ▶ Prime example - a call going on hold
- ▶ Here we have a regular call. Notice the a= line



SESSION DESCRIPTION PROTOCOL

- ▶ Notice that the UAC sends other INVITE, but the attribute has changed to "sendonly"
- ▶ The UAS accepts the new invite, and sets their attribute to "recvonly"



```
INVITE sip:bob@biloxi.example.com SIP/2.0
<headers removed>
```

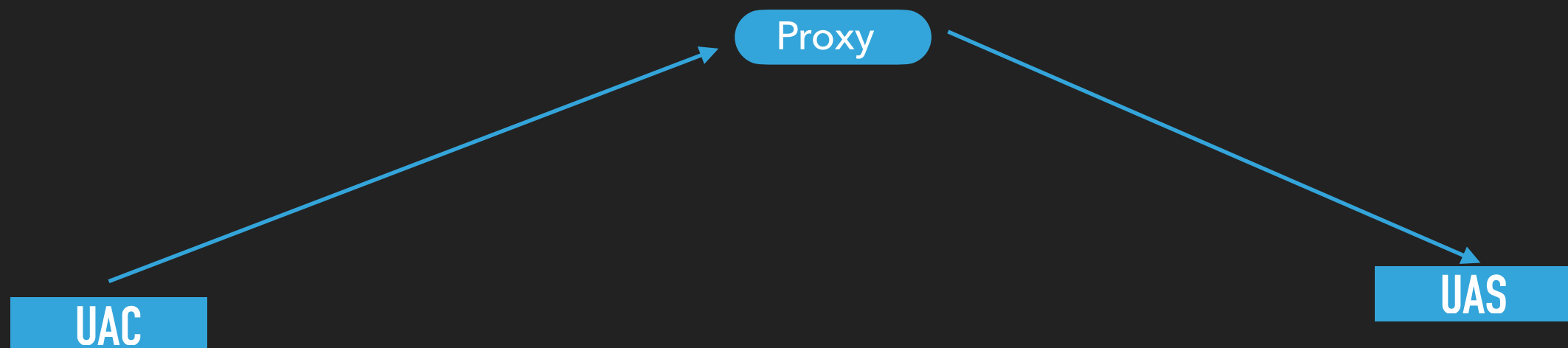
```
v=0
o=alice 333333 4445 IN IP4 192.0.2.101
s=-
c=IN IP4 192.0.2.101
t=0 0
a=sendonly
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

```
SIP/2.0 200 OK
<headers removed>
```

```
v=0
o=bob 2222 3334 IN IP4 192.0.2.102
s=-
c=IN IP4 192.0.2.102
t=0 0
a=recvonly
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

SESSION DESCRIPTION PROTOCOL

- ▶ This call is now on hold.
- ▶ When the UAC wants to resume the call, they will send another INVITE with the a= line set to "sendrecv"



```
INVITE sip:bob@biloxi.example.com SIP/2.0
<headers removed>
```

```
v=0
o=alice 333333 4446 IN IP4 192.0.2.101
s=-
c=IN IP4 192.0.2.101
t=0 0
a=sendrecv
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

```
SIP/2.0 200 OK
<headers removed>
```

```
v=0
o=bob 2222 3335 IN IP4 192.0.2.102
s=-
c=IN IP4 192.0.2.102
t=0 0
a=sendrecv
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=cat
 - ▶ session-level attribute
 - ▶ This provides a dot-separated category of the session. This allows the receiver to filter undated session by category.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=keywords
 - ▶ session-level attribute
 - ▶ Again - allows the receiver to filter unwanted sessions by the keywords sent.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=tool
 - ▶ session-level attribute
 - ▶ Gives the name and session number of the tool used to create the SDP

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=ptime
 - ▶ media-level attribute
 - ▶ This is the packetization interval of the RTP packets.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=maxptime
 - ▶ media-level attribute
 - ▶ Specifies the maximum amount of media that can be encapsulated in each packet.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:

- ▶ a=rtpmap

- ▶ media-level attribute

- ▶ **format** : a=rtpmap:<payload type> <encoding name>/<clock rate> [/<encoding parameters>

- ▶ This maps an RTP payload type to an encoding name that lists the payload format used.

- ▶ Example:

- ▶ a=rtpmap:0 PCMU/8000

- ▶ a=rtpmap:18 G729/8000

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=recvonly
 - ▶ session or media level attribute
 - ▶ States that the UA sending this will only receive RTP, it will not send.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=sendonly
 - ▶ session or media level attribute
 - ▶ States the the UA sending this attribute will send media, but not receive.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=inactive
 - ▶ session or media level attribute
 - ▶ Nothing is sent, or expected to be received.
 - ▶ Note - RTCP SHOULD still be sent when inactive.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=framerate
 - ▶ media-level attribute - only for video media.
 - ▶ This will specify the maximum video frame rate in frame per second.

SESSION DESCRIPTION PROTOCOL

- ▶ Additional attributes:
 - ▶ a=fmtp
 - ▶ Media level attribute.
 - ▶ Specifies parameters that are specific to a format and conveyed in a way that the SDP does not have to understand them.
 - ▶ Example:
 - ▶ a=fmtp:18 annexb=no
 - ▶ This implies that we are not using g.729b in this session.

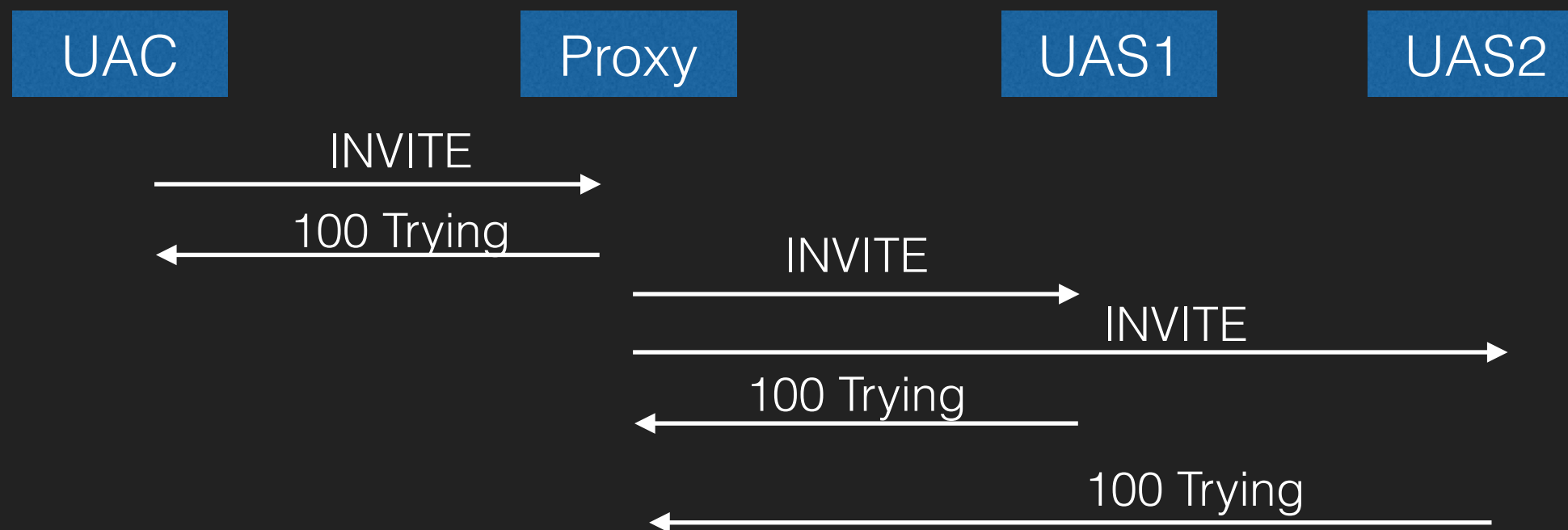
SIP MOBILITY

SIP MOBILITY

There are times where a SIP user needs to be reachable at different locations. A prime example is a user has a phone at their desk, but they also need a phone in another room when a call is sent to them.

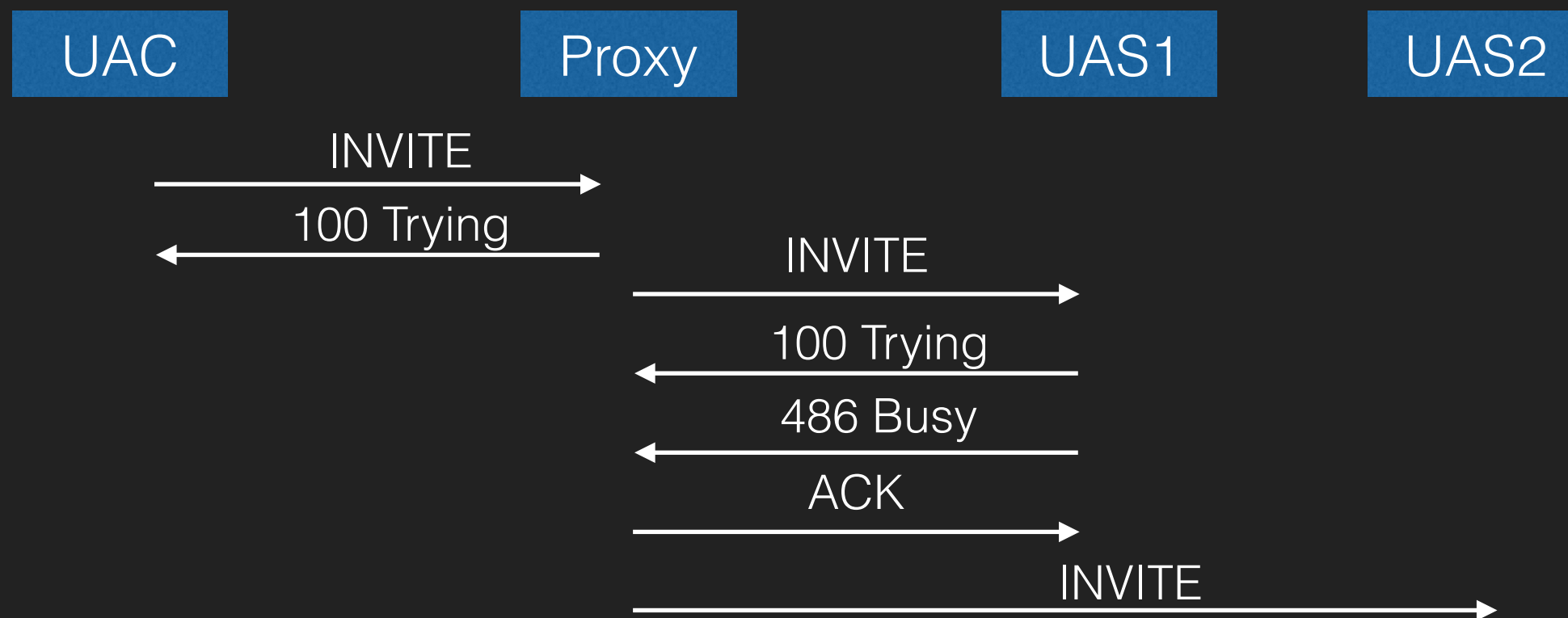
SIP MOBILITY

To accomplish this a proxy will send an INVITE to a number of locations pre-defined by the user or SIP network administrator. This is known as “Parallel Call Forking”



SIP MOBILITY

A proxy may also send INVITEs to one location, wait for a failure, then send an INVITE to another destination. This is known as “Sequential Call Forking”



SIP MOBILITY

These features are available on most SIP servers on the market. While each SIP vendor may have different names for them, forking is forking.

BRANCHES IDS

In a previous module we discussed branch IDs briefly. When forking is used, branch IDs are required to match responses to forked requests.

Each proxy that forks a request will add it's own unique branch ID.

REPLACES HEADER

Some SIP stacks will utilize a “Replaces” header. When this header is present it is meant to logically replace one dialog, with another.

The Replaces header is specified in RFC 3891.

REPLACES HEADER

Here's the scenario : Alice is talking to Bob on her desk phone. She needs to run into another room to look up the information that Bob requires.

At this time, Alice transfers the call to a "parking lot" or hold location.

REPLACES HEADER

Alice runs to the other room and picks up the phone using a special sequence (unique to her PBX).

When she does this her SIP PBX sends an INVITE with a replaces header.

REPLACES HEADER

INVITE sip:bob@bobster.example.org
To: <sip:bob@example.org>
From: <sip:alice@phone2.example.org>;tag=8983
Call-ID: 09870@phone2.example.org
CSeq: 1 INVITE
Contact: <sip:alice@phone2.example.org>
Require: replaces
Replaces: 425928@bobster.example.org;to-tag=7743;from-tag=6472

The Replaces header will lists the Call-ID, to tag and from-tag of the previous dialog.

REPLACES HEADER

When the UAS receives this INVITE it searches through the existing dialogs on the system.

If a match is found the UAS MUST determine if the user issuing the Replaces header is authorized to make the request. If they are, they dialog id successfully replaced.

REPLACES HEADER

If no match is found, the UAS will reject the call with a SIP 481 Transaction Does Not Exist.

REPLACES HEADER

If more than one Replaces header is seen in the INVITE, the UAS will reject the call with a 400 Bad Request

```
INVITE sip:bob@bobster.example.org
To: <sip:bob@example.org>
From: <sip:alice@phone2.example.org>;tag=8983
Call-ID: 09870@phone2.example.org
CSeq: 1 INVITE
Contact: <sip:alice@phone2.example.org>
Require: replaces
```

Replaces: 425928@bobster.example.org;to-tag=7743;from-tag=6472

Replaces: 44443@bobster.example.org;to-tag=7843;from-tag=6872

REPLACES HEADER

In the event that the Replaces header matches more than one dialog, the UAS must reject the call as if no match was found.

REPLACES HEADER

Finally, if the Replaces header matches a dialog that has already terminated the UAS will reject the call with a SIP 603 Declined.

This is to prevent a race condition where the UA rings or alerts for a call that is unwanted.

DIVERSION HEADER

Often a call is forwarded through a mechanism on the SIP PBX hosting user services. The SIP PBX may use the Diversion header to indicate details about the forward.

DIVERSION HEADER

A diversion header is structured in the following way :

Diversion : "Name" <Diverting user URI>;
parameters

DIVERSION HEADER

Example :

**Diversion: "9948" <sip:
9948@192.168.1.254>;reason=unconditional;privacy=off;screen=yes**

DIVERSION HEADER

The diversion parameters may contain :

1. The diversion reason
2. The diversion counter.
3. The diversion limit.
4. The privacy settings
5. Screening settings

DIVERSION HEADER

There are multiple options for diversion reasons:

unknown = The PBX is diverting for an unknown reason

user-busy = The user is busy.

no-answer = The user did not answer the call. This one of often seen when a call is sent to voicemail.

unavailable = The user may be offline.

unconditional = The user has configured an "Always forward calls rule"

time-of-day = The system is following a time of day route. This could imply that after business hours the call needs to go to another department

DIVERSION HEADER

More diversion reasons.

do-not-disturb = The user has configured a DND parameter on their line. Go to voicemail, or another destination.

deflection = Often used when a caller is busy. The call is deflected to another destination.

follow-me = This could be a sequential or parallel fork.

out-of-service = The user specified has been disconnected.

away = The user has specified that they are away from their phone.

DIVERSION HEADER

The diversion counter contains information about how many times the call has been diverted. For each successful diversion, the proxy needs to increase this # by one.

DIVERSION HEADER

The diversion limit specifies the maximum number of diversions that are permitted.

DIVERSION HEADER

Diversion-privacy specifies how much information the proxy needs to share with the next proxy, or terminating UAS about the originating user.

DIVERSION HEADER

Diversion screening is used to specify information about the calling party.

If screen = Yes , then the Calling party ID is network provided.

If screen = no, the Calling Party ID is User provided.

SIP REFER

Another request method for mobility is the REFER method.

SIP REFER

The REFER method is used to indicate that the Request recipient needs to contact a third party to complete the request.

A common example is a call is received by the receptionist of a company. The caller asks to speak with "Mrs. Smith". The receptionist then **transfers** the call to "Mrs. Smith".

SIP REFER

REFER method example

REFER sip:b@atlanta.example.com SIP/2.0

Via: SIP/2.0/UDP agenta.atlanta.example.com;branch=z9hG4bK2293940223

To: <sip:b@atlanta.example.com>

From: <sip:a@atlanta.example.com>;tag=193402342

Call-ID: 898234234@agenta.atlanta.example.com

CSeq: 93809823 REFER

Max-Forwards: 70

Refer-To: <sip:fred@atlanta.example.com>

Contact: sip:a@atlanta.example.com

Content-Length: 0

SIP REFER

Processing a REFER:

When a UA receives a REFER request it must contact the resource identified within the Refer-to header field.

Refer-To: <sip:fred@atlanta.example.com>

SIP REFER

Bad REFER request :

If the request has no Refer-To header, or more than one Refer-To header, the UA that received the request will respond with a SIP 400 Bad Request.

REFER sip:b@atlanta.example.com SIP/2.0

Via: SIP/2.0/UDP

agenta.atlanta.example.com;branch=z9hG4bK2293940223

To: <sip:b@atlanta.example.com>

From: <sip:a@atlanta.example.com>;tag=193402342

Call-ID: 898234234@agenta.atlanta.example.com

CSeq: 93809823 REFER

Refer-To: <sip:fred@atlanta.example.com>

Refer-To: <sip:james@atlanta.example.com>

Max-Forwards: 70

Contact: sip:a@atlanta.example.com

Content-Length: 0

REFER sip:b@atlanta.example.com SIP/2.0

Via: SIP/2.0/UDP

agenta.atlanta.example.com;branch=z9hG4bK2293940223

To: <sip:b@atlanta.example.com>

From: <sip:a@atlanta.example.com>;tag=193402342

Call-ID: 898234234@agenta.atlanta.example.com

CSeq: 93809823 REFER

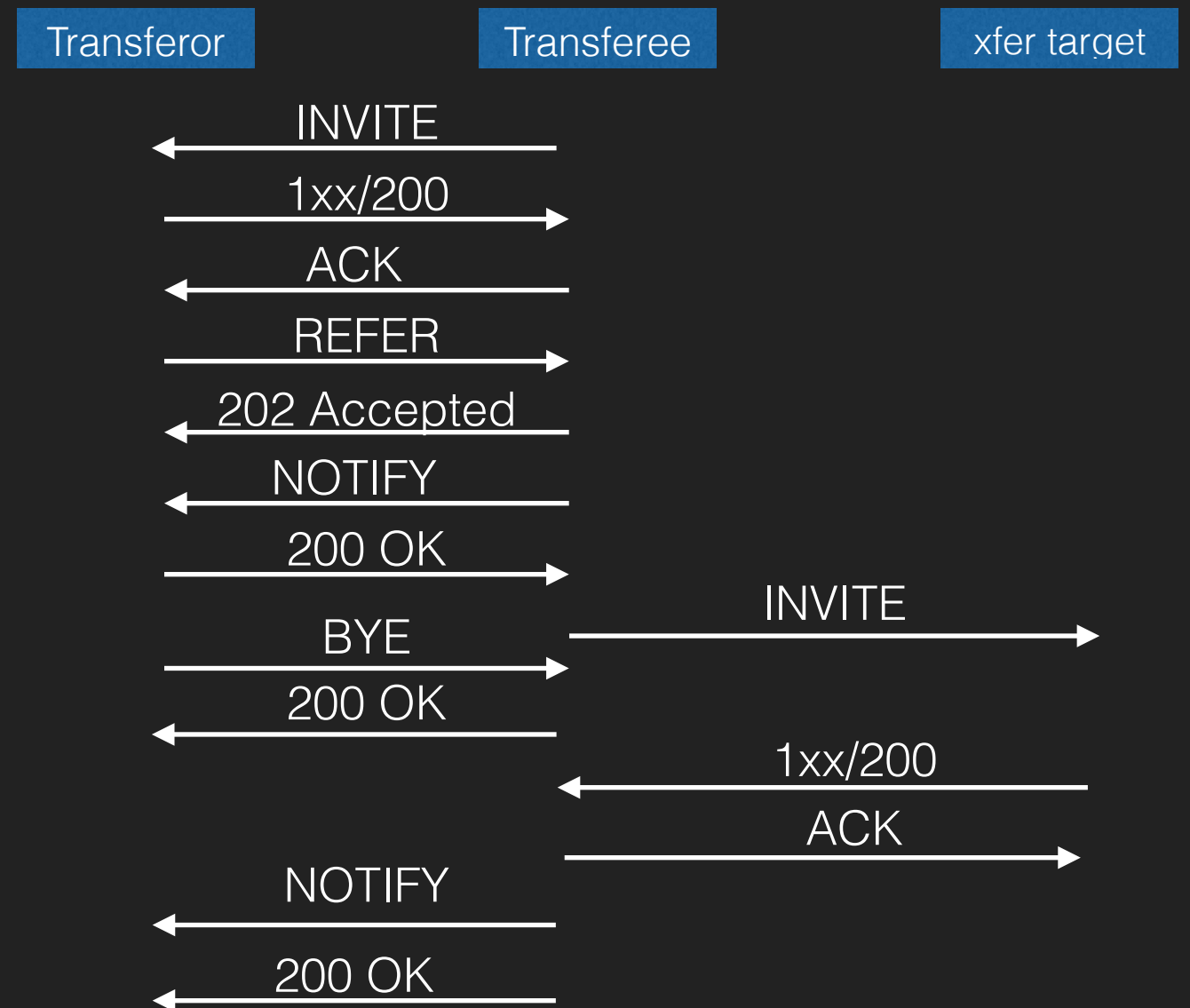
Max-Forwards: 70

Contact: sip:a@atlanta.example.com

Content-Length: 0

SIP REFER

If a REFER request is accepted, the recipient must create a subscription and notify the referring party of the request status.



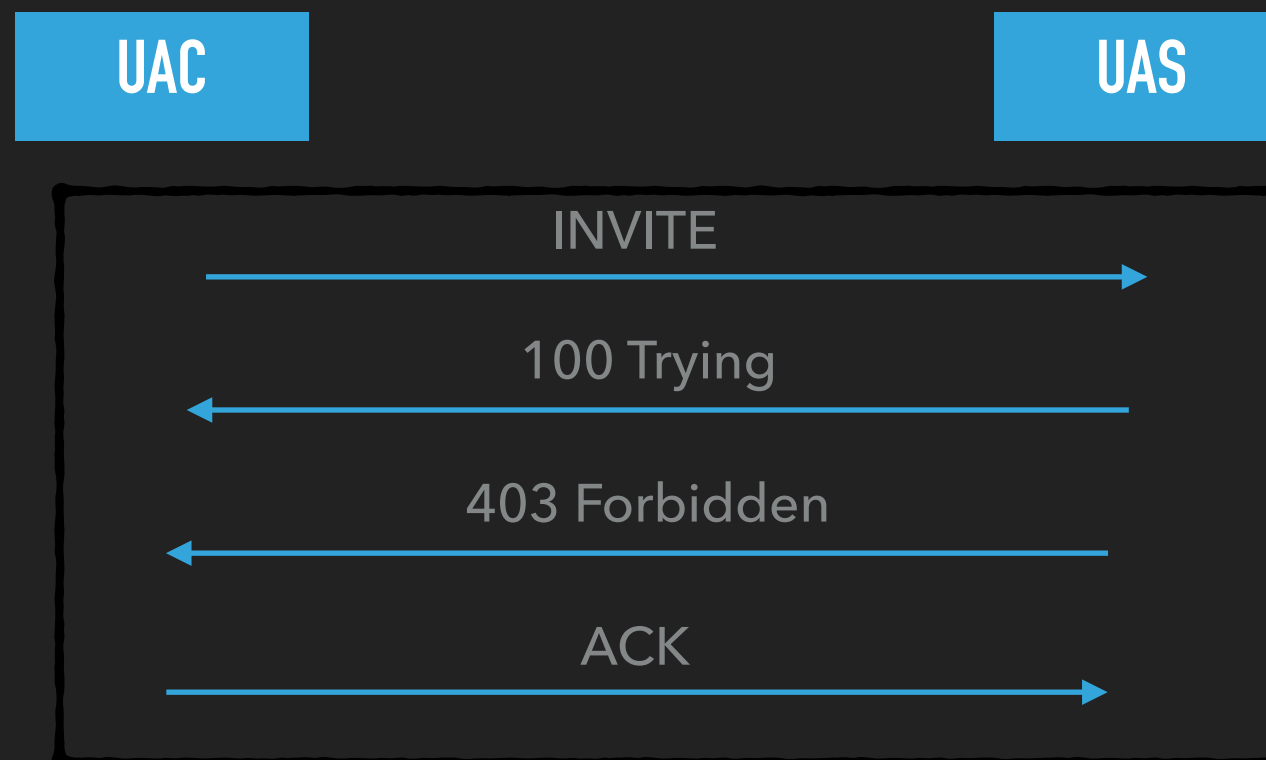
A high-angle, black and white photograph of a massive concrete dam. The dam's surface is composed of large, rectangular concrete panels with visible vertical joints. A narrow walkway runs along the top edge of the dam, where a small figure of a person is walking, providing a sense of scale. The sky is a uniform, dark grey.

SIP TRANSACTIONS AND DIALOGS

[VOIPENGINEERTRAINING.COM](https://voipengineertraining.com)

SIP TRANSACTIONS

- ▶ SIP is a transactional protocol. Everything that occurs between a UAC and a UAS consists of a series of independent messages.
- ▶ In a nutshell, a SIP transaction consists of a single request, zero or more provisional responses AND a final response.



SIP TRANSACTIONS

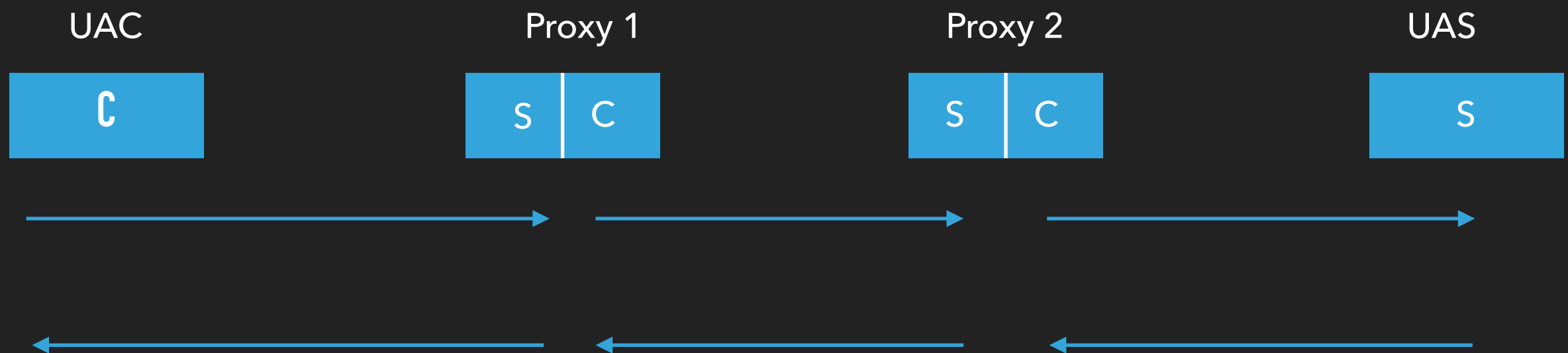
- ▶ If the transaction is an INVITE transaction, AND the final response is a 200OK, the ACK is not included within the transaction.
- ▶ The ACK... is it's own transaction.
- ▶ According to the RFC - The reason for this is rooted in the importance of delivering all 200 OK responses to an INVITE back to the UAC.



SIP TRANSACTIONS

All Transaction have a client side and server side. The client side is know as the "client transaction" and the server side is known as the "server transactions. (Yeah.. that makes sense!)

The client sends a request and the server replies with a response (aka response code). Transactions exist between User-Agents and Stateful proxies.

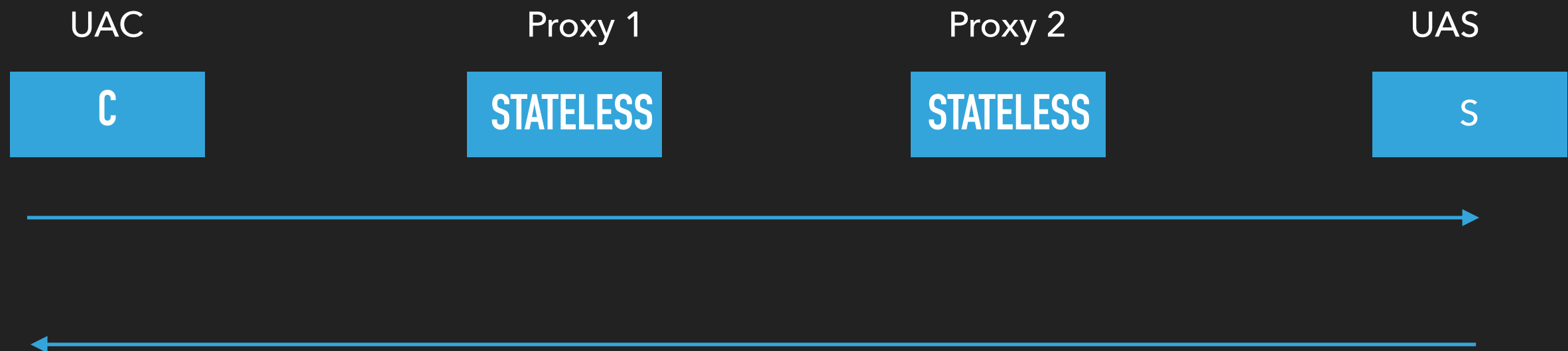


SIP TRANSACTIONS

But...

When dealing with stateless proxies, the transaction only exists between the User-Agents (or stateful proxies on each side).

In terms of SIP transactions, stateless proxies are basically transparent.

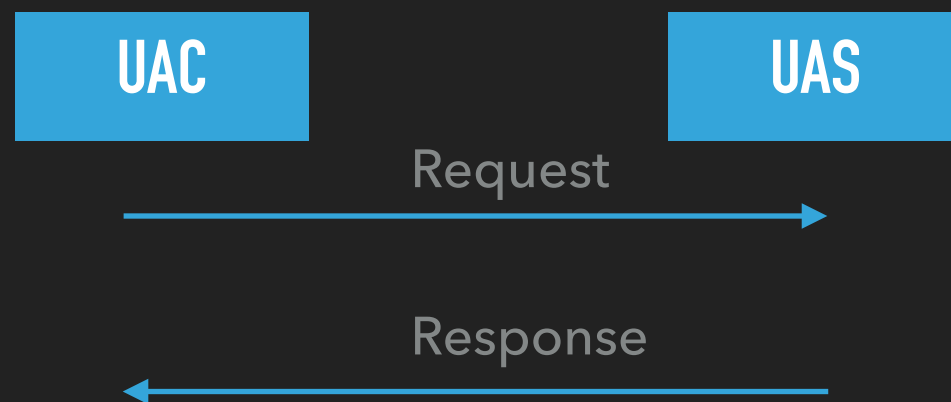


SIP DIALOG

A dialog represents a peer to peer relationship between two User-Agents that persists for some amount of time.

The dialog is responsible for facilitating the sequences of messages between the two user-agents and for proper routing of requests between the two UA's.

A dialog is identified at each User-Agent by a dialog ID. This dialog ID consists of the Call ID header value, a local tag and a remote tag.



Request from UAC

From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl

To: Bob <sip:bob@biloxi.example.com>

Call-ID: 3848276298220188511@atlanta.example.com

Response from UAS

From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl

To: Bob <sip:bob@biloxi.example.com>;tag=8321234356

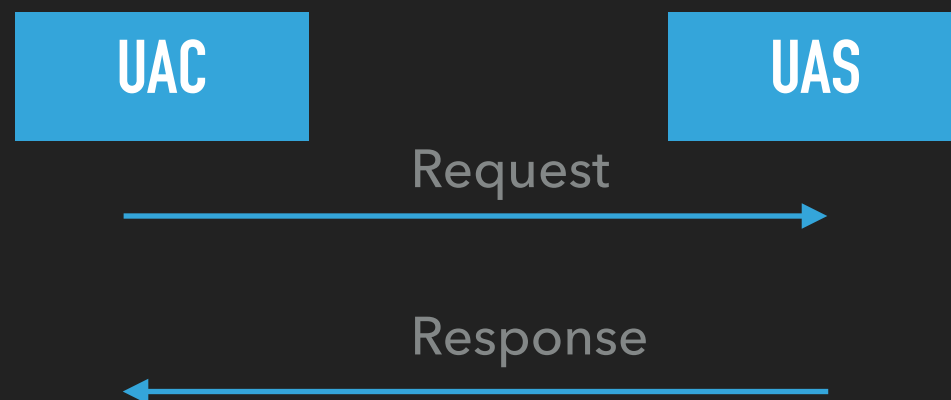
Call-ID: 3848276298220188511@atlanta.example.com

SIP DIALOG

Keep in mind - the dialog ID at each UA is NOT the same

For the UAC the Call-ID value of the dialog ID is set to the Call-ID header value, the remote tag is set to the tag in the "To" field, and the local tag is set to the value in the "From" field. These rules apply the requests and responses.

For the UAS- it's flipped a bit. The Call-ID value remains the same, however the remote tag is set to the value in the "From" field of the message, while the local tag is set to the value in the "To" field.



Request from UAC

From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl

To: Bob <sip:bob@biloxi.example.com>

Call-ID: 3848276298220188511@atlanta.example.com

Response from UAS

From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl

To: Bob <sip:bob@biloxi.example.com>;tag=8321234356

Call-ID: 3848276298220188511@atlanta.example.com

SIP DIALOG

Dialogs are created through the generation of non failure responses to requests within specified methods.

Only 2xx and 101-199 responses WITH a "To" tag where the request is INVITE will establish a dialog.

Once a dialog has been established either UA may initiate new transactions within the same dialog.

If a UAC initiates a new INVITE transaction within an existing dialog, it is known as a RE-INVITE. If a request is made where either the call-id, local tag or remote tag differ, the request is known as "out of dialog"

SUMMARY

A transaction is a SIP request, zero or more 1xx responses, a final response code plus an ACK.

It's like... buying a drink at the gas station.

SUMMARY

You walk to the counter and **request** to buy something.

The cashier acknowledges that you exist by looking at you, but the transaction hasn't been finalized yet. (**Provisional response**)

They say your total is \$5.99 (**Final Response**)

You **ACK** that by whipping out exactly \$5.99

SUMMARY

Don't forget though... If you are dealing with an INVITE transaction, the ACK is not part of the initial transaction.

SUMMARY

Dialogs occur when a successful INVITE transaction has occurred.

The dialog consists of the call-id header value, the local tag, and the remote tag.

All messages (request and response) with the same call-id, local tag and remote tag are considered "In Dialog".

SUMMARY

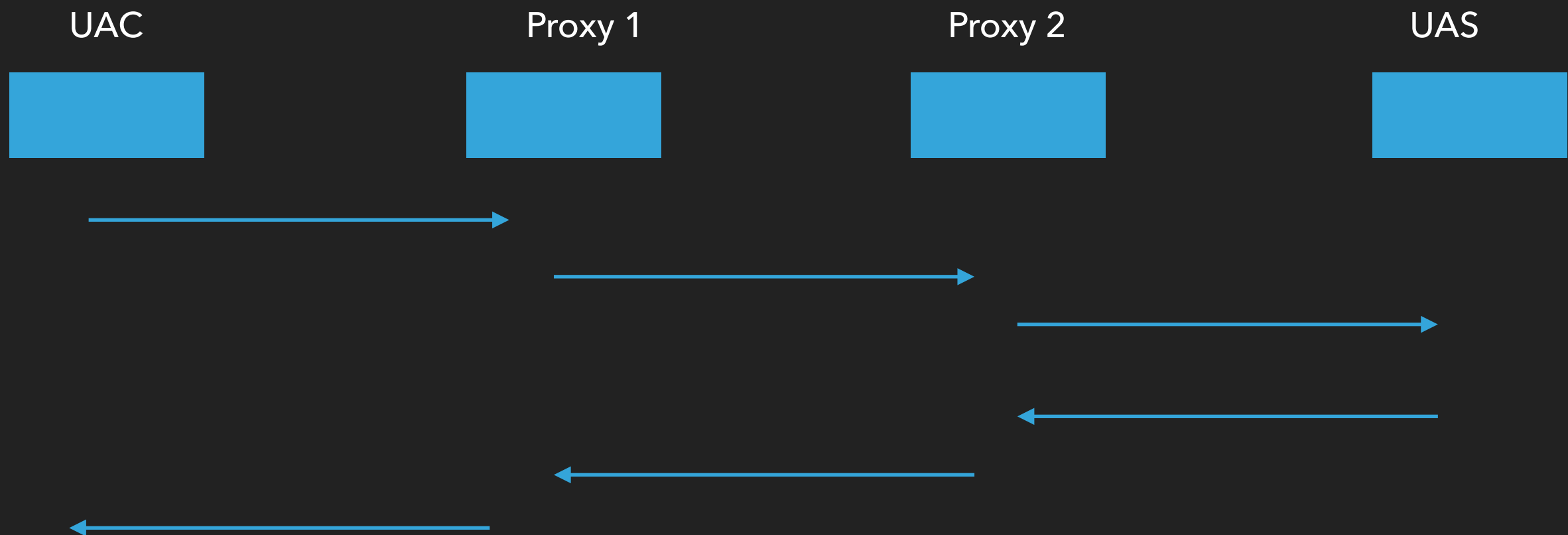
If either party initiates an INVITE *within* a dialog, we call that a “Re-INVITE”.

You will learn more about why Re-INVITEs occur later in this course.

PROXY BEHAVIOR AND ROUTING

PROXY ROUTING

SIP Proxies are elements that route requests to UASs and responses to UACs.



PROXY ROUTING

Each proxy along the path will make routing decisions and modify the request along the way.

Responses will take the same path, but in the reverse order.

PROXY ROUTING

A proxy may operate in a stateful, or stateless manner.

If a proxy is acting in a stateless mode, it simply forwards the request downstream toward a single element.

A stateless proxy makes the routing decision based upon the request

PROXY ROUTING

Once a stateless proxy forwards the request it forgets all information about the request.

A stateful proxy will remember the transaction state of each request. It uses the information when processing additional messages associated with the request.

PROXY ROUTING

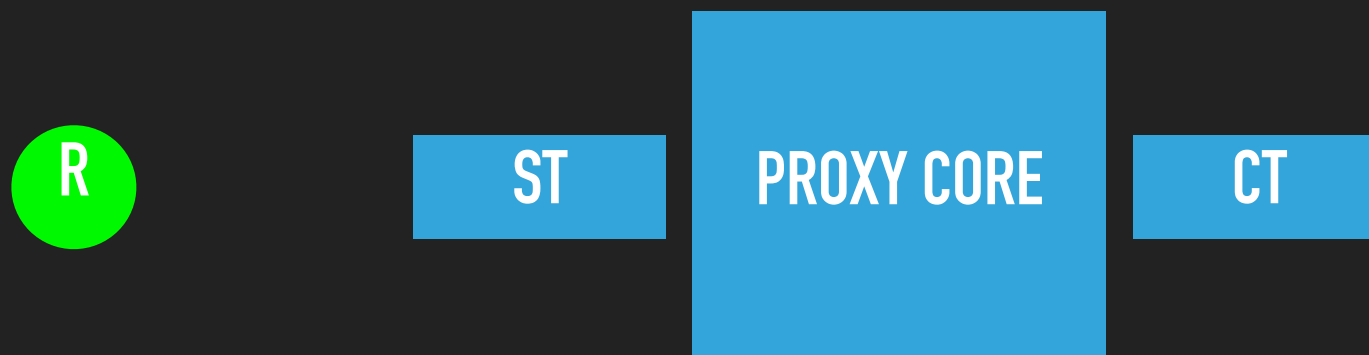
Once a stateless proxy forwards the request it forgets all information about the request.

A stateful proxy will remember the transaction state of each request. It uses the information when processing additional messages associated with the request.

STATEFUL PROXY – DEEP DIVE

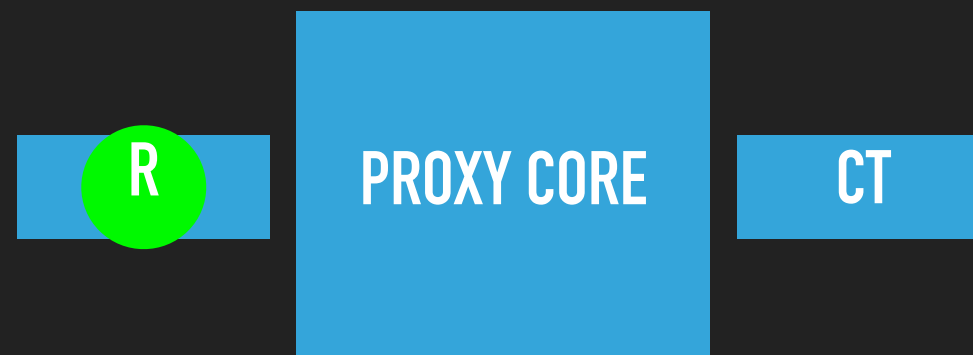
A stateful proxy is able to handle server transactions, and client transactions.

When a request is received, it is handled by the “server” side of the proxy.



STATEFUL PROXY – DEEP DIVE

Once the request has been received it is passed to the proxy core to determine how to route the request.



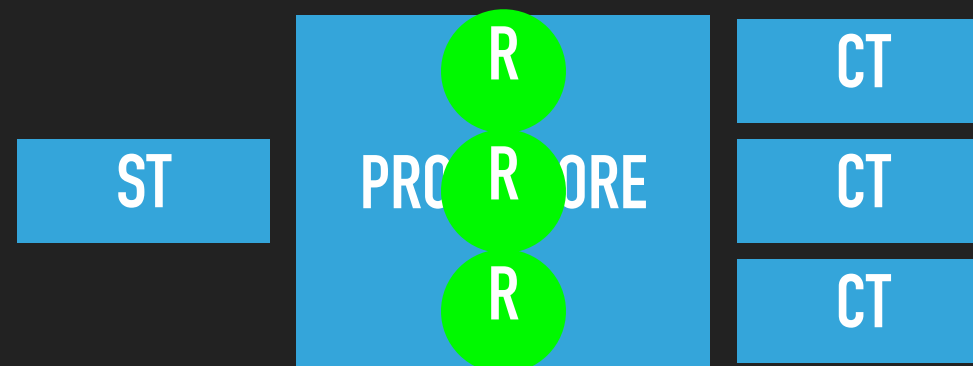
STATEFUL PROXY – DEEP DIVE

Once processed, an outgoing request is sent to the next hop location through it's own client transaction.



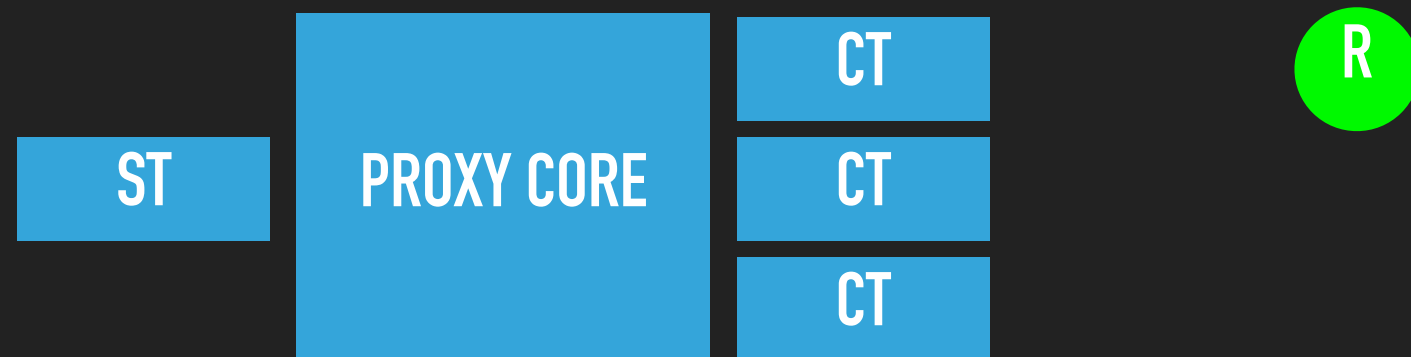
STATEFUL PROXY – DEEP DIVE

The fun thing about stateful proxies - They can fork a request. In other words, they can send the request to multiple destinations, each using their own client transactions.



STATEFUL PROXY – DEEP DIVE

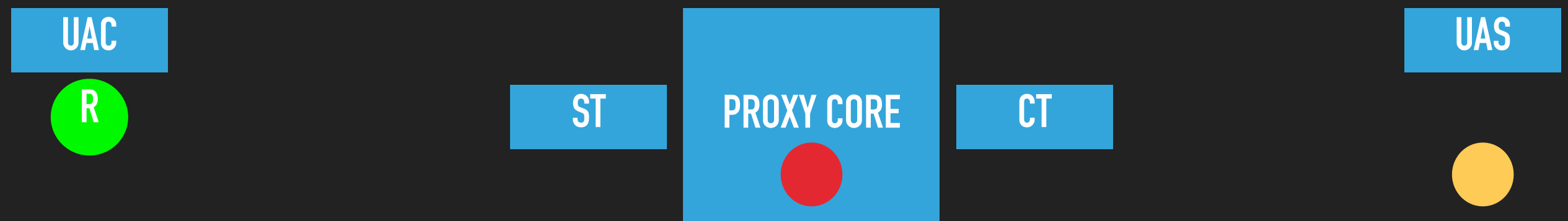
When a response is received, the proxy core will collect the response from the client transaction, process, then send the response through the server transaction



STATEFUL PROXY – DEEP DIVE

The proxy core **MUST** behave like a UAS when receiving a request. This implies that the proxy will send a 100 trying when it has received an INVITE request.

When the CT receives the 100 trying from the UAS, it does not route this back to the UAC



STATEFUL PROXY – DEEP DIVE

There are additional steps that the proxy takes before sending the request downstream.

1. The proxy must validate the request.
2. The proxy must preprocess routing information
3. Determine the target(s) for the request
4. Forward the request to each target
5. Process each response.

STATEFUL PROXY – DEEP DIVE

Let's explore these steps a little bit further.

STATEFUL PROXY – DEEP DIVE

▶ Step 1 - Validate the request.

▶ This is actually a six step sub process.

1. Check for reasonable syntax : This verifies that the request conforms the the RFC standards for formatting.
2. Check the URI scheme : If the URI scheme is unknown the proxy should generate a 416 Unsupported Scheme response
3. Check the Max-Forwards value : If the value is "0" the request will be reject with a 483 response code.
4. Check for loop detection through the VIA header : This reviews the "sent-by" value and verifies that it does not contain something that the proxy has already processes for this transaction.
5. Review the Proxy-Require header : If the header exists, examine it.
6. Review the Proxy-Authorization: Does this request require authentication before it can forward?

STATEFUL PROXY – DEEP DIVE

- ▶ Step 2 - Preprocess route information
 - ▶ The proxy will inspect the RURI of the request. If the RURI contains a value that the proxy previously placed into a Record-Route header, then the proxy will replace the RURI with the last value from the Route header.
 - ▶ Once replaced, the value will be removed from the route header.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 3 - Determine the request target(s)
 - ▶ The set of targets will be taken from either the contents of the original request, or through an abstract location service.
 - ▶ This location service could be a database lookup.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 4 - Forward the request
 - ▶ Once the proxy knows the target destinations it then....
 - ▶ Performs another set of sub processes.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 4 - Forward the request
 - ▶ For each target :
 - ▶ 1. Make a copy of the received request
 - ▶ 2. Update the request-url
 - ▶ 3. Update the Max-Forwards value
 - ▶ 4. (Optional) Add a Record-Route header
 - ▶ 5. (Optional) Add additional headers

STATEFUL PROXY – DEEP DIVE

- ▶ Step 4 - Forward the request
 - ▶ For each target :
 - ▶ 6. Postprocess the routing information
 - ▶ 7. Determine the next-hop address, port, and transport method
 - ▶ 8. Add a via header
 - ▶ 9. Add a Content-Length header (if needed)
 - ▶ 10. Forward the request.
 - ▶ 11. Set "Timer C" - this handles INVITEs that do not received a final response

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ Are you ready for it...
 - ▶ You guessed it! Another set of processes!

STATEFUL PROXY – DEEP DIVE

▶ Step 5 - Process the response

▶ When the response is received it takes 10 steps the process it.

- ▶ 1. Find the response context.
- ▶ 2. Update timer C for provisional responses.
- ▶ 3. Remove the topmost VIA header
- ▶ 4. Add the response to the response context.
- ▶ 5. Check to see if the response should be forwarded immediately.
- ▶ 6. (If necessary) Choose the best final response from the context.
- ▶ 7. Aggregate the auth header (if needed)
- ▶ 8. Rewrite the Record-Router fields
- ▶ 9. Forward the response.
- ▶ 10. Send a CANCEL request if needed

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 1. Find the response context.
 - ▶ The proxy finds the context it created before forwarding the original request using a special key.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 2. Update timer C.
 - ▶ If the response is 101-199 the proxy must reset timer C for this client transaction.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 3. Via
 - ▶ The proxy will remove the topmost via header.
 - ▶ If no Via field values remain, then the response must not be forward.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 4. Add response to context
 - ▶ The proxy will add a final response code and send toward the server transaction.
 - ▶ For example, the proxy may receive a “404 Not Found”, yet it will send a 6XX response toward the server transaction.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 5. Check response before forwarding
 - ▶ If the response is provisional (101-199) or 2XX forward it immediately.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 6. Choose the best response.
 - ▶ Often no final response was received from the downstream UAS. In this case, a SIP 408 (request timeout) will be sent to the server transaction.
 - ▶ Otherwise pick a response from the response context.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 7. Aggregate Auth Header field values.
 - ▶ If the response is 401 or 407, make sure the WWW-Authenticate header field values are copied to all responses before sending to the server transaction.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 8. Record-Route
 - ▶ If the response contains an RR header field originally added by this proxy, the proxy may choose to rewrite it forward forwarding the response.

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 9. Forward the response
 - ▶ After taking all of the processing steps, the proxy may forward the response toward the server transaction.
 - ▶ Note - proxy must not add, remove, or modify anything in the message body at this point

STATEFUL PROXY – DEEP DIVE

- ▶ Step 5 - Process the response
 - ▶ 10. Generate CANCELS
 - ▶ If the response was a final response, the proxy must generate a cancel request for all pending client transactions within this response context.

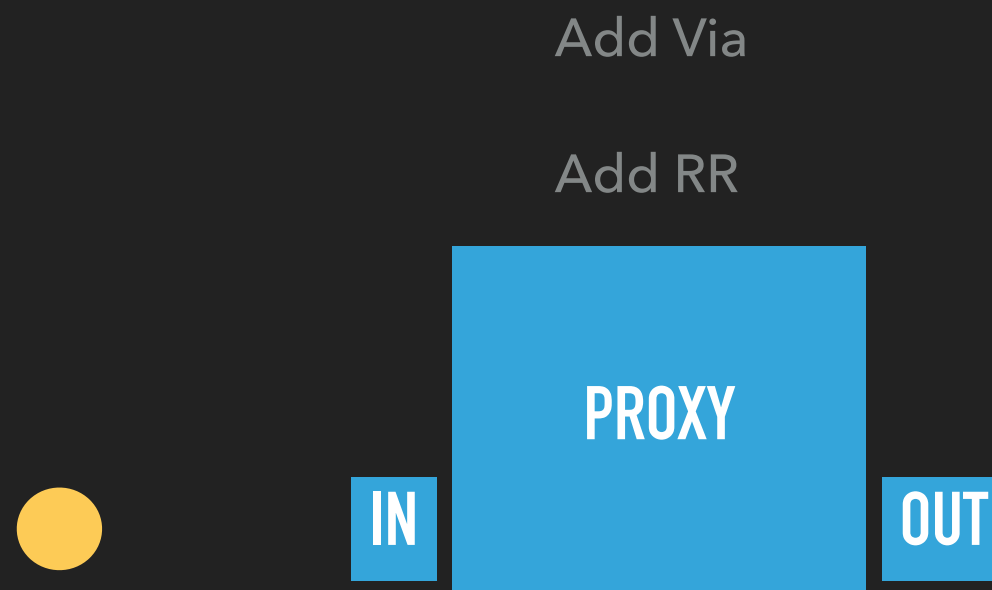
STATELESS PROXY – DEEP DIVE

- ▶ When a proxy is acting in Stateless mode, it is simply a message forwarder.



STATELESS PROXY – DEEP DIVE

- ▶ The stateless proxy add a VIA and record route headers.



STATELESS PROXY – DEEP DIVE

- ▶ Beyond that, the stateless proxy does not add, remove, or modify the request.
- ▶ This allows stateless proxies to be very quick and efficient when processing SIP messages.

THE SIP TRAPEZOID

You will often hear of something called the “SIP Trapezoid” when discussing routing.

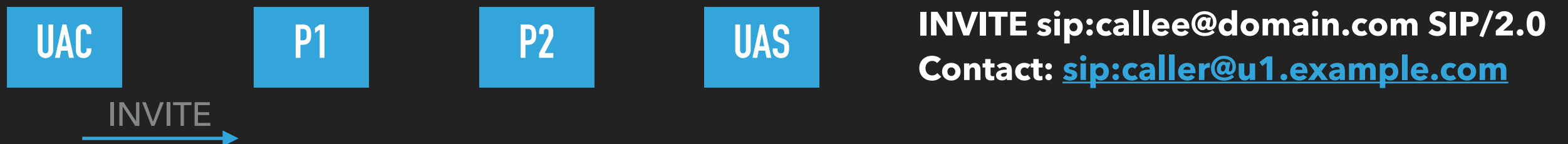
The SIP trapezoid looks something like this :



THE SIP TRAPEZOID

THE SIP TRAPEZOID

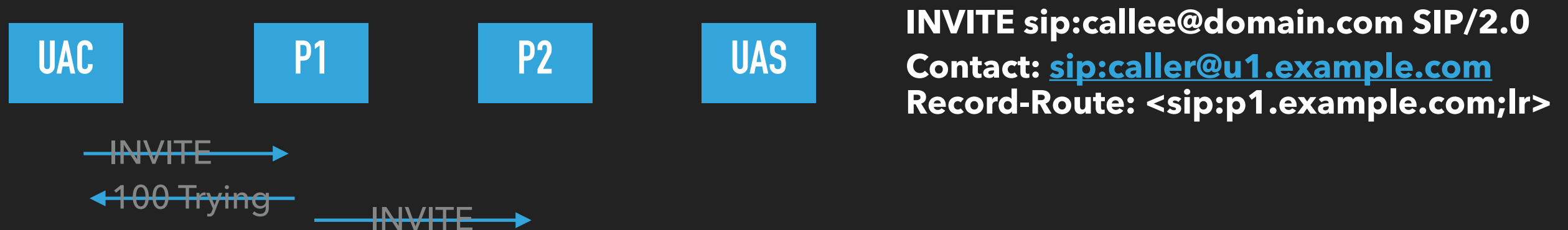
The UAC sends an INVITE to it's outbound proxy (P1)



THE SIP TRAPEZOID

P1 knows that it is not responsible for "[domain.com](#)" so does a DNS lookup. It determines that P2 is responsible for this domain and sends INVITE there.

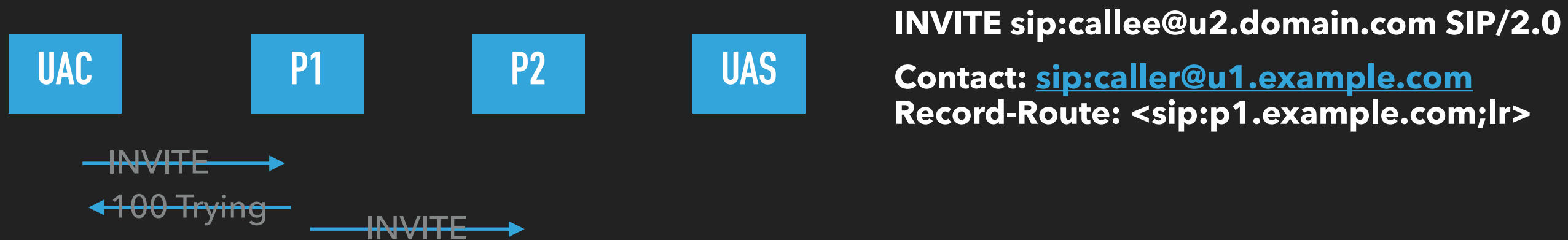
P1 also adds a Record-Route Header.



THE SIP TRAPEZOID

P2 receives the request, and says "yes I am responsible for ["domain.com"](#). I know where this user is. They are located at ["u2.domain.com"](#).

The RURI is now modified.



THE SIP TRAPEZOID

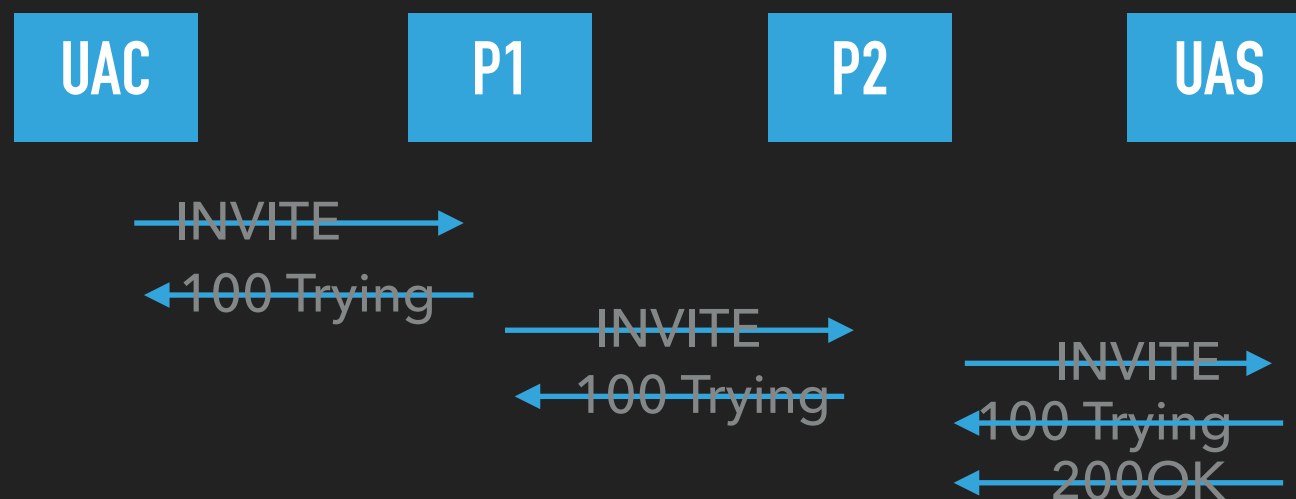
P2 then says “I do not see a route header, therefore I will send this to the address in the RURI”. Before doing that, P2 adds a Record-Route header as well.



THE SIP TRAPEZOID

The user "callee" located at the UAS says. "Ohh this is for me. I'm going to answer immediately with a 200OK."

The UAS also adds a route set (through a Route header), then sends the response to P2



SIP/2.0 200 OK

Contact: sip:callee@u2.domain.com

Record-Route: <sip:p2.domain.com;lr>

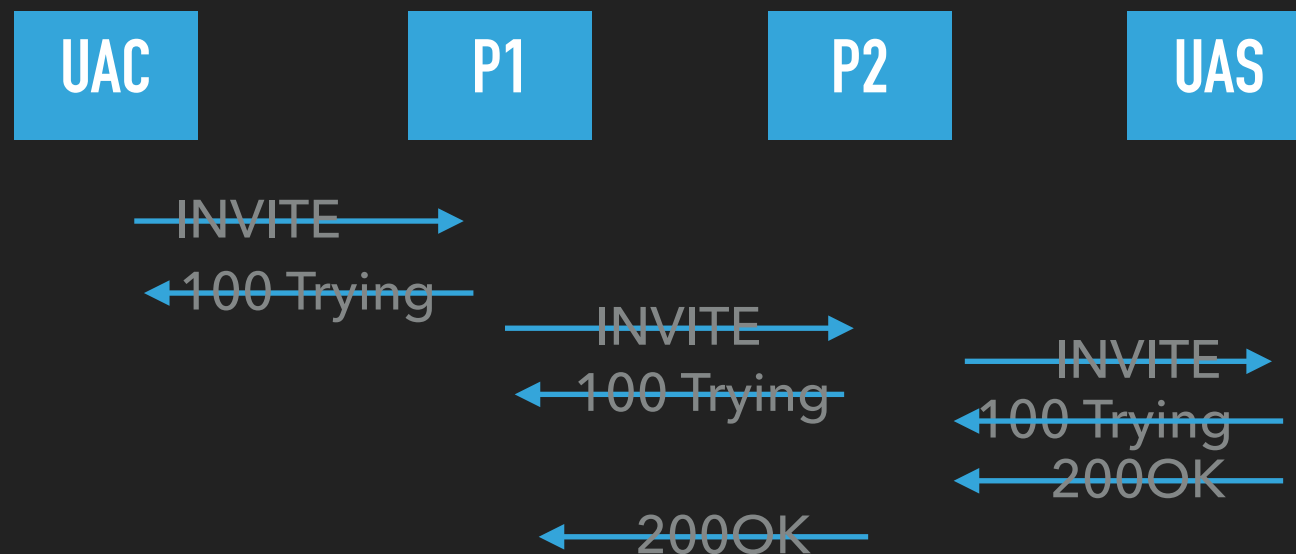
Record-Route: <sip:p1.example.com;lr>

Route: <sip:p2.domain.com;lr> ,

<sip:p1.example.com;lr>

THE SIP TRAPEZOID

P2 receives the response, and forwards this to P1



SIP/2.0 200 OK

Contact: sip:callee@u2.domain.com

Record-Route: <sip:p2.domain.com;lr>

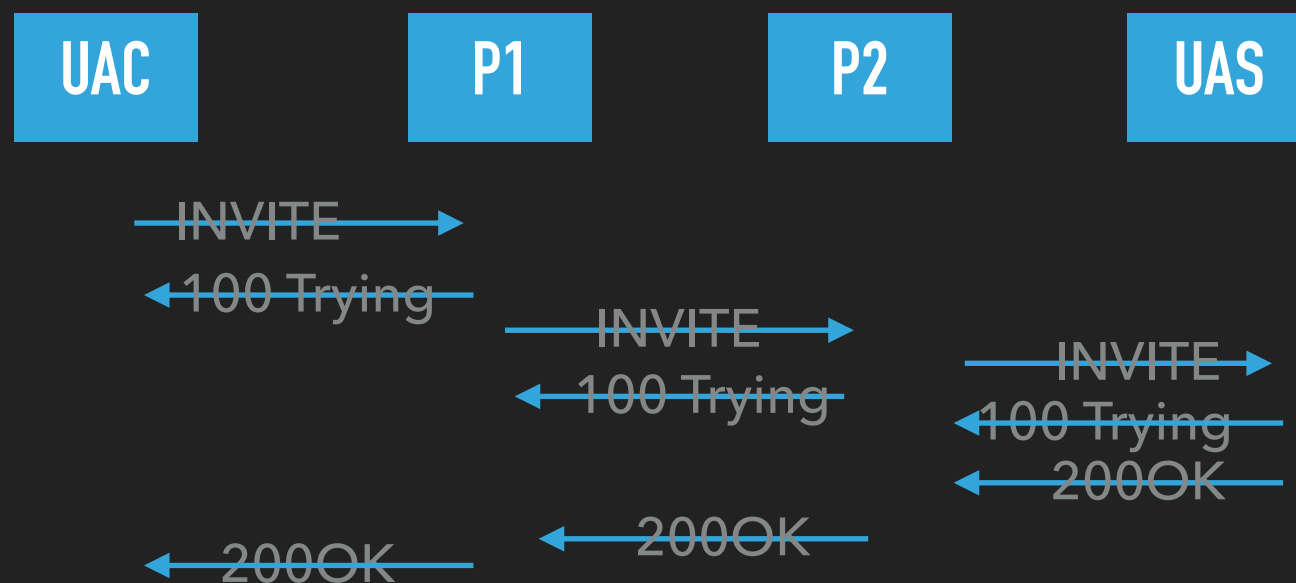
Record-Route: <sip:p1.example.com;lr>

Route: <sip:p2.domain.com;lr> ,

<sip:p1.example.com;lr>

THE SIP TRAPEZOID

P1 receives the response and forwards this to the UAC



SIP/2.0 200 OK

Contact: sip:callee@u2.domain.com

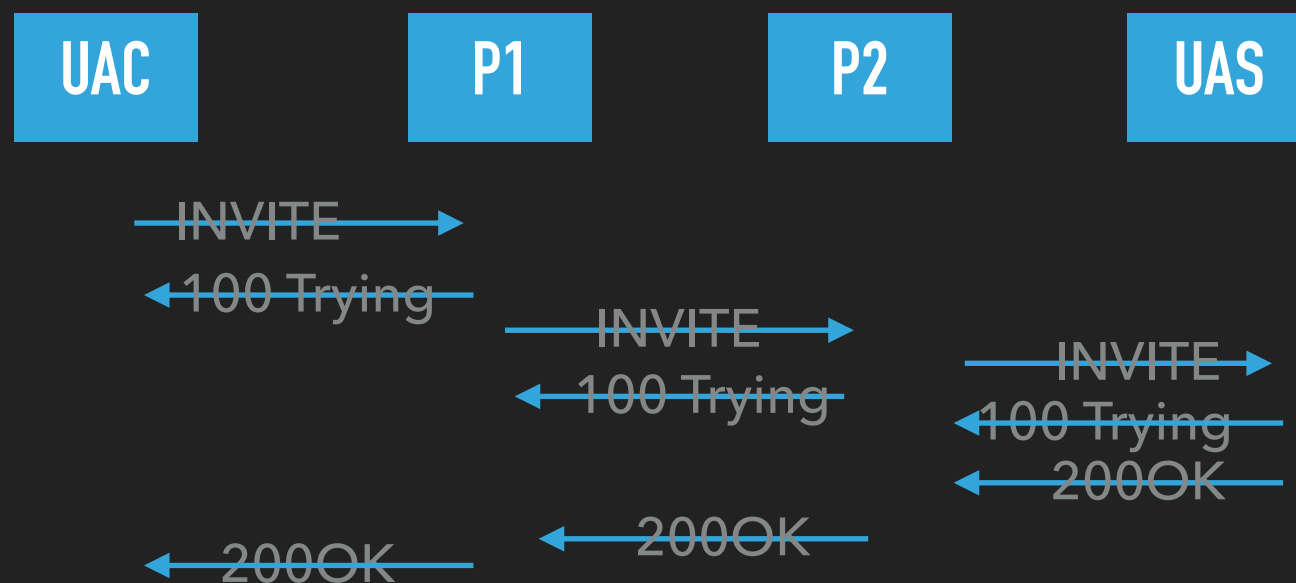
Record-Route: <sip:p2.domain.com;lr>

Record-Route: <sip:p1.example.com;lr>

Route: <sip:p2.domain.com;lr>,
<sip:p1.example.com;lr>

THE SIP TRAPEZOID

The UAC receives this 200OK and says. "I see that contact header (remote target) is "callee@u2.domain.com"".



SIP/2.0 200 OK

Contact: sip:callee@u2.domain.com

Record-Route: <sip:p2.domain.com;lr>

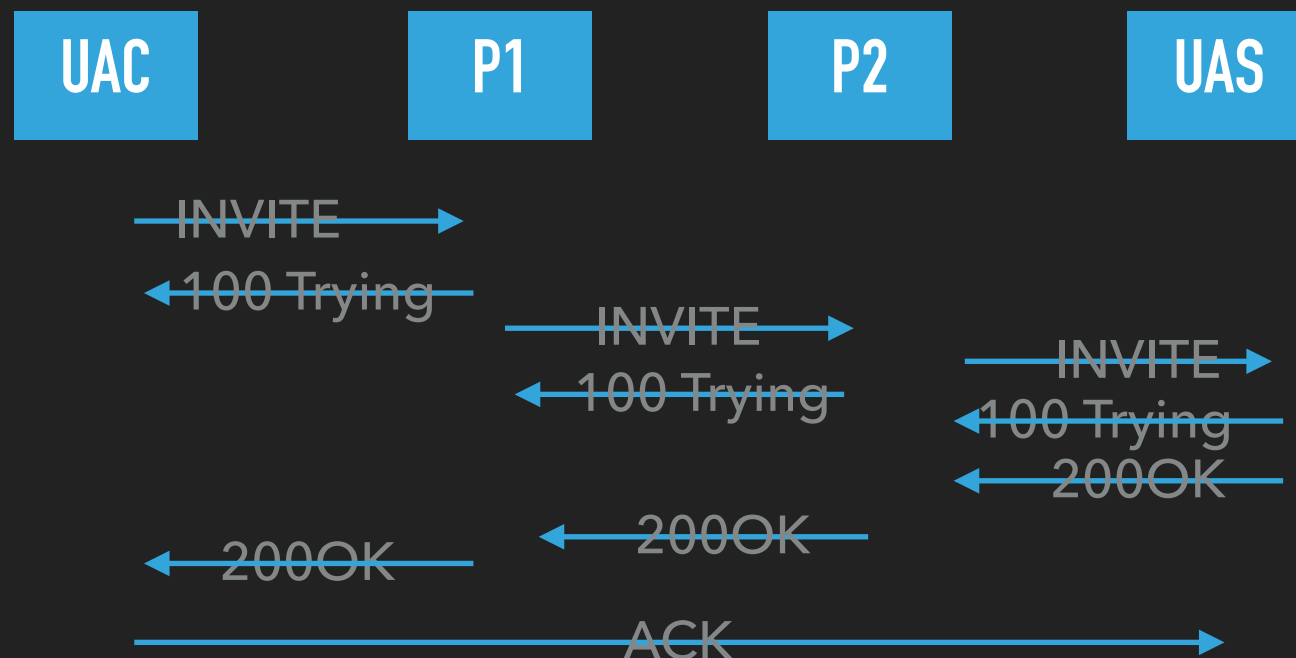
Record-Route: <sip:p1.example.com;lr>

Route: <sip:p2.domain.com;lr>, <sip:p1.example.com;lr>

THE SIP TRAPEZOID

Like a good SIP client, the UAC wants to create an ACK. Before it does that, it notices that all elements in the route-set, contain the 'lr' parameter.

So it constructs the ACK as shown and sends it directly to the UAS

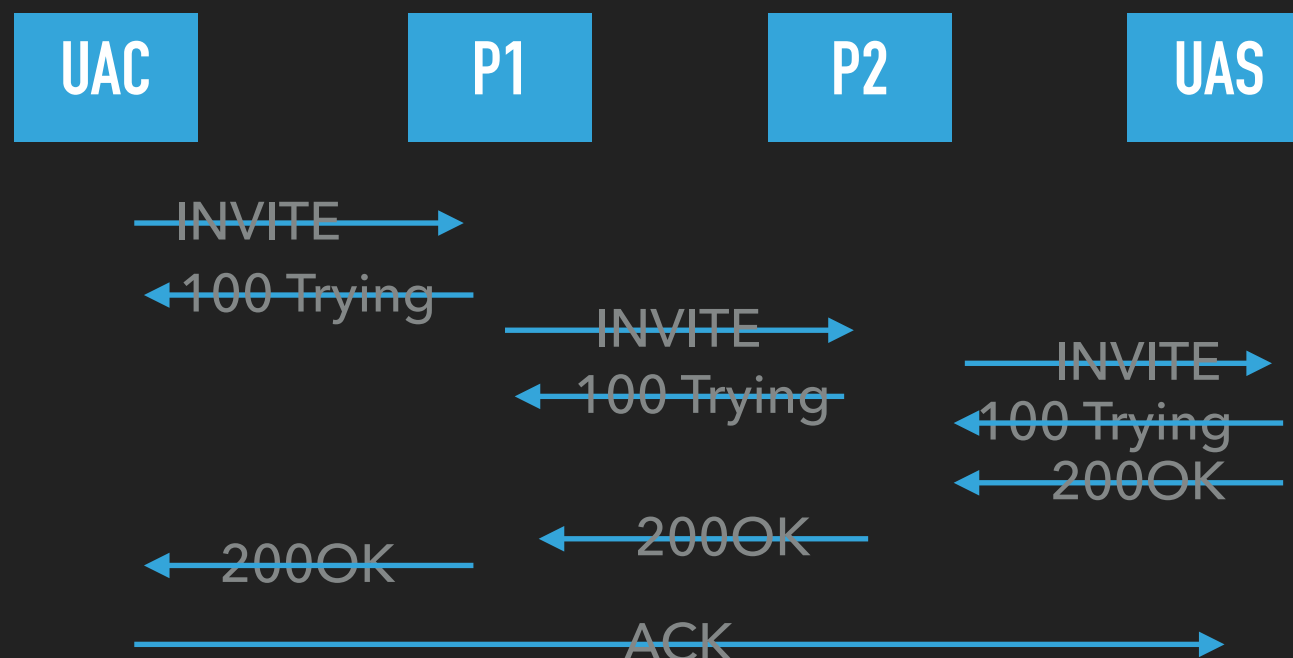


ACK sip:callee@u2.domain.com SIP/2.0
Route: <sip:p1.example.com;lr>, <sip:p2.domain.com;lr>

THE SIP TRAPEZOID

Wait... Why did it do that?

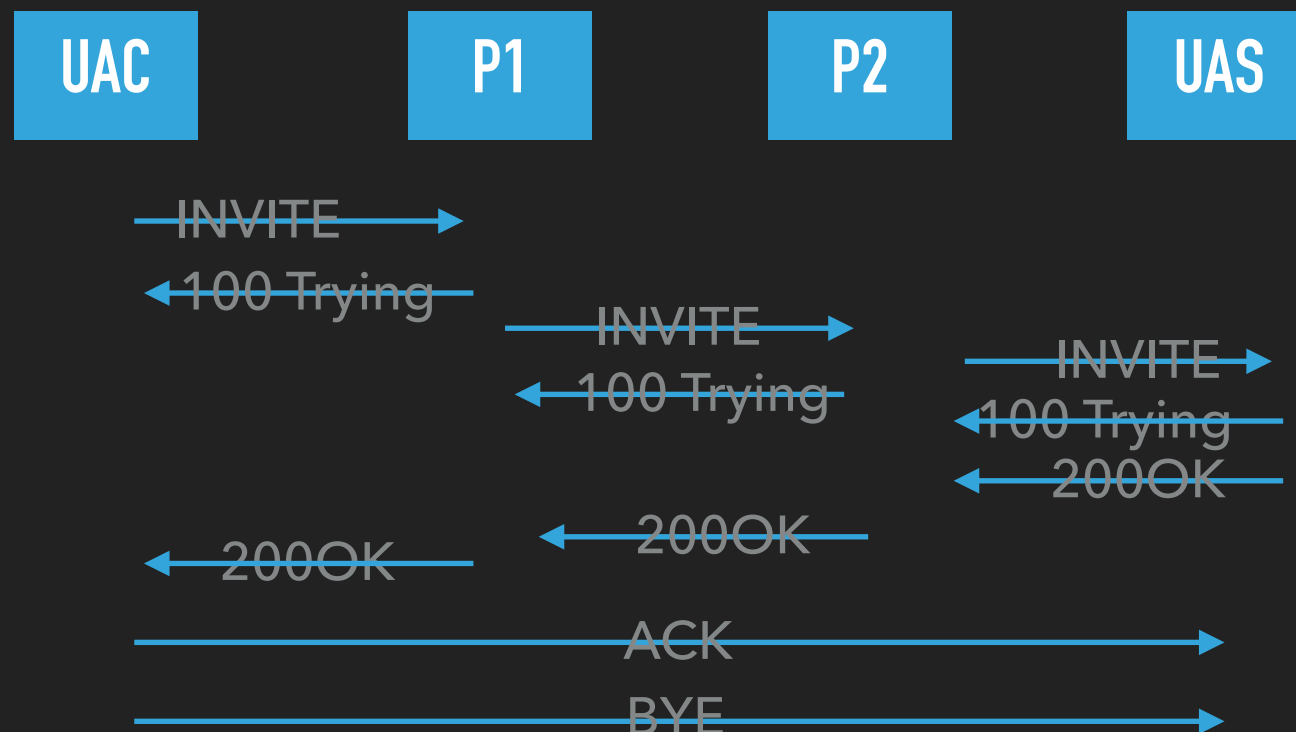
Well the UAC knows the proper remote target is "callee@u2.domain.com". As the loose routing (lr) param is in place, the UAC knows that it does not need to send the request back through the loose routing proxies. Instead, it can send directly to the UAS.



ACK sip:callee@u2.domain.com SIP/2.0
Route: <sip:p1.example.com;lr>, <sip:p2.domain.com;lr>

THE SIP TRAPEZOID

When it is time to end this dialog the UAC sends a BYE directly to the UAS.



BYE sip:callee@u2.domain.com SIP/2.0
Route: <sip:p1.example.com;lr>,
<sip:p2.domain.com;lr>

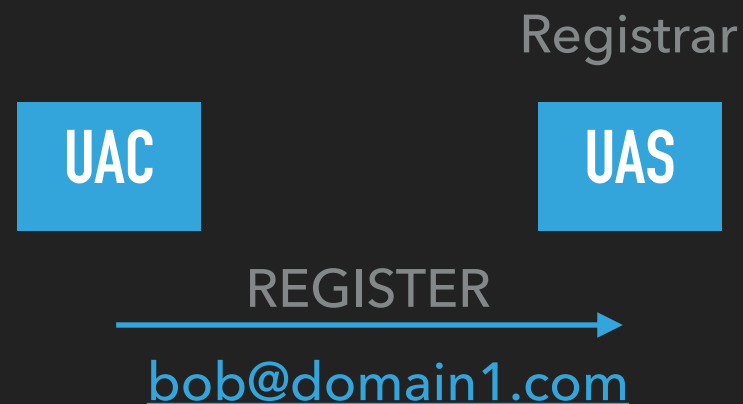
SIP

REGISTRATIONS

SIP REGISTRATIONS

In the world of SIP, User Agents may frequently change IP addresses. This creates an issue when one user wants to call another and they cannot be found.

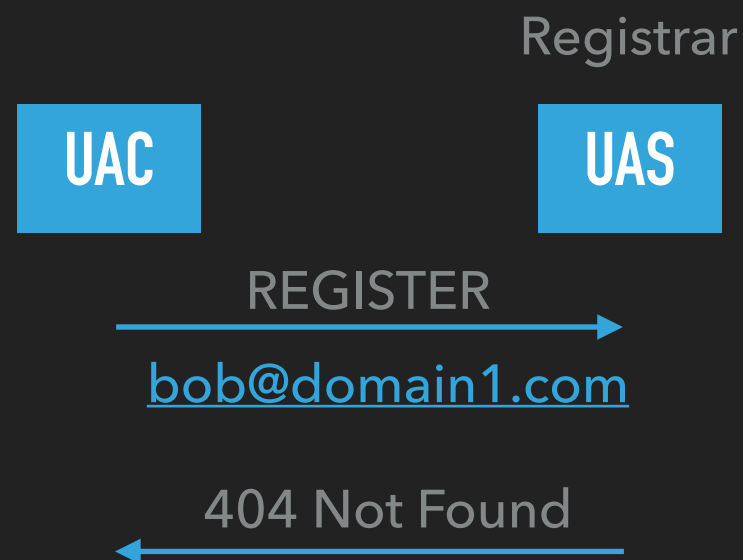
To combat this, SIP allows users to REGISTER with a special type of UAS known as a registrar.



SIP REGISTRATIONS

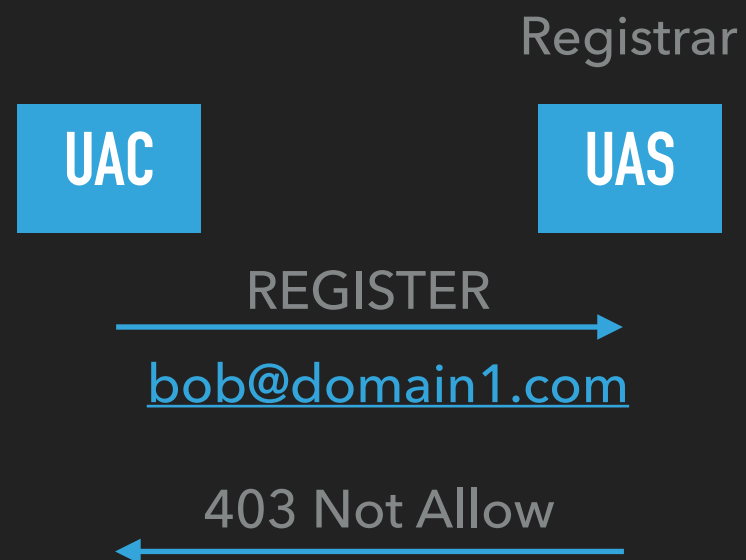
When the registrar receives the REGISTER request, it can take a number of steps.

First the Registrar may perform a lookup to see if this user even exists. If not, the Registrar will most likely send a 404 Not Found in response.



SIP REGISTRATIONS

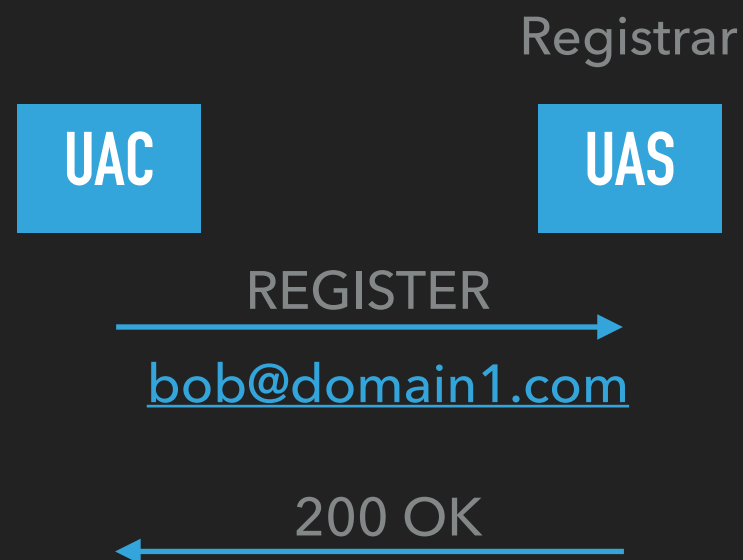
It's also possible that the registrar is configured to not accept requests from this specific user or UAC. In this case, it's likely that the registrar will send a 403 response



SIP REGISTRATIONS

In the event that this user is permitted to register, the registrar will respond with a 200OK.

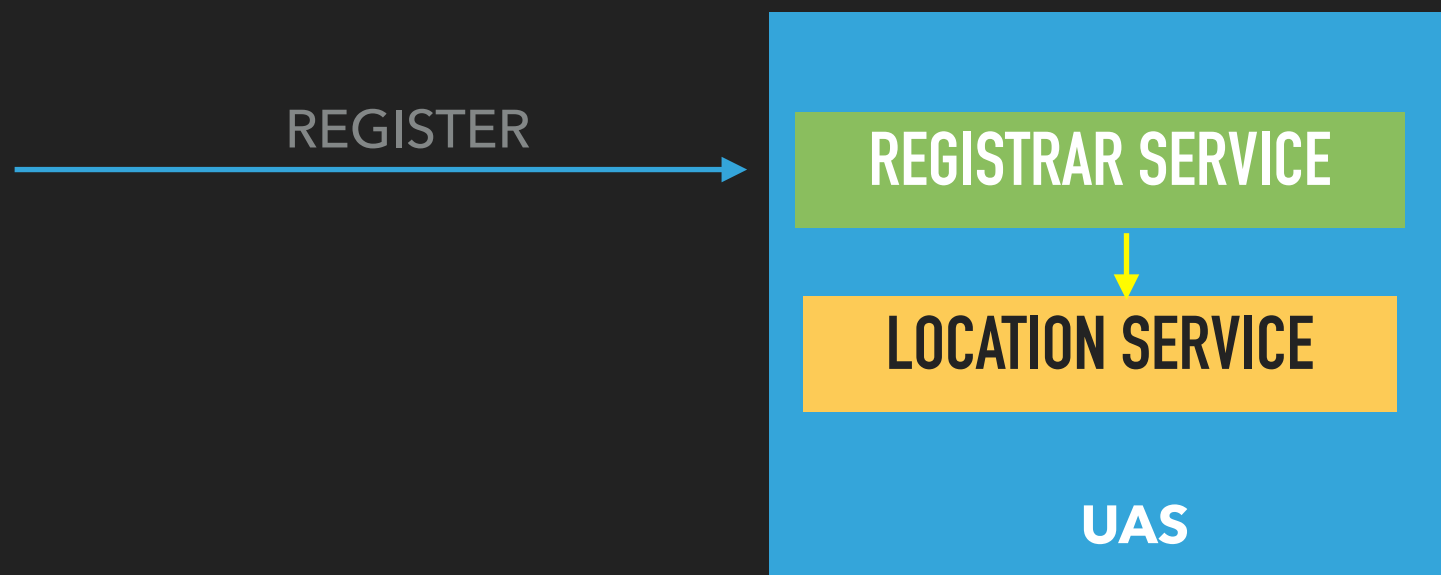
Of course, it could respond with a 401 Unauthorized as well before allowing registration to take place.



SIP REGISTRATIONS

In the background of the registrar server it communicates with an abstract service known as the "Location Service".

Keep in mind - the registrar and location service COULD be on the same server, or as diverse elements.



SIP REGISTRATIONS

Once a user has successfully registered, a binding is created for this Address or Record (AOR) .

The AOR is taken from the "To" header.

The information on how to reach is user, is taken from the "Contact" header if present.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

```
SIP/2.0 200 OK
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashd92
;received=192.0.2.201
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>;tag=37GkEhw16
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 2 REGISTER
Contact: <sip:bob@domain1.com>;expires=3600
Content-Length: 0
```



Host : domain.com:5060

Registration Time : Fri, June 2 2016 22:00

Username : Bob

State: Registered

SIP REGISTRATIONS

Now keep in mind, a REGISTER request can add, remove, or simply query an existing binding.

Although a REGISTER request and response contain the necessary components of a SIP dialog, a dialog is not created.

Record-Route field has no meaning when it comes to Registrations. In fact a registrar and UAC must ignore them if added

SIP REGISTRATIONS

Let's take a deeper look at the components of a REGISTER request message.

SIP REGISTRATIONS

Request-URI : The RURI names the domain of the location service for which registration is meant.

But notice the username and "@" components of the RURI are missing.

REGISTER sip:domain1.com SIP/2.0

Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7

Max-Forwards: 70

From: Bob <sip:bob@domain1.com>;tag=a73kszlfl

To: Bob <sip:bob@domain1.com>

Call-ID: 1j9FpLxk3uxtm8tn@domain1.com

CSeq: 1 REGISTER

Contact: <sip:bob@domain1.com>

Content-Length: 0

SIP REGISTRATIONS

From Header: The From header will contain the AOR of the person responsible for the registration. The value is the same as the To header unless it originated from a third party registration

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

SIP REGISTRATIONS

To Header : This contains the AOR of the registration that is to be created, queried or modified.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
```

To: Bob <sip:bob@domain1.com>

```
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

SIP REGISTRATIONS

Call-ID: All Registrations from a UAC should use the same Call-ID value when REGISTER requests are sent to the same registrar.

Often we will have a 401 Unauthorized challenge occur. In the event that this happens, the Call ID from REG attempt 1 and 2 will remain the same.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
```

Call-ID: 1j9FpLxk3uxtm8tn@domain1.com

```
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

SIP REGISTRATIONS

CSeq - Each time a REGISTER request with the same Call-ID is sent by a UAC, it must increase the value of this by 1.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com

CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```



SIP REGISTRATIONS

Contact : The REGISTER request MAY contain a contact header field with zero or more address bindings.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
```

Contact: <sip:bob@domain1.com>

```
Content-Length: 0
```

SIP REGISTRATIONS

Tip : A UAC MUST NOT send another register request (ie - one with a new contact header value) until they have received a final response from the previous REGISTER attempt. Or, if the previous attempt timed out.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
```

Contact: <sip:bob@domain1.com>

```
Content-Length: 0
```

SIP REGISTRATIONS

Tip : A UAC MUST NOT send another register request (ie - one with a new contact header value) until they have received a final response from the previous REGISTER attempt. Or, if the previous attempt timed out.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
```

Contact: <sip:bob@domain1.com>

Content-Length: 0

SIP REGISTRATIONS

Registration bindings are only valid for a certain amount of time. There are a two methods that a UAC can use to suggest an expiration interval of the binding.

SIP REGISTRATIONS

Method 1 : When the client sends a REGISTER request it can add an Expires header field.

Method 2 : The client can added an “expires” header parameter to the contact field.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Expires: 3600
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>;expires=3600
Content-Length: 0
```

SIP REGISTRATIONS

Or course the client could user neither method.

In that situation the client is telling to server to choose the value.

You will see the server response by setting an expires parameter in the Contact header

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

```
SIP/2.0 200 OK
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashd92
;received=192.0.2.201
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>;tag=37GkEhw16
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 2 REGISTER
Contact: <sip:bob@domain1.com>;expires=3600
Content-Length: 0
```

SIP REGISTRATIONS

To remain REGISTERED a UAC must send another REGISTER request before the expiry timer is up.

As discussed earlier - the REGISTER refresh request will contain the same Call-ID, and increment the CSeq value.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

SIP REGISTRATIONS

Tearing down a binding:

There are times where a UAC may wish to gracefully unregister. One example is when a physical phone is rebooted, or a soft phone is shutdown.

The gracefully tear down the registration the UAC send a REGISTER request with “expires=0” in the contact header, or through an “Expires” header field with the zero value.

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Contact: <sip:bob@domain1.com>;expires=0
Content-Length: 0
```

```
REGISTER sip:domain1.com SIP/2.0
Via: SIP/2.0/UDP domain1.com:5060;branch=z9hG4bKnashds7
Max-Forwards: 70
From: Bob <sip:bob@domain1.com>;tag=a73kszlfl
To: Bob <sip:bob@domain1.com>
Call-ID: 1j9FpLxk3uxtm8tn@domain1.com
CSeq: 1 REGISTER
Expires: 0
Contact: <sip:bob@domain1.com>
Content-Length: 0
```

SIP REGISTRATIONS

Jumping back to the UAS (the registrar) that receives these requests, it runs through series of steps each time a REGISTER request is received.

SIP REGISTRATIONS

Step 1: The registrar looks at the RURI to determine if it has access to the bindings for the requested domain. If not, and the registrar does not act as a proxy server, it can reject the request.

SIP REGISTRATIONS

Step 2 : The registrar will inspect and process all values in the "Require" header field.

SIP REGISTRATIONS

Step 3 : The registrar SHOULD authenticate the UAC. If no authentication method has been configured the registrar will take the From address value and assume that to be the originator of the request.

SIP REGISTRATIONS

Step 4 : The registrar should determine if the authenticated user is actually authorized to make changes to bindings.

If the user is not, the registrar MUST send a SIP 403 Forbidden as a response.

SIP REGISTRATIONS

Step 5 : The registrar examines the AOR taken from the To header.

If the AOR is not valid for the the domain seen in the RURI, the registrar must send a SIP 404 Not found in response.

SIP REGISTRATIONS

Step 6 : The registrar looks for a Contact header field. If not, it skips to the final step in the process.

If the Contact header field does exist, it will review the Call-ID and make sure this is the same Call-ID as the existing binding (if present).

IF a binding exists and the Call-ID is different, the old binding is removed.

SIP REGISTRATIONS

Step 7 - Each Contact address is now processed. This is where the registrar determines who is responsible for setting the expiration interval.

SIP REGISTRATIONS

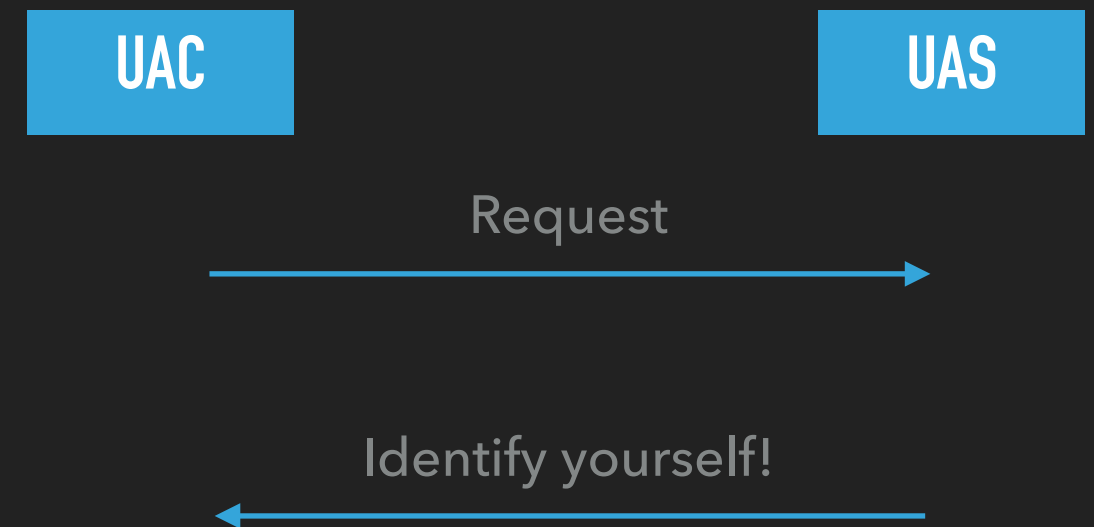
Step 8 - The registrar can now return a 200 OK as a response.

SIP

AUTHENTICATION

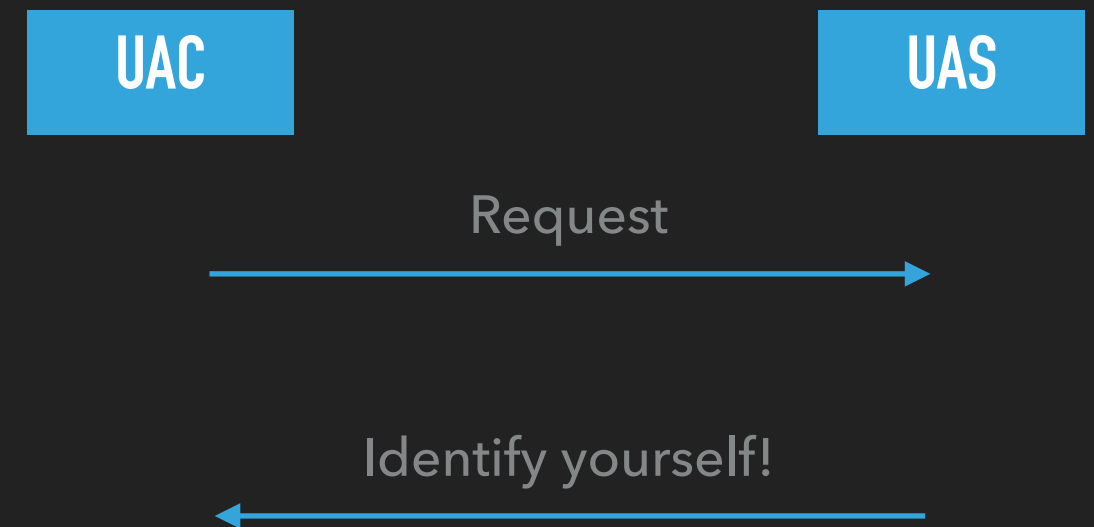
SIP AUTHENTICATION

- ▶ SIP provides a stateless, challenge-based authentication
- ▶ Based on the same authentication methods used by HTTP
- ▶ Anytime a UA (or proxy) receives a request it MAY challenge it to issue the identity of the originator.
- ▶ A proxy MUST use a SIP 407 to issue a challenge. A UAS, registrar and redirect server issue a 401 response.
- ▶ Once the originator is identified the UA challenging SHOULD determine if the user is authorized to make the request.



SIP AUTHENTICATION

- ▶ Under RFC 2543 servers could accept requests from UACs that wished to use "Basic" authentication.
- ▶ Under RFC 3261 Servers must NOT accept credentials using "Basic" authentication.
- ▶ Under RFC 3261 - servers also must not challenge an auth request where basic is used.



SIP AUTHENTICATION

- ▶ All but two SIP request methods can be challenged.
- ▶ Cancel is not challenged because they cannot be resubmitted
- ▶ ACK is not challenged. If an INVITE was challenged, the UAC MUST send the authorization fields in the ACK, that it used for the original INVITE.
- ▶ Fair game on everything else

SIP AUTHENTICATION

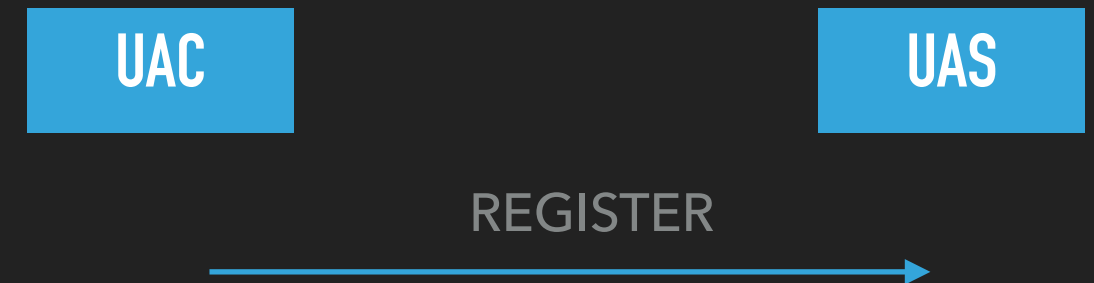
- ▶ Since Basic is out of the question, we are going to focus on **Digest authentication.**

SIP AUTHENTICATION

- ▶ Digest authentication works by challenging the original request.
- ▶ When the request is challenged, the UAS will reply with a nonce and domain, presented within the WWW-Authenticate header.
- ▶ When the request originator responds to the challenge, the challenge response will contain an md5 checksum (hash) or various parameters, such as the username, password, realm etc. These are contained in the "Authorization Header"

SIP AUTHENTICATION

```
REGISTER sip:ss2.wcom.com SIP/2.0
Via: SIP/2.0/UDP there.com:5060
From: LittleGuy <sip:UserB@there.com>
To: LittleGuy <sip:UserB@there.com>
Call-ID: 123456789@there.com
CSeq: 1 REGISTER
Contact: <sip:UserB@110.111.112.113>
Content-Length: 0
```



- ▶ Notice that no credentials are sent in this request

SIP AUTHENTICATION

SIP/2.0 401 Unauthorized

Via: SIP/2.0/UDP there.com:5060

From: LittleGuy <sip:UserB@there.com>

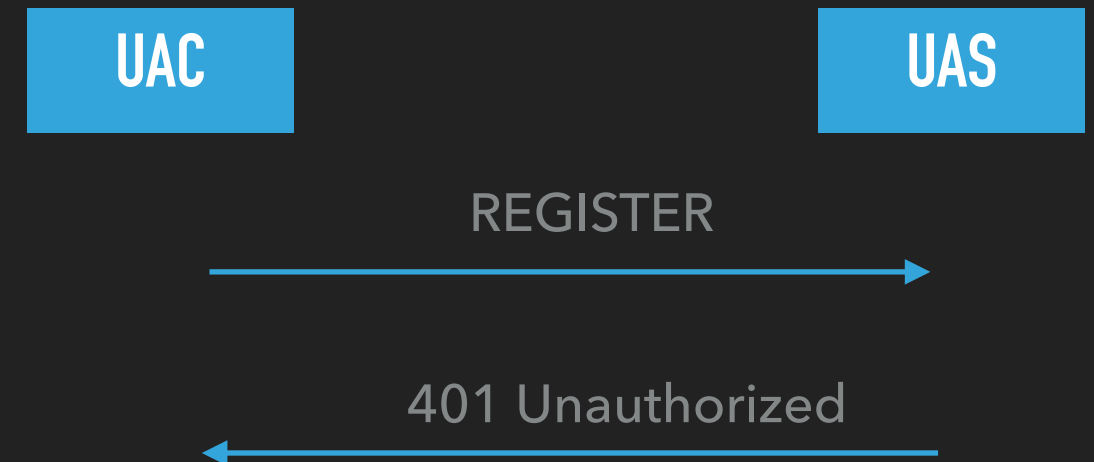
To: LittleGuy <sip:UserB@there.com>

Call-ID: 123456789@there.com

CSeq: 1 REGISTER

**WWW-Authenticate: Digest realm="fakeserver",
domain="sip:sip.fakeserver.net",
nonce="ea9c8e88df84f1cec4341ae6cbe5a359"**

Content-Length: 0



- ▶ Notice the WWW-Authenticate header

SIP AUTHENTICATION

- ▶ This is where the hashing algorithm comes into place.
- ▶ When the UAC receives the challenge, it needs to compute the response properly.
- ▶ There are two algorithms : one where qop (quality of protection) is requested, and one where.. it is not.

SIP AUTHENTICATION - WITH QOP

- ▶ There are three hashes that we are going to create.
 - ▶ Hash 1 is the hash of the username:realm:password
 - ▶ Hash1 =
md5hex(UserB:fakeserver:password123")
 - ▶

```
[Keiths-MacBook-Air:~ keithcroxford$ md5 -s "UserB:fakeserver:password123"  
MD5 ("UserB:fakeserver:password123") = b2310cc50edf3e2616d51ba7232db3d7]
```
- ▶ Username : UserB
 - ▶ Password : password123
 - ▶ realm : fakeserver
 - ▶ h1 =
b2310cc50edf3e2616d51ba7232db3d7

SIP AUTHENTICATION – WITH QOP

- ▶ Hash2 is going to be a hash of the Method used and the URI.
- ▶ We know that this is a REGISTER Method. the the URI is going to be what we gleaned from the domain in the 401 .
- ▶ Hash2 =
md5hex(REGISTER:sip:sip.fakeserver.net)
- ▶ Method : REGISTER
- ▶ URI : <sip:sip.fakeserver.net>
- ▶ H2 =
6bd40f0e73fcfbe5b5af7c8397ea12bc

```
[Keiths-MacBook-Air:~ keithcroxford$ md5 -s "REGISTER:sip:sip.fakeserver.net"  
MD5 ("REGISTER:sip:sip.fakeserver.net") = 6bd40f0e73fcfbe5b5af7c8397ea12bc]
```

SIP AUTHENTICATION – WITH QOP

- ▶ The third hash, is a hash of hash1, the nonce, and hash2)
- ▶ H3 =
`md5.hex(b2310cc50edf3e2616d51ba7232db3d7:ea9c8e88df84f1cec4341ae6cbe5a359:6bd40f0e73fcfbe5b5af7c8397ea12bc)`
- ▶ H3 will be sent as the response in the Authentication header.
- ▶ H1 =b2310cc50edf3e2616d51ba7232db3d7
- ▶ Nonce:
ea9c8e88df84f1cec4341ae6cbe5a359
- ▶ H2: 6bd40f0e73fcfbe5b5af7c8397ea12bc
- ▶ Response (h3) :
5b235ab9a09ff2d74da111d00d8b97eb

```
Keiths-MacBook-Air:~ keithcroxford$ md5 -s "b2310cc50edf3e2616d51ba7232db3d7:ea9c8e88df84f1cec4341ae6cbe5a359:6bd40f0e73fcfbe5b5af7c8397ea12bc"
MD5 ("b2310cc50edf3e2616d51ba7232db3d7:ea9c8e88df84f1cec4341ae6cbe5a359:6bd40f0e73fcfbe5b5af7c8397ea12bc") = 5b235ab9a09ff2d74da111d00d8b97eb
```


SIP AUTHENTICATION

REGISTER sip:ss2.wcom.com SIP/2.0

Via: SIP/2.0/UDP there.com:5060

From: LittleGuy <sip:UserB@there.com>

To: LittleGuy <sip:UserB@there.com>

Call-ID: 123456789@there.com

CSeq: 2 REGISTER

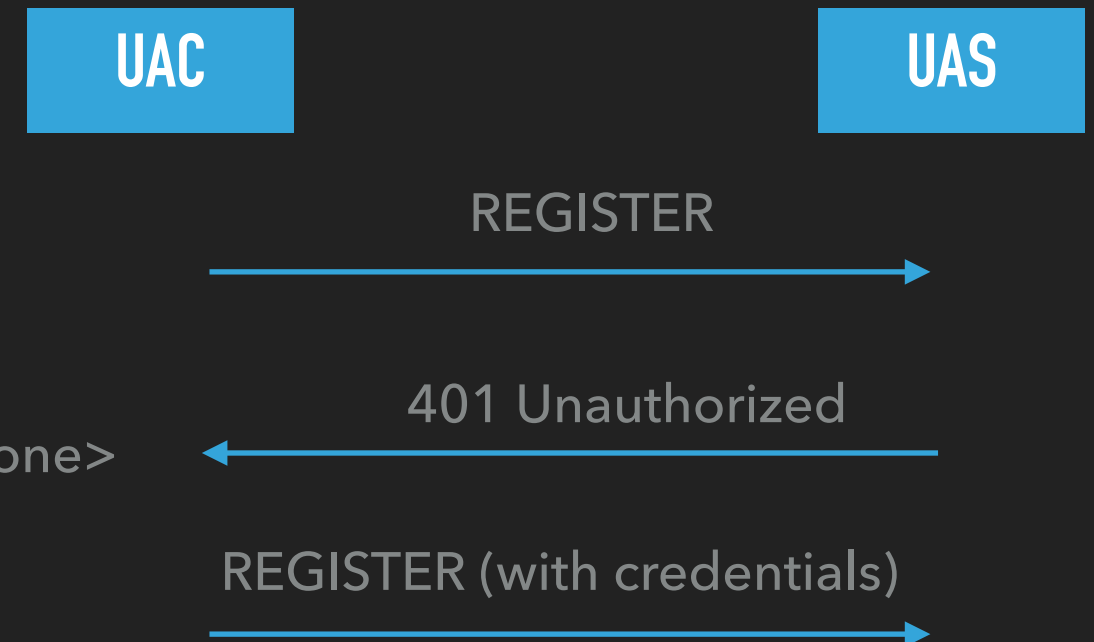
Contact: <sip:UserB@110.111.112.113>

Contact: <sip:+1-972-555-2222@gw1.wcom.com;user=phone>

Contact: tel:+1-972-555-2222

**Authorization: Digest username="UserB",
realm="fakeserver",
nonce="ea9c8e88df84f1cec4341ae6cbe5a359",
uri="sip:sip.fakeserver.net",
response="5b235ab9a09ff2d74da111d00d8b97eb"**

Content-Length: 0



- ▶ Notice the Authorization header, and the response value.

SIP AUTHENTICATION

- ▶ In the event that QOP is required, the third hash (response) is slightly different.
- ▶ **$H3 = \text{md5.hex}(h1:\text{nonce}:nc:cnonce:qop:h2)$**
- ▶ Notice the extra fields added in the middle.

SIP AUTHENTICATION

- ▶ Section Review :
 - ▶ SIP Authentication is based upon HTTP authentication.
 - ▶ When a UAS, registrar or redirect server requires authentication it will send a 401 response.
 - ▶ When a proxy requires authentication, it sends a 407 response.
 - ▶ Basic authentication is not used. Instead SIP uses Digest authentication.

SIP AUTHENTICATION

- ▶ Section Review :
 - ▶ SIP Authentication is based upon HTTP authentication.
 - ▶ When a UAS, registrar or redirect server requires authentication it will send a 401 response.
 - ▶ When a proxy requires authentication, it sends a 407 response.
 - ▶ Basic authentication is not used. Instead SIP uses Digest authentication.

SIP AUTHENTICATION

▶ Section Review :

- ▶ When a 401 or 407 is sent a "WWW-Authenticate" header is sent with the challenge information.
- ▶ The challenged UAC sends a response within the "Authentication header".
- ▶ The response is an MD5 Hash created by two other hashes, a nonce and optionally a few extra parameters.

SIP & NAT

SIP & NAT

- ▶ What is NAT?
- ▶ NAT is Network Address Translation. This is where a device such as a router or firewall translates one IP address to another.



SIP & NAT

- ▶ Along with NAT which is simple translating one IP to another, we also have PAT.
- ▶ PAT is Port Address translation. This is where all traffic from the inside of the network is presented as the same IP in the outside port, but from a different source port.



SIP & NAT

- ▶ NAT and PAT work very well with normal web traffic.
- ▶ SIP... is another story.
 - ▶ Remember SIP is an application layer protocol.
 - ▶ This is because what is listed in the headers and SDP might not match the actual IP packet!



IP PACKET : 192.168.1.20

```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP 192.168.1.20:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@192.168.1.20;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.168.1.20
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

INSIDE OUTSIDE



IP PACKET : 216.1.1.5:24593

```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP 192.168.1.20:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@192.168.1.20;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.168.1.20
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```


SIP & NAT



```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP 192.168.1.20:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@192.168.1.20;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.168.1.20
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

ONE WAY AUDIO!

```
SIP/2.0 200 OK
Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74bf9
;received=72.5.5.5
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>;tag=8321234356
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:bob@client.biloxi.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 174
```

NO ROUTE!

```
v=0
o=bob 2890844527 2890844527 IN IP4 client.biloxi.example.com
s=-
c=IN IP4 72.5.5.5
t=0 0
m=audio 3456 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

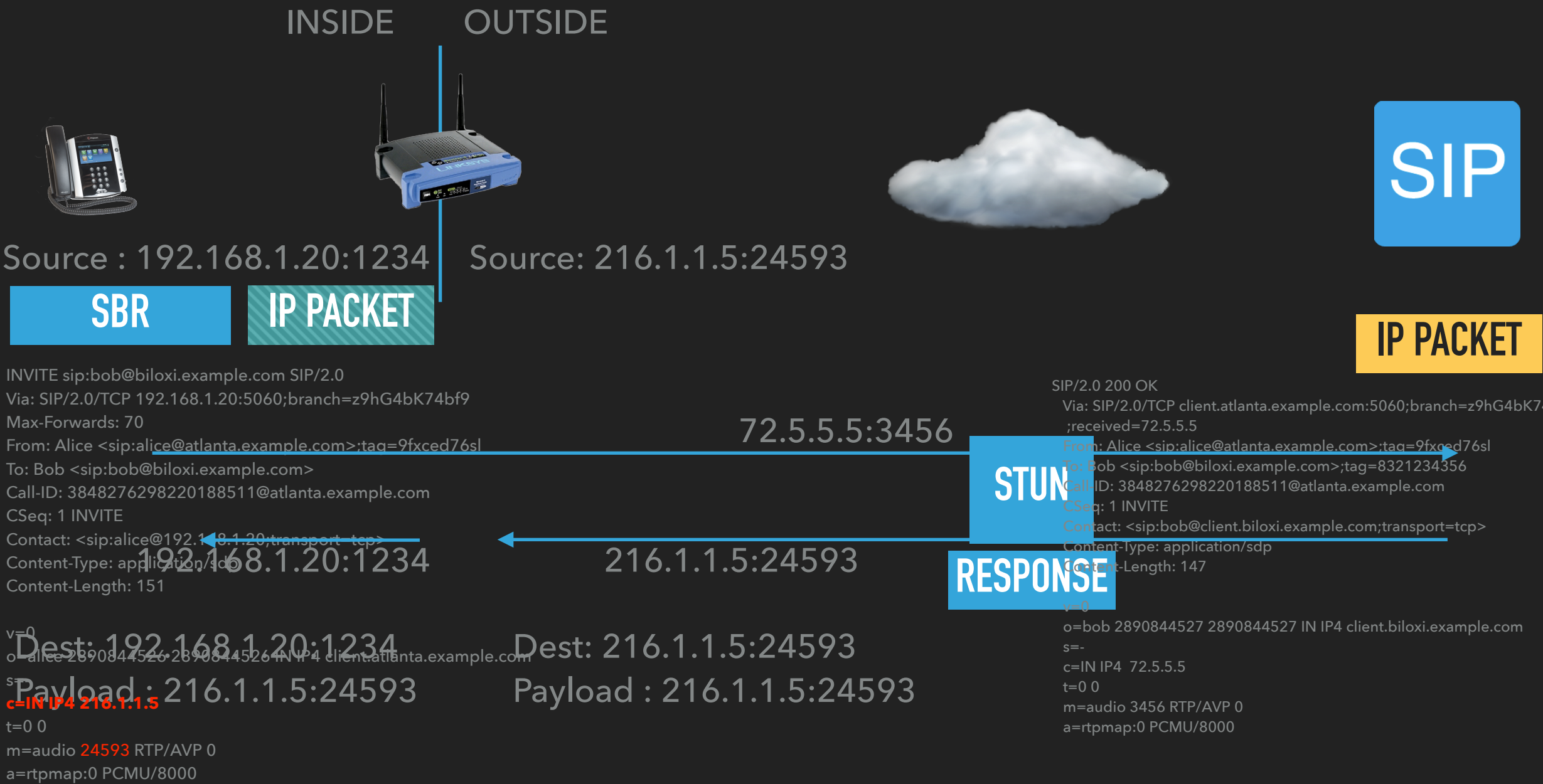
SIP & NAT

- ▶ How do we fix this?
 - ▶ There are a few methods that one can use to resolve this issue. Some are better than others.

SIP & NAT

- ▶ STUN Server -
 - ▶ STUN stands for Simple Traversal Utilities for NAT
 - ▶ Defined in RFC 5389 (replaces RFC 3489)
 - ▶ STUN is a simple protocol for discovering the public IP addresses

STUN



SIP & NAT

▶ SIP ALG

- ▶ An ALG is an Application Layer Gateway
- ▶ Most modern consumer and “prosumer” routers have this feature enabled.
- ▶ The ALG modifies the SIP headers & SDP before the packet is forward to the upstream device.

SIP ALG



```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP 192.168.1.20:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@192.168.1.20;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 192.168.1.20
t=0 0
m=audio 49172 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

```
INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP 216.1.1.1:8855;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@216.1.1.1;transport=tcp>
Content-Type: application/sdp
Content-Length: 151
```

```
v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 216.1.1.5
t=0 0
m=audio 24593 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

SIP & NAT

- ▶ The problem with SIP ALG
 - ▶ Most firewall vendors... have an AWFUL implementation of it.
 - ▶ Often it fails to forward the SIP message to the NAT'd endpoint and /or fails to forward the RTP packets properly.
 - ▶ This often needs to be disabled.
 - ▶ Note - A GOOD ALG can behave badly if your service provider has an SBC capable of handling NAT.

SIP & NAT

- ▶ Who has an GOOD ALG implementation? (From my experience)



SIP & NAT

- ▶ Another solution is TURN
 - ▶ TURN stands for Traversal Using Relays around NAT.
 - ▶ TURN is built off of STUN and has similar behavior.
 - ▶ Except return RTP (media) flows through the TURN server, rather than direct

TURN

INSIDE OUTSIDE



Source : 192.168.1.20:1234

Source: 216.1.1.5:24593

SBR

IP PACKET

IP PACKET

INVITE sip:bob@biloxi.example.com SIP/2.0
Via: SIP/2.0/TCP 192.168.1.20:5060;branch=z9hG4bK74bf9
Max-Forwards: 70
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:alice@192.168.1.20;transport=tcp>
Content-Type: application/sdp
Content-Length: 151

v=0
o=alice 2890844526 2890844526 IN IP4 client.atlanta.example.com
s=-
c=IN IP4 216.1.1.5
t=0 0
m=audio 24593 RTP/AVP 0
a=rtpmap:0 PCMU/8000

Dest: 192.168.1.20:1234

Payload : 216.1.1.5:24593

Dest: 216.1.1.5:24593

Payload : 216.1.1.5:24593

SIP/2.0 200 OK
Via: SIP/2.0/TCP client.atlanta.example.com:5060;branch=z9hG4bK74bf9
;received=72.5.5.5
From: Alice <sip:alice@atlanta.example.com>;tag=9fxced76sl
To: Bob <sip:bob@biloxi.example.com>;tag=8321234356
Call-ID: 3848276298220188511@atlanta.example.com
CSeq: 1 INVITE
Contact: <sip:bob@client.biloxi.example.com;transport=tcp>
Content-Type: application/sdp
Content-Length: 147
v=0
o=bob 2890844527 2890844527 IN IP4 client.biloxi.example.com
s=-
c=IN IP4 72.5.5.5
t=0 0
m=audio 3456 RTP/AVP 0
a=rtpmap:0 PCMU/8000

TURN
RESPONSE

SIP & NAT

- ▶ Turn guarantees communication in all NAT scenarios unless a firewall policy is blocking communication.
- ▶ It's superior to STUN in many ways, yet has disadvantages
 - ▶ It's part of the forwarding path
 - ▶ Requires a lot of bandwidth
 - ▶ The TURN server must remain available through the entire session

SIP & NAT

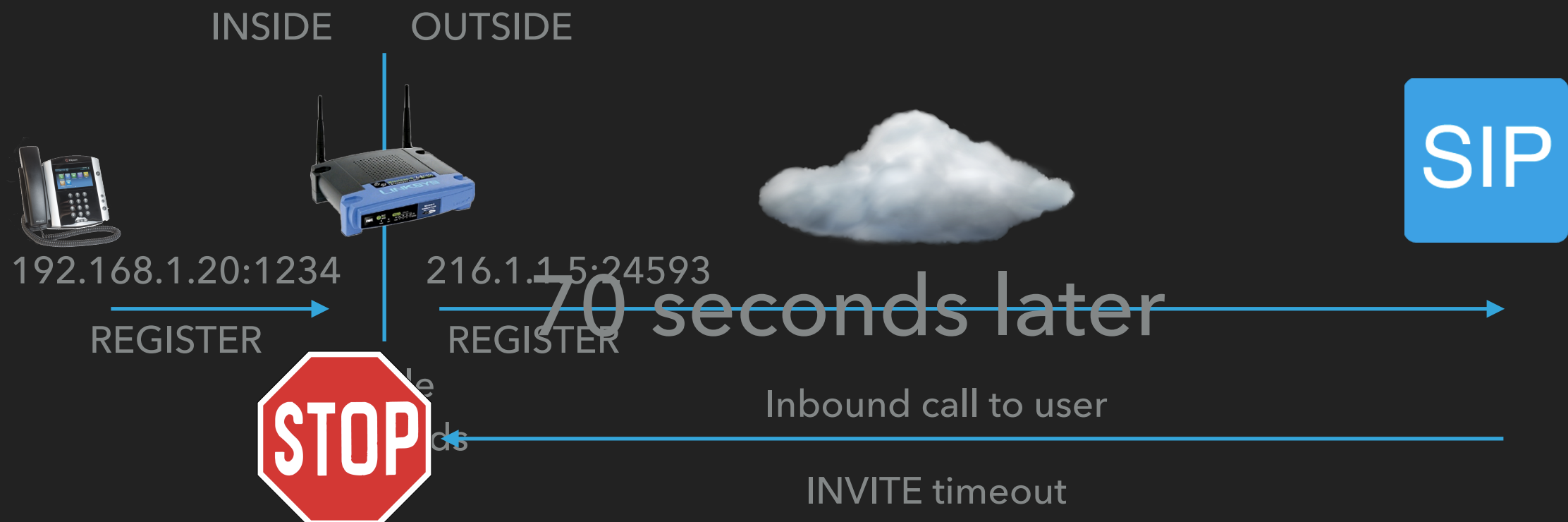
- ▶ My favorite solutions for NAT issues with SIP is...
 - ▶ Having an intelligent SBC that handles it, and it just works!
 - ▶ Most of the time.
 - ▶ A high end SBC like Oracle (Acme packet) uses a feature called "HNT" or Header NAT Traversal to accomplish NAT detection and mitigation
 - ▶ In Oracle/Acme terms, the SBC looks at the IP datagram source and determines if the actual IP matches with the VIA, Contact or SDP C= line.
 - ▶ If not - The SBC manipulates headers and routes traffic to where it needs to go.

SIP & NAT

- ▶ The biggest downfall of this : High end SBCs are expensive.
 - ▶ Usually only large enterprises and carriers can afford these devices.
 - ▶ I'm not kidding. Two SBCs in a High Availability setup with a few thousand concurrent session licenses can easily run \$250,000.
 - ▶ As mentioned earlier - These SBCs can still experience problems if a SIP ALG is enabled on the customer side.

SIP & NAT

- ▶ Aside from audio issues, NAT is prone to causing call control (SIP signaling) issues as well.
- ▶ This is often related to the amount of time that a firewall keeps a NAT pinhole open.



SIP & NAT

- ▶ How do we fix this?
 - ▶ Method 1 - Often an ISP or someone that manages your proxy / B2BUA will adjust the timer on the registration interval.
 - ▶ This will be set to a low number to force REGISTER messages to constantly be sent, thus keeping the pinhole open.
 - ▶ Method 2 - Port forwarding is configured on the firewall / router.
 - ▶ The downfall to this method is ports for RTP are typically dynamic.
 - ▶ Method 3 - You may have lucked out and have a GOOD ALG running on your firewall. It will handle these requests properly.

SUMMARY

- ▶ NAT can cause RTP and signaling issues
- ▶ STUN and TURN servers are often used to work around audio issues.
- ▶ SIP ALGs are often activated by default on consumer firewalls. Sometimes these work, but from my experience they must usually be disabled.
- ▶ High end Session Border Controllers work well to resolve NAT issues.
 - ▶ These can even adjust the REGISTRATION interval to make sure NAT pinholes are open for signaling traffic.

RTP OVERVIEW

RTP OVERVIEW

- ▶ In our SDP module we briefly mentioned the Real Time Protocol (RTP).
- ▶ RTP is not an element of SIP, but a protocol that SIP uses to transport data with real time characteristics.



RTP OVERVIEW

- ▶ RTP is defined in RCF # 3550.
- ▶ When we talk about RTP there are typically two aspects.
 - ▶ The Real Time Protocol (RTP) - Which is used to carry data with real time properties.
 - ▶ The Real Time Control Protocol (RTCP) - This is used to monitor the quality of the session and convey information.

RTP OVERVIEW

- ▶ As we mentioned in the SDP video when endpoints establish a session the SDP will list elements of where RTP must be received.

INVITE SDP

```
v=0
o=root 42852867 42852867 IN IP4 10.130.130.114
s=call
c=IN IP4 10.130.130.114
t=0 0
m=audio 61896 RTP 0 8 3 101
a=rtpmap:0 pcmu/8000
a=rtpmap:8 pcma/8000
a=rtpmap:3 gsm/8000
a=rtpmap:101 telephone-event/8000
a=fmtp:101 0-16
a=ptime:20
a=sendrecv
```

200 OK SDP

```
v=0
o=root 12345 54321 IN IP4 10.130.130.113
s=call
c=IN IP4 10.130.130.113
t=0 0
m=audio 44444 RTP 0 8 3 101
a=rtpmap:0 pcmu/8000
a=rtpmap:8 pcma/8000
a=rtpmap:3 gsm/8000
a=rtpmap:101 telephone-event/8000
a=fmtp:101 0-16
a=ptime:20
a=sendrecv
```

RTP OVERVIEW

- ▶ We need to begin with a few definitions and terms that you will hear when dealing with RTP.
 - ▶ RTP Payload : This is the data transported within an RTP packet. This could be audio samples or compressed video data.
 - ▶ RTP Packet : This is a data packet consisting of a fixed RTP header, a list of contributing sources (may be empty) and the RTP Payload data.
 - ▶ RTCP packet : A data packet very similar to the RTP packet, but used to send session control information

RTP OVERVIEW

- ▶ More Definitions:

- ▶ Transport Address: This is a combination of the network address and port that creates a transport level endpoint. Packets are sent from a "Source Transport Address" to a "Destination Transport Address"
- ▶ RTP Media Types : This is a collection of payload types that can be carried within a single RTP session.
- ▶ Multimedia Session : This is a set of concurrent RTP sessions that contain mixed forms of media. For example, two users might be on a video call, while a third joins in from a non video device.

RTP OVERVIEW

- ▶ More Definitions:
 - ▶ RTP Session : Simply an association of multiple endpoints communicating with RTP.
 - ▶ **Synchronization Source (SSRC):** This is the source of a stream of RTP packets. It is identified by a 32 bit numeric SSRC identifier carried in the RTP header. Now this is not dependent on the network address.
 - ▶ Contributing Source Identifier (CSRC) : This is the source of an RTP stream that has contributed to the combined stream produced by an RTP mixer.

RTP OVERVIEW

- ▶ I highlighted this in red for a reason. The SSRC has help me to resolve various issues in the past.
- ▶ For example: You have a client that states they occasionally do not head any audio. You perform a capture and to the naked eye, you see that RTP is flowing properly.

No.	Time	Source	Destination	Protocol	Length	Info
60	28.448409	188.40.69.75	92.127.221.146	RTP	74	PT=DynamicRTP-Type-111, SSRC=0x3B359CA0, Seq=5, Time=72
61	28.456185	92.127.221.146	188.40.69.75	RTP	144	PT=DynamicRTP-Type-111, SSRC=0x1EC9BA4B, Seq=2224, Time=73

+	Frame 60: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
+	Ethernet II, Src: 00:24:21:9c:37:ed (00:24:21:9c:37:ed), Dst: 00:21:59:c5:74:e0 (00:21:59:c5:74:e0)
+	Internet Protocol Version 4, Src: 188.40.69.75 (188.40.69.75), Dst: 92.127.221.146 (92.127.221.146)
+	User Datagram Protocol, Src Port: 31030 (31030), Dst Port: 58732 (58732)
-	Real-Time Transport Protocol
10..	 = Version: RFC 1889 Version (2)
..0.	 = Padding: False
...1	 = Extension: True
....	0000 = Contributing source identifiers count: 0
0...	 = Marker: False
Payload type: DynamicRTP-Type-111 (111)	
Sequence number: 5	
Synchronization source identifier: 0x3b359ca0 (993369248)	
Extension length: 0	
Payload: 415f46b58f40d12a1631ecd37d41c629	

0000	00 21 59 c5 74 e0 00 24	21 9c 37 ed 08 00 45 00	..Y.t..\$!.7...E.
0010	00 3c 00 00 40 00 40 11	ff 2b bc 28 45 4b 5c 7f	.<..@.@. .+. (EK\.
0020	dd 92 79 36 e5 6c 00 28	ad 52 90 6f 00 05 00 01	..y6.l.(.R.o....
0030	1d 00 3b 35 9c a0 03 e7	00 00 41 5f 46 b5 8f 40	..;5.... ..A_F..@
0040	d1 2a 16 31 ec d3 7d 41	c6 29	..%.1..}A .)

Frame (frame), 74 bytes Packets: 1057 · Displayed: 1057 (100,0%) · Load time: 0:00.013 Profile: Default

RTP OVERVIEW

- ▶ However you notice that the SSRC changes to another value a second after RTP begins.
- ▶ What happened? Well there may have been a call control change that occurred upstream from your network. Traffic may have shifted to a different media gateway, but you don't see the RE-INVITE.

No.	Time	Source	Destination	Protocol	Length	Info
60	28.448409	188.40.69.75	92.127.221.146	RTP	74	PT=DynamicRTP-Type-111, SSRC=0x3B359CA0, Seq=5, Time=72
61	28.456185	92.127.221.146	188.40.69.75	RTP	144	PT=DynamicRTP-Type-111, SSRC=0x1EC9BA4B, Seq=2224, Time=73

+	Frame 60: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
+	Ethernet II, Src: 00:24:21:9c:37:ed (00:24:21:9c:37:ed), Dst: 00:21:59:c5:74:e0 (00:21:59:c5:74:e0)
+	Internet Protocol Version 4, Src: 188.40.69.75 (188.40.69.75), Dst: 92.127.221.146 (92.127.221.146)
+	User Datagram Protocol, Src Port: 31030 (31030), Dst Port: 58732 (58732)
-	Real-Time Transport Protocol
10..	 = Version: RFC 1889 Version (2)
..0.	 = Padding: False
...1	 = Extension: True
....	0000 = Contributing source identifiers count: 0
0...	 = Marker: False
Payload type: DynamicRTP-Type-111 (111)	
Sequence number: 5	
Synchronization source identifier: 0x3b359ca0 (993369248)	
Extension length: 0	
Payload: 415f46b58f40d12a1631ecd37d41c629	

0000	00 21 59 c5 74 e0 00 24	21 9c 37 ed 08 00 45 00	..Y.t..\$!.7...E.
0010	00 3c 00 00 40 00 40 11	ff 2b bc 28 45 4b 5c 7f	..<..@.@. .+. (EK\.
0020	dd 92 79 36 e5 6c 00 28	ad 52 90 6f 00 05 00 01	..y6.l.(.R.o....
0030	1d 00 3b 35 9c a0 03 e7	00 00 41 5f 46 b5 8f 40	...;5.... ..A_F..@
0040	d1 2a 16 31 ec d3 7d 41	c6 29	..%.1..}A .)

Frame (frame), 74 bytes Packets: 1057 · Displayed: 1057 (100,0%) · Load time: 0:00.013 Profile: Default

RTP OVERVIEW

- ▶ After you speak with the upstream partner or carrier and provide them this information they realize that they have a device that is experiencing intermittent issues.
- ▶ You just resolve the issue for your customer AND potentially thousands of others.

No.	Time	Source	Destination	Protocol	Length	Info
60	28.448409	188.40.69.75	92.127.221.146	RTP	74	PT=DynamicRTP-Type-111, SSRC=0x3B359CA0, Seq=5, Time=72
61	28.456185	92.127.221.146	188.40.69.75	RTP	144	PT=DynamicRTP-Type-111, SSRC=0x1EC9BA4B, Seq=2224, Time=72

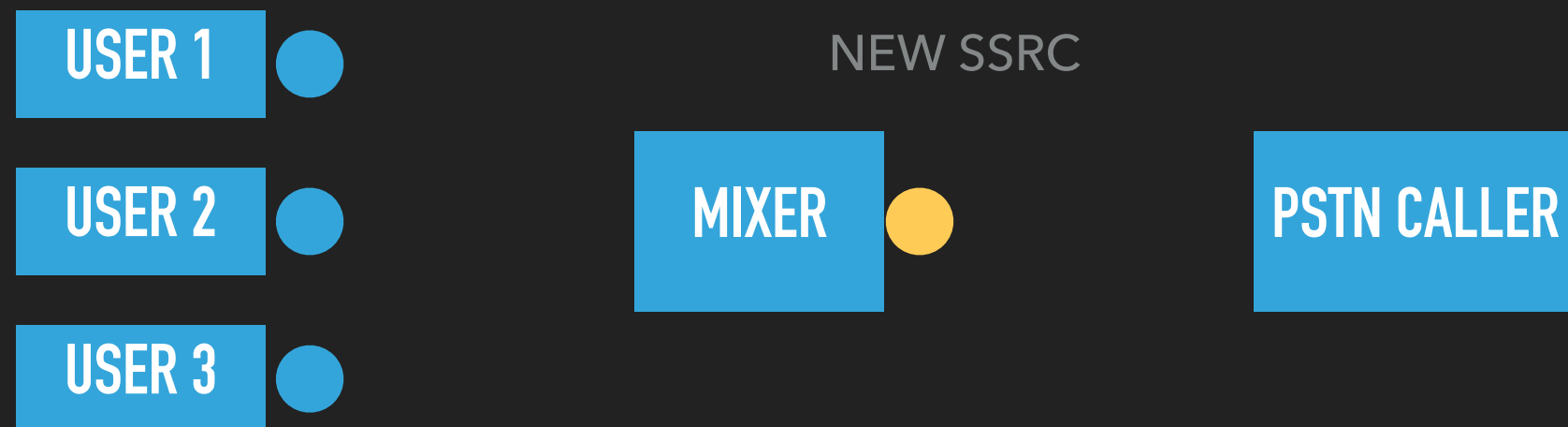
+	Frame 60: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
+	Ethernet II, Src: 00:24:21:9c:37:ed (00:24:21:9c:37:ed), Dst: 00:21:59:c5:74:e0 (00:21:59:c5:74:e0)
+	Internet Protocol Version 4, Src: 188.40.69.75 (188.40.69.75), Dst: 92.127.221.146 (92.127.221.146)
+	User Datagram Protocol, Src Port: 31030 (31030), Dst Port: 58732 (58732)
-	Real-Time Transport Protocol
10..	 = Version: RFC 1889 Version (2)
..0.	 = Padding: False
...1	 = Extension: True
....	0000 = Contributing source identifiers count: 0
0...	 = Marker: False
Payload type: DynamicRTP-Type-111 (111)	
Sequence number: 5	
Synchronization source identifier: 0x3b359ca0 (993369248)	
Extension length: 0	
Payload: 415f46b58f40d12a1631ecd37d41c629	

0000	00 21 59 c5 74 e0 00 24	21 9c 37 ed 08 00 45 00	..Y.t..\$!.7...E.
0010	00 3c 00 00 40 00 40 11	ff 2b bc 28 45 4b 5c 7f	..<..@.@. .+. (EK\.
0020	dd 92 79 36 e5 6c 00 28	ad 52 90 6f 00 05 00 01	..y6.l.(.R.o....
0030	1d 00 3b 35 9c a0 03 e7	00 00 41 5f 46 b5 8f 40	...;5.... ..A_F..@
0040	d1 2a 16 31 ec d3 7d 41	c6 29	..%.1..}A .)

Frame (frame), 74 bytes Packets: 1057 · Displayed: 1057 (100,0%) · Load time: 0:00.013 Profile: Default

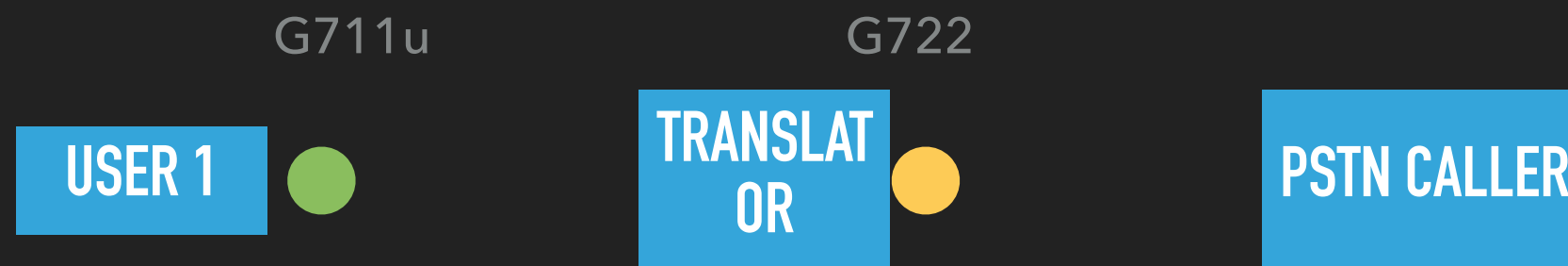
RTP OVERVIEW

- ▶ Back to definitions :
 - ▶ Mixer : This is an intermediate system that receives RTP from multiple sources. It might change the data format, combine the streams then forward the combined RTP as a single new packet.



RTP OVERVIEW

- ▶ Back to definitions :
 - ▶ Translator - This is an intermediate system that forwards RTP packets with the original SSRC intact. This is typically used to change the RTP encoding type.
 - ▶ In the event that media is changed from one codec to another, we call this transcoding.



RTP OVERVIEW

V	P	X	CC	M	PT	SEQUENCE #
TIME STAMP						
SSRC						
CSRC						

RTP OVERVIEW

▶ RTP Payload Types

- ▶ Recall the M line within an SDP. You will see series of numbers after where it states "RTP". These are the payload types.

```
v=0
o=root 42852867 42852867 IN IP4 10.130.130.114
s=call
c=IN IP4 10.130.130.114
t=0 0
m=audio 61896 RTP 0 8 3 101
a=ptime:20
a=sendrecv
```

RTP OVERVIEW

PT	Encoding	A/V	Clock Rate	Channel
0	PCMU	A	8000	1
1	Reserved			
2	Reserved			
3	GSM	A	8000	1
4	G723	A	8000	1
5	DVI4	A	8000	1
6	DVI4	A	16000	1
7	LPC	A	8000	1
8	PCMA	A	8000	1
9	G722	A	8000	1
10	L16	A	44100	2
11	L16	A	44100	1
12	QCELP	A	8000	1
13	CN	A	8000	1
14	MPA	A	90000	
15	G728	A	8000	1
16	DVI4	A	11025	1
17	DVI4	A	22050	1
18	G729	A	8000	1
19	Reserved	A		
20	Unassigned		A	
21	Unassigned		A	
22	Unassigned		A	
23	Unassigned		A	
24	Unassigned		V	
25	CelB	V	90000	
26	JPEG	V	90000	
27	Unassigned		V	
28	nv	V	90000	
29	Unassigned		V	
30	Unassigned		V	
31	H261	V	90000	
32	MPV	V	90000	
33	MP2T	AV	90000	
34	H263	V	90000	
35-71	Unassigned		?	
72-76	Reserved for RTCP conflict avoidance			
77-95	Unassigned		?	
96-127	dynamic		?	

```
v=0
o=root 42852867 42852867 IN IP4 10.130.130.114
s=call
c=IN IP4 10.130.130.114
t=0 0
m=audio 61896 RTP 0 8 3 101
a=ptime:20
a=sendrecv
a=rtpmap:101 telephone-event/8000
```

0 = G711 (PCMU)

8 = G711A (PCMA)

3 = GSM

101 = RTP events

RTP OVERVIEW

- ▶ Telephone events
 - ▶ Telephone events, or RFC 2833 is a method for transporting Dual Tone Multiple Frequency (DTMF) packets.
 - ▶ Why do these exist?
 - ▶ Some lower quality codecs cannot properly generate DTMF tones. They need to use another mechanism with higher quality to get this data to the other side.

RTP OVERVIEW

- ▶ Other DTMF transport methods :
 - ▶ In the audio stream
 - ▶ SIP INFO messages
- ▶ Note - When DTMF is sent within the audio stream it is known as “in band”. When it is sent through another method such as telephone events or SIP INFO, it is called “out of band”

RTP OVERVIEW

▶ Section Review

- ▶ Real Time Protocol is used for sending data (audio or video) between devices.
- ▶ Real Time Control Protocol is used along with RTP to convey information about the session.
- ▶ RTP payload types used in a call can be found in the SDP.
- ▶ Mixers combine media from multiple sources and place them in a new RTP packet.
- ▶ Translators are able to modify the encoding, yet keep the SSRC the same

BONUS!
CONVERTING VOICE INTO
PACKETS

BONUS

- ▶ In the early 1900s a man named Dr. Harry Nyquist was employed by Bell Labs.
- ▶ Dr. Nyquist along with Claude Shannon discovered a process to allow analog signals to convert into a digital format.
- ▶ This process was discovered while trying to determine how to deploy more voice circuits across with less wire.



BONUS

- ▶ Back in the early 1900s if an organization needed 500 phone “lines” into their building, the telephone company would need to deliver 500 physical cables from the Telco Central office.
- ▶ Obviously, this was not a scalable model.



Photo - Cussett & Co., Um

BONUS

- ▶ Dr. Nyquist and Claude Shannon found that they could reconstruct audio stream by taking samples at twice the highest frequency used in the audio stream.
- ▶ What's a sample? A sample in the voice world is simply a numeric value that consumes a single byte of information.

BONUS

- ▶ These samples vary based upon the volume and pitch that comprise the sound.
- ▶ The human ear is able to hear sounds from the 20-20,000Hz range
- ▶ Human speech is around 200 - 9,000Hz
- ▶ Telephones typically transmit between 300-3400Hz

BONUS

- ▶ It was found that the lower range used by telephones (300-3400Hz) gives enough sound quality to identify the person on the other side AND sense their mood.
- ▶ Telephones do not send the full spectrum of human voice inflection.



BONUS

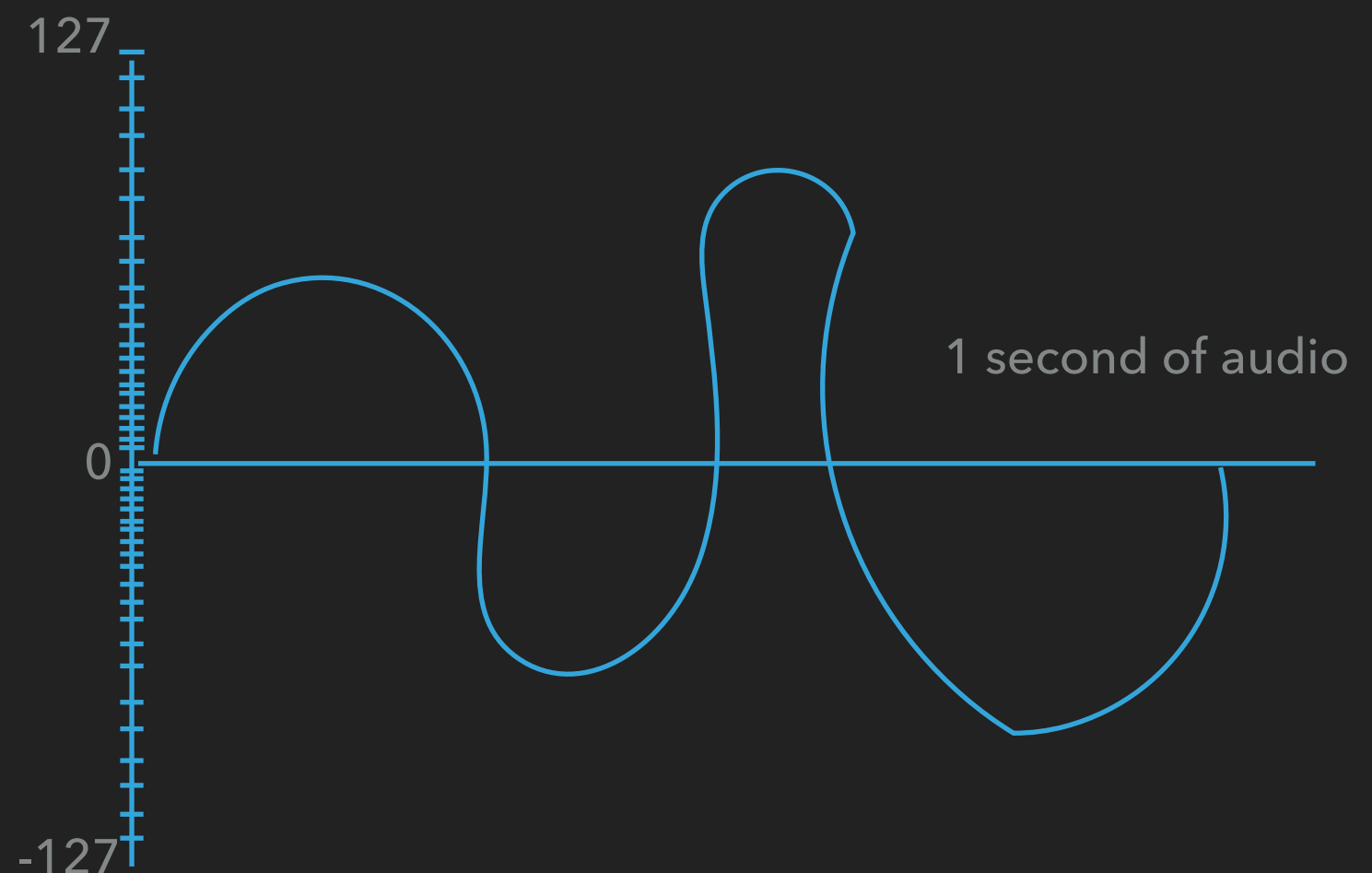
- ▶ So by this logic Dr. Nyquist thought that taking the range of 300-4000Hz would suffice for testing.
- ▶ To sample this at 2x the highest frequency, this means we have 8,000 ($2 * 4000$) samples every second.

BONUS

- ▶ When samples are taken, the audio wave form is converted into digital, numeric values.
- ▶ This process is called quantization.

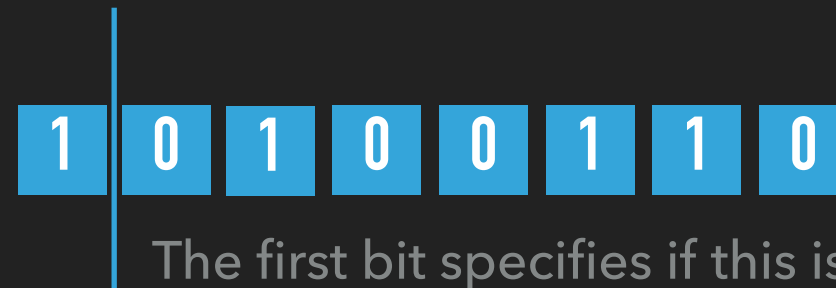
Note - Common frequencies within a conversation are packed closer together on the scale.

Frequencies that are less common are known as fridge frequencies and are spaced further apart.



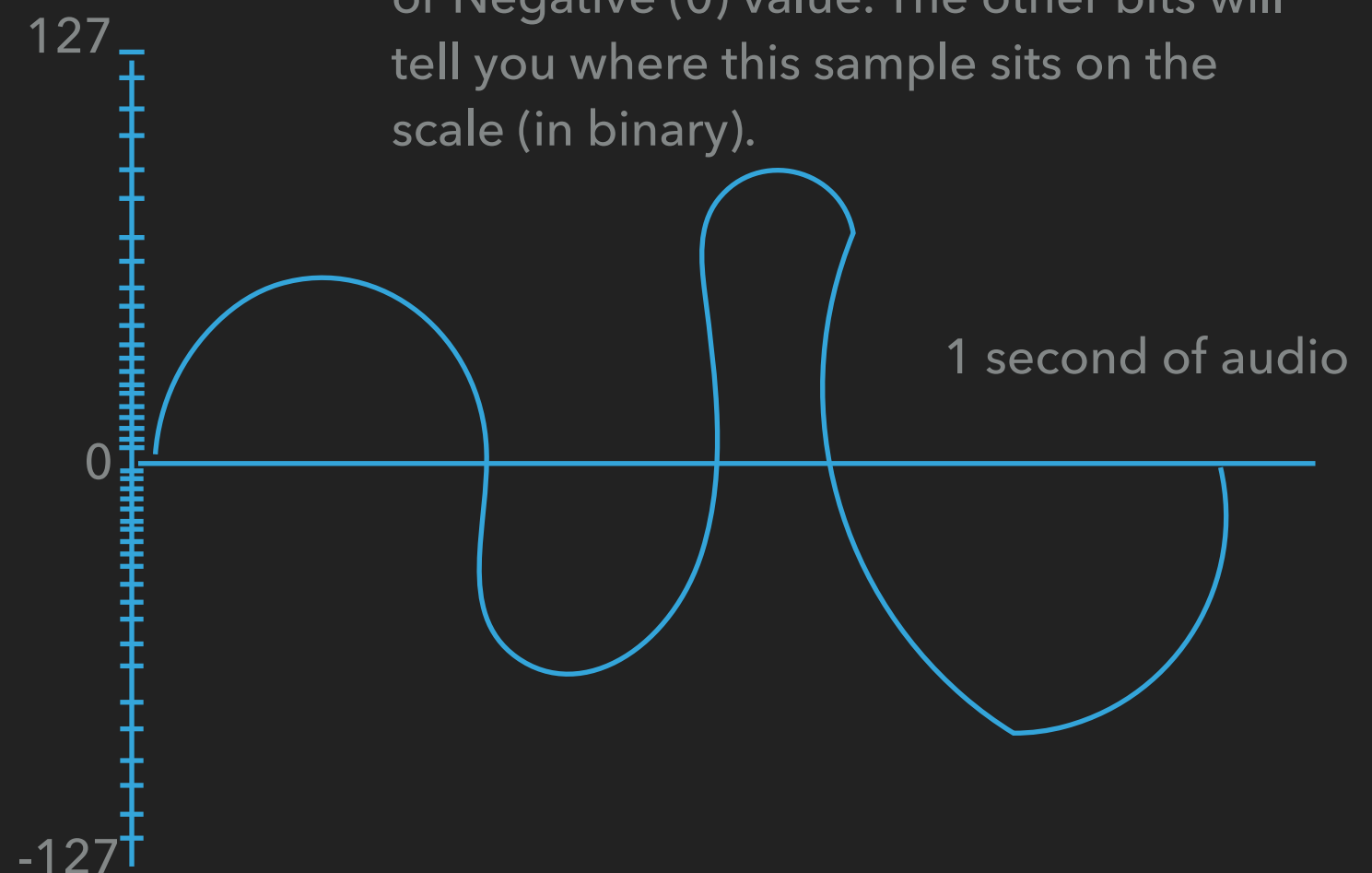
BONUS

- ▶ Now that we have the samples turned into a number, it must be encoded in binary format.



The first bit specifies if this is a positive (1)

or Negative (0) value. The other bits will tell you where this sample sits on the scale (in binary).



BONUS

- ▶ We now have 1 second of audio quantized.
- ▶ Again.. we have 8,000 samples in this one second.
- ▶ Each sample has 8 bits.
- ▶ If we multiple $8 * 8,000$ we have 64,000 bits per second.
- ▶ **64,000 or 64kbps is the amount of bandwidth consumed by an uncompressed codec in a telephone conversation.**

BONUS

- ▶ All of these conversions take place through what is known as a Digital Signal Processor.
- ▶ In IP phones, DSPs are built into the phone hardware. In many routers, you will need to purchase an add-on module before they can be voice capable.
- ▶ When using a softphone, your computer hardware performs the sampling, quantization, and encoding.



BONUS

- ▶ No before the encoded data is turned into a packet there is one last option step.
- ▶ Compressing the number of samples sent per second.
- ▶ Some voice codecs do this to minimize the amount of bandwidth consumed.
- ▶ Sadly compression is used, the audio quality is degraded.
- ▶ Note - cellular networks have historically compressed the audio data. In modern 4G / LTE networks compression is no longer needed and many providers now utilize high definition voice codecs!

BONUS

- ▶ There are a number of voice codecs used in modern networks. The most common ones are PCMU (G.711u) PCMA (G.711a) and G.729.
- ▶ PCMU and PCMA are uncompressed.
- ▶ G.729 is highly compressed!

BONUS

Codec	BW	
G.711	64,000	
ILBC	15,200	
G.729	8,000	
G.726	32,000	
G.729a	8,000	
G.722	64,000	Also available in 48,000 and 56,000 versions

BONUS

- ▶ Recap -
- ▶ The four steps for converting analog signals into digital signals :
 - ▶ 1. Sampling
 - ▶ 2. Quantization
 - ▶ 3. Encoding
 - ▶ 4 Compressing (optional)

BONUS

- ▶ Ok, so G.711 used 64kbps and G.729a uses 8kbps right? I can calculate bandwidth with that?

WRONG!

- ▶ Those values are just the payload size!

BONUS

- ▶ There is another process to calculate the true bandwidth requirements!
- ▶ Step 1 : Determine bandwidth of the codec used.
- ▶ Step 2: Determine the data link (layer 2), Network and transport layer overhead
- ▶ Step3 : Add additional overhead amounts (MPLS, IPSEC etc)
- ▶ Step 4 : Do the math

BONUS

- ▶ Let's use G.711u as an example. This is going across an ethernet circuit, with no optional overhead amounts.
- ▶ We know this consumes 64kbps, however we are not going to send 1 giant packet per second!
- ▶ The actual audio samples typically contain 20ms of audio. Other values can be used, however 20ms is common. This is often called the "packetization rate"

BONUS

- ▶ First we need to figure out how many Bytes are going to be sent per packet.
- ▶ The equation for this is $\text{Bytes_per_Packet} = (\text{Sample_size} * \text{Codec Bandwidth}) / 8$.
- ▶ PEMDAS!
- ▶ So our 20ms sample size * 64,000 = 1280
- ▶ Divide that by 8 and we have 160
- ▶ **160** = $(20 * 64,000) / 8$
- ▶

BONUS

- ▶ Now that we have the amount of voice contained in each packet, we need to get the overhead values added in.
- ▶ Layer 2 overhead:
 - ▶ Ethernet = 20 bytes
 - ▶ Frame Relay = 4-6 Bytes
 - ▶ PPP = 6 Bytes
- ▶ Network and Transport Overhead:
 - ▶ IP = 20 bytes
 - ▶ UDP = 8 bytes
 - ▶ RTP = 12 bytes

Note - All packets will use RTP, UDP and IP - just sum this up as 40 bytes

BONUS

- ▶ Optional Overhead Values :
 - ▶ GRE = 24 bytes
 - ▶ MPLS = 4 bytes
 - ▶ IPSEC = 50-57 bytes

BONUS

- ▶ Add it all up
- ▶ $160 + 20 \text{ (ethernet)} + 40 \text{ (IP,UDP,RTP)} = 220\text{bytes}$
- ▶ Now the packet per second. If we are using a 20ms sample rate, what does that translate to as a PPS?
 - ▶ 1 second = 1000 ms
 - ▶ $1000 / 20 = 50.$
 - ▶ 50 packets per second

BONUS

- ▶ We now take the Packet size (220) and multiple it by the PPS (50)
- ▶ This ends up as:

$$220 * 50 = 110,000$$

110kbps

BONUS

- ▶ You try.
- ▶ Scenario :
 - ▶ We have a voice call using G.729a (8kbps)
 - ▶ Samples are 20ms each.
 - ▶ Layer 2 = Frame Relay (use 4 bytes)
 - ▶ IPSec is in use across the link as well (57 bytes)

BONUS

- ▶ Did you come up with this?
 - ▶ Bytes per packet = $(.02 * 8,000) / 8$
 - ▶ Bytes per packet = 20
 - ▶ Add the layer 2 over header $20 + 4 = 24$
 - ▶ Add the IP, UDP, and RTP overhead : $24 + 40 = 64$
 - ▶ Add the IPSEC overhead : $64 + 57 = 121$
 - ▶ Total packet size = 121 bytes
 - ▶ $121 * \text{the packets per second (50)} = 60,050$ or 60kbps.

BONUS

- ▶ Summary :
 - ▶ To turn analog voice into digital voice it goes through the process of :
 - ▶ Sampling
 - ▶ Quantization
 - ▶ Encoding
 - ▶ Compression (Optional)

BONUS

- ▶ Summary :
 - ▶ Each codec consumes a different amount of bandwidth.
 - ▶ Adding additional overhead can greatly change the amount of bandwidth consumed per second.

SECURING SIP

GOAL OF THIS MODULE

- ▶ After completing this module you will understand
 - ▶ Common security issues with SIP
 - ▶ How TLS and SRTP Function
 - ▶ Tools and methods used to attack SIP networks
 - ▶ Best practices for a SIP deployment

COMMON SIP SECURITY ISSUES

SIP SECURITY ISSUES

- ▶ Eavesdropping - Listening into another party's conversation.



SIP SECURITY ISSUES

- ▶ Caller ID Spoofing

SIP SECURITY ISSUES

- ▶ Registration Hijacking

SIP SECURITY ISSUES

- ▶ Impersonating of a valid proxy

SIP SECURITY ISSUES

- ▶ Call Redirection

SIP SECURITY ISSUES

- ▶ Unauthenticated Calling

SIP SECURITY ISSUES

- ▶ Signaling Denial of Service

SIP SECURITY ISSUES

- ▶ Media Denial of Service

SIP SECURITY ISSUES

- ▶ **API attacks**

- ▶ Not SIP specific - but relevant to modern VOIP security.

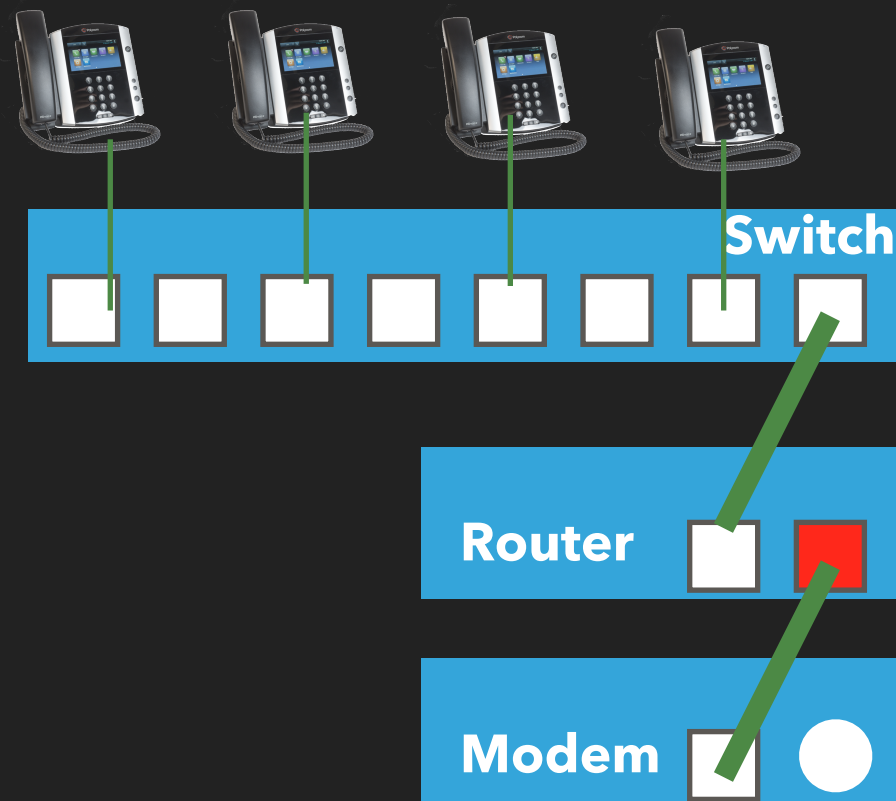
CAPTURING SIP TRAFFIC

CAPTURING SIP TRAFFIC

- ▶ Troubleshooting SIP / VOIP requires you have a special environment to capture traffic.
- ▶ Let's go over a few options

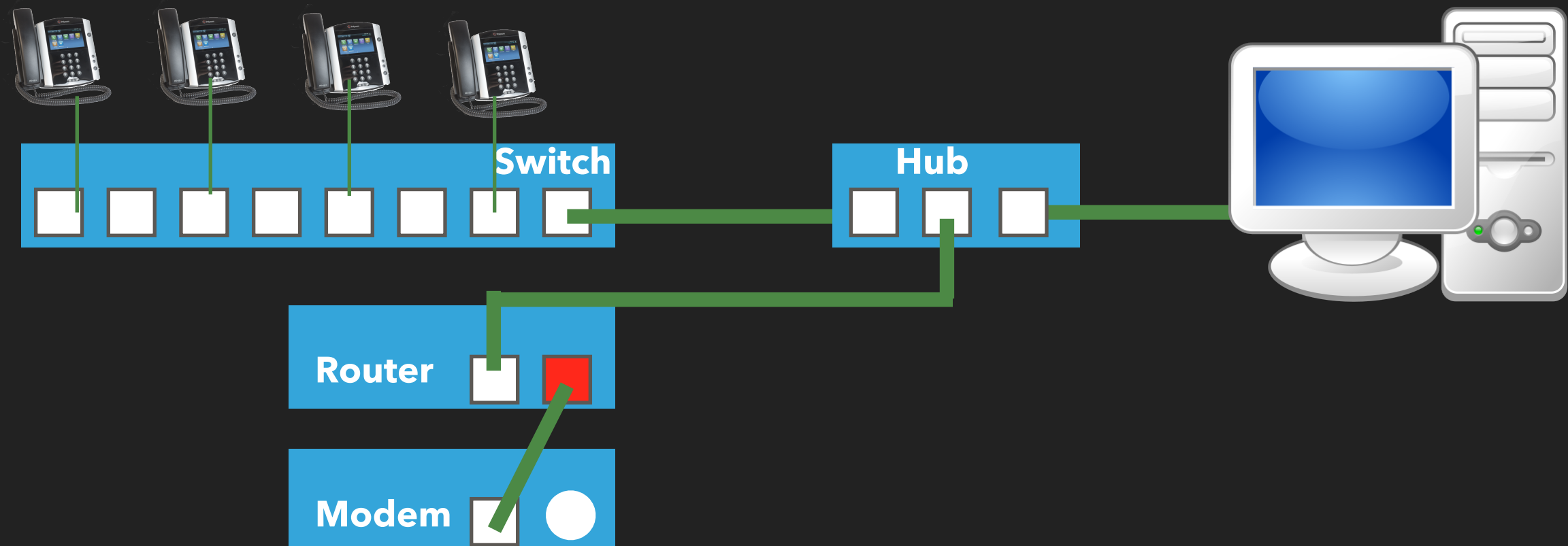
CAPTURING SIP TRAFFIC

- ▶ Generic Home / Small office network



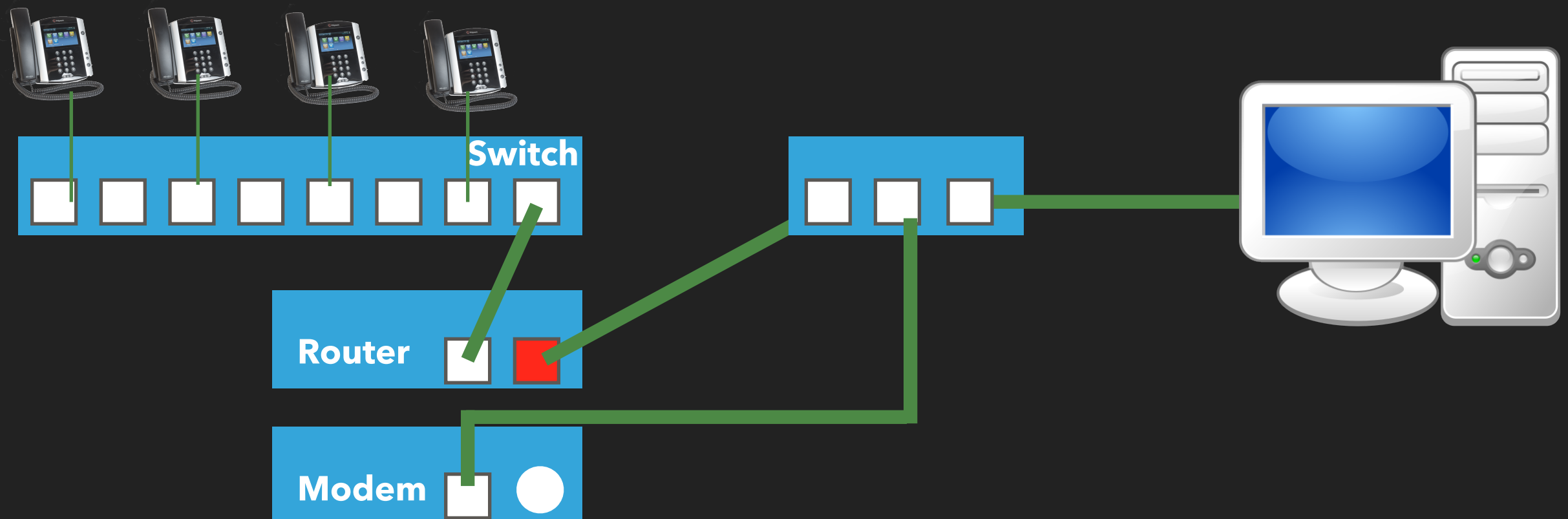
CAPTURING SIP TRAFFIC

- ▶ Placing a HUB in-between switches
- ▶ Capturing LAN traffic



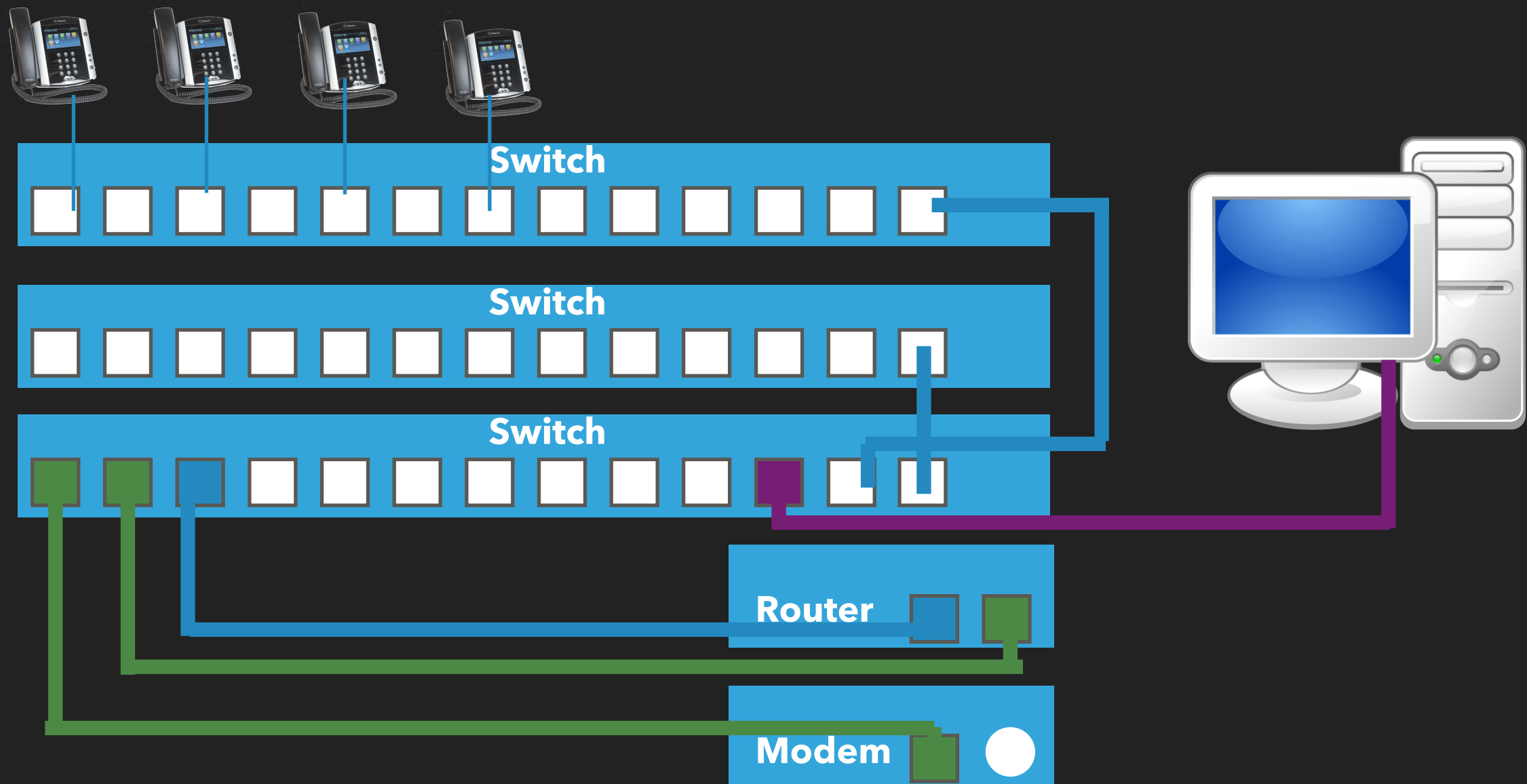
CAPTURING SIP TRAFFIC

- ▶ Placing a HUB in-between switches
- ▶ Capturing WAN traffic



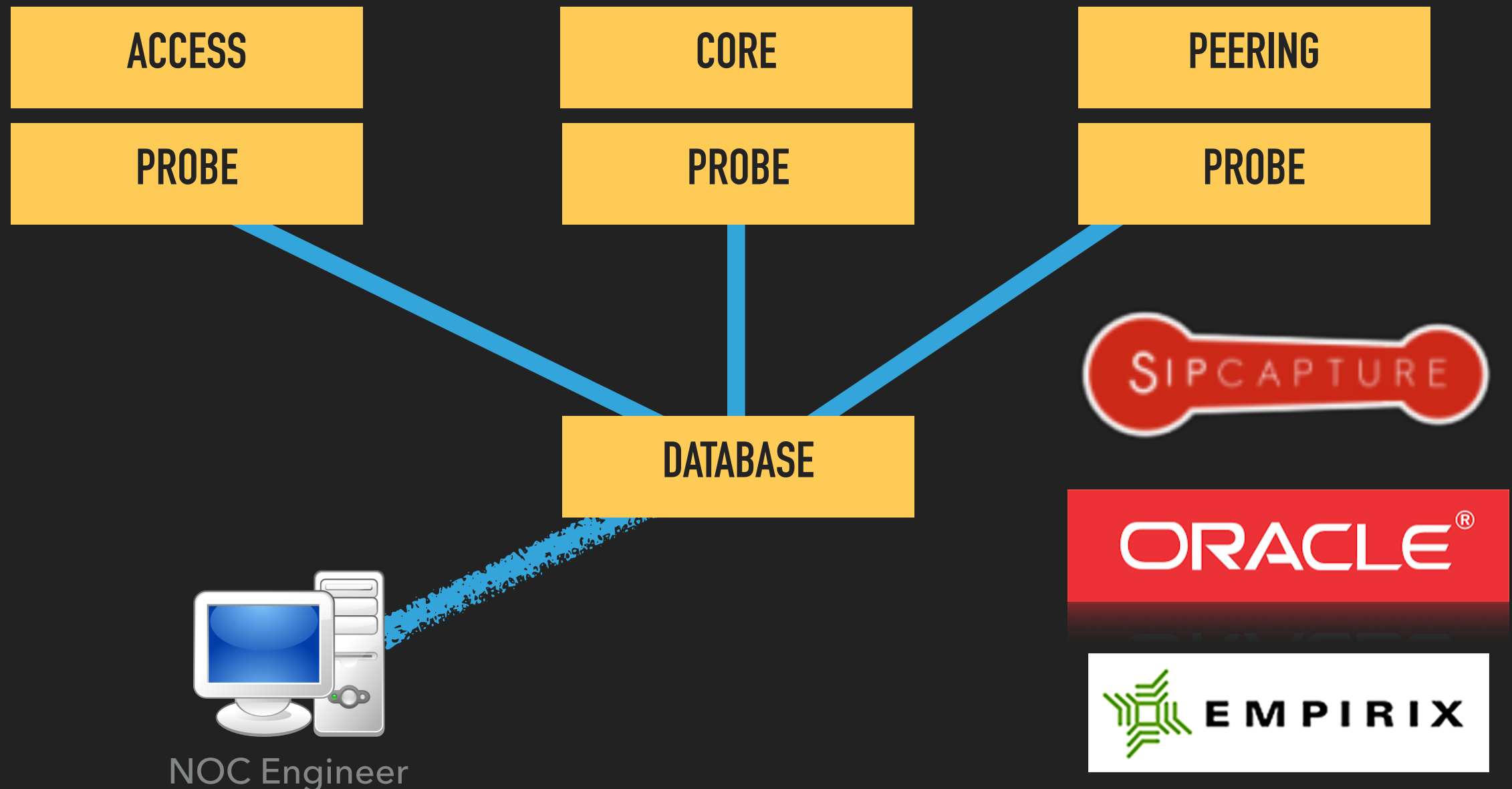
CAPTURING SIP TRAFFIC

- ▶ Using a managed switch
- ▶ Port Mirroring / SPAN



CAPTURING SIP TRAFFIC

- ▶ Using a managed switch
- ▶ Port Mirroring / SPAN + Advanced capturing software



CAPTURING SIP TRAFFIC



<https://www.wireshark.org/download.html>